

Modular models of monoids with operations by lifting functors along fibrations

ZHIXUAN YANG

University of Exeter, United Kingdom
(email: z.yang2@exeter.ac.uk)

NICOLAS WU

Imperial College London, United Kingdom
(email: n.wu@imperial.ac.uk)

Abstract

Inspired by Plotkin and Power’s algebraic treatment of computational effects and the principle of *notions of computations as monoids*, we propose a categorical framework for *equational theories* and *models* of monoids equipped with operations. This framework generalises Plotkin and Power’s algebraic treatment of effectful operations taking or returning *values* as input or output to operations that may take or return *computations* as input or output. Additionally, to give semantic models of computational effects in a modular way, we introduce a formal theory of modular constructions of algebraic structures based on the framework of lifting functors along fibrations.

1 Introduction

In his seminal work, Moggi (1989a,b, 1991) pioneered modelling *notions of computation*, which are now more commonly referred to as *computational effects*, using *monads* and *monad transformers*. The understanding of this approach was later deepened by Plotkin and Power (2001, 2002, 2004), who characterise many monads that model computational effects as arising from *equational theories* of some primitive effectful operations and equational laws characterising their interaction.

In parallel with these developments in denotational semantics, Spivey (1990) independently applied monads to the treatment of exceptions, anticipating their use as a general abstraction for effectful computation. Building on Moggi’s theoretical foundation, Wadler (1990) explored a wider range of examples and played a key role in popularising monads within the functional programming community.

Later, variations of monads were proposed to model more forms of computational effects, including *arrows* (Hughes 2000; Jacobs et al. 2009), *applicative functors* (McBride and Paterson 2008; Paterson 2012), *parameterised monads* (Atkey 2009), *graded monads* (Katsumata 2014). All these notions can be unified in the framework of *monoids in monoidal categories*, leading up to the principle of *notions of computation as monoids* (Pieters et al. 2020; Rivas and Jaskelioff 2017).

Many refinements have been proposed for making programming with effects more *modular*. A fundamental idea is to *separate syntax from semantics*, so syntactic effectful programs can be written without resorting to a specific semantics, and (possibly many kinds of) semantics



© 2026 Copyright held by the owner/author(s).

This work is licensed under a Creative Commons Attribution 4.0 International License.

can be given later. Building on this, there are two further kinds of modularity that are both desirable but are sometimes conflated:

Syntactic modularity is where the syntactic specification of two languages Σ_1 and Σ_2 can be combined into a larger language $\Sigma_1 +_{\text{syn}} \Sigma_2$.

Semantic modularity is where semantic models M_1 and M_2 for the sub-languages can be combined into a model $M_1 +_{\text{sem}} M_2$ of the larger language $\Sigma_1 +_{\text{syn}} \Sigma_2$.

In terms of this classification, *free monads* and their variants provide syntactic modularity (Kiselyov and Ishii 2015; Swierstra 2008). Typeclasses of monads with operations in Haskell (Gill and Kmett 2012; Liang et al. 1995) also achieve syntactic modularity.

On the other hand, *monad transformers* (Jaskelioff and Moggi 2010; Moggi 1989a), *layered monads* (Filinski 1999), and combining monads by coproducts (Ghani and Uustalu 2004) are about semantic modularity.

Effect handlers. Notably, *effect handlers*, introduced by Plotkin and Pretnar (2013) and further developed by many people, achieve both kinds of modularity, which explains their quick adoption. In this approach, effectful operations are described by an algebraic theory (sometimes without any equations), whose free-algebra monad is used as the monad for effectful computations. Effect handlers are (not necessarily free) algebras of this theory, so they can ‘handle’ effectful computations, i.e. the free algebra, using the unique homomorphism out of the free algebra. Algebraic theories and handlers are both composable, so syntactic and semantic modularity are both achieved.

However, effectful operations in this framework *necessarily* fall into a kind of effectful operation known as *algebraic operations* (Plotkin and Power 2003). An algebraic operation on a monad $M : \mathcal{C} \rightarrow \mathcal{C}$ is a morphism $\alpha : A \circ M \rightarrow M$ for an endofunctor $A : \mathcal{C} \rightarrow \mathcal{C}$, typically a polynomial functor, such that it is compatible with monad multiplication:

$$\mu^M \cdot (\alpha \circ M) = \alpha \cdot (A \circ \mu^M) : A \circ M \circ M \rightarrow M.$$

This intuitively says that the operation α commutes with sequential composition.

Not all effectful operations have this property though: for example, exception catching in Haskell $\text{catch} :: IO\ a \rightarrow (Exc \rightarrow IO\ a) \rightarrow IO\ a$ does not satisfy

$$(\text{catch } p\ h) \gg k = \text{catch } (p \gg k) (h \gg k)$$

since the left-hand side only catches the exception in p while the right-hand side catches the exception in p and k . Such non-algebraic operations *can* be programmed as handlers (Plotkin and Pretnar 2013), but they do not have the benefit of (syntactic or semantic) modularity, since as handlers they themselves cannot be handled to give alternative semantics (Wu et al. 2014; Yang et al. 2022).

A line of research seeks to lift the expressivity of effect handlers (Bach Poulsen and van der Rest 2023; Piróg et al. 2018; Wu et al. 2014; Yang et al. 2022) by considering signatures/theories of broader ranges of operations and their models. The leading example is *scoped operations* considered by Piróg et al. (2018), which are operations on monads M of the following form for some $A : \mathcal{C} \rightarrow \mathcal{C}$:

$$s : \int^{X \in \mathcal{C}} A(MX) \times (M-)^X \cong A \circ M \circ M \rightarrow M, \quad (1)$$

where the isomorphism is by the co-Yoneda lemma. Typically, the functor A is $(-)^n$ for some natural number n . The intuition is that s is an operation delimiting n scopes: the coend $\int^X$

is like an existential type $\exists X$, and $A(MX) = (MX)^n$ is the computation inside the scopes, returning some type X , and $(M-)^X$ is the computation after these scopes. For example, the operation of *exception catching* delimits two scopes, one for ‘try’ and the other for ‘catch’, so $A = (-)^2 \cong (- \times -)$.

However, existing work in this direction only considers syntactic modularity (with the exception (Wu et al. 2014)). The root of the difficulty with semantic modularity is that only algebraic operations have a canonical lifting along a monad morphism: given a monad morphism $f : M \rightarrow N$ and an algebraic operation $\alpha : A \circ M \rightarrow M$ on M , there is a unique algebraic operation $\bar{\alpha} : A \circ N \rightarrow N$:

$$\bar{\alpha} = (A \circ N \xrightarrow{A \circ \eta^M \circ N} A \circ M \circ N \xrightarrow{\alpha \circ N} M \circ N \xrightarrow{f \circ N} N \circ N \xrightarrow{\mu^M} N) \quad (2)$$

making $f : M \rightarrow N$ an algebra homomorphism from α to $\bar{\alpha}$. In contrast, if the operation takes the form Equation 1 or more generally the form $\alpha : \Sigma M \rightarrow M$ for a functor $\Sigma : \text{ENDO}(\mathcal{C}) \rightarrow \text{ENDO}(\mathcal{C})$, we do not have a formula to define a meaningful $\bar{\alpha} : \Sigma N \rightarrow N$ from $f : M \rightarrow N$ and $\alpha : \Sigma M \rightarrow M$.

Overview. Motivated by the lack of semantic modularity in existing frameworks, the present paper has two aims/parts: the first one is to develop a unifying account of equational theories of (algebraic and non-algebraic) effectful operations; the second one is to develop a framework of *modular models* that provide semantic modularity for non-algebraic operations. These two parts together achieve both modularities in the style of effect handlers, but for a much wider range of operations.

In the first part (Sections 2 to 6), we start by developing a convenient way of presenting equational theories over monoidal categories by using *equational systems* proposed by Fiore and Hur (2009) and the internal language of monoidal categories, which we call *monoidal algebraic theories*. Then we study *operation families*, which are subcategories of equational theories of *monoids with operations*. Examples include the family of *algebraic operations*, and the family of *scoped operations*, and the family of *variable-binding operations*. Syntactic modularity is achieved by taking colimits in operation families.

In the second part (Sections 7 to 8), we propose a general theory for *modularity in algebraic structures*. The main idea is to define a *modular model* M of an algebraic theory Σ to be a *lifting* of the functor $(- + \Sigma) : \mathcal{T} \rightarrow \mathcal{T}$ along the fibration $P : \mathcal{A} \rightarrow \mathcal{T}$

$$\begin{array}{ccc} \mathcal{A} & \xrightarrow{M} & \mathcal{A} \\ P \downarrow & & \downarrow P \\ \mathcal{T} & \xrightarrow{- + \Sigma} & \mathcal{T} \end{array}$$

where $\mathcal{T}$ is the category of algebraic theories in question and $P : \mathcal{A} \rightarrow \mathcal{T}$ is the fibration given by the Grothendieck construction for $(-)\text{-ALG} : \mathcal{T}^{\text{op}} \rightarrow \text{CAT}$, so $\mathcal{A}$ is the category that contains all models of all theories in $\mathcal{T}$. This idea is then generalised to considering liftings $M : \mathcal{A} \rightarrow \mathcal{A}'$ of an arbitrary functor $T : \mathcal{T} \rightarrow \mathcal{T}'$ along two possibly different fibrations $P : \mathcal{A} \rightarrow \mathcal{T}$ and $P' : \mathcal{A}' \rightarrow \mathcal{T}'$. The informal intuition is that the functor T is a theory $T\Gamma$ parameterised by a generic ‘future extension’ Γ to the theory, and the functor $M : \mathcal{A} \rightarrow \mathcal{A}'$ is then a model of T parameterised by a generic model of Γ , so we call the lifting M a *model transformer* of T . A number of universal constructions and concrete constructions of model transformers are then given.

Example. We sketch a small concrete example here to demonstrate the ideas. Let $\mathcal{E}$ be the monoidal category $(\text{ENDO}_f(\text{SET}), \circ, \text{Id})$ of finitary endofunctors on sets. The equational theory Ec of monads M with *exception catching* has two operations besides those of monads:

$$\text{throw} : 1 \rightarrow M \quad \text{and} \quad \text{catch} : (\text{Id} \times \text{Id}) \circ M \circ M \cong (M \times M) \circ M \rightarrow M,$$

where 1 and $\times$ are respectively the terminal object and the cartesian product in the category $\text{ENDO}_f(\text{SET})$. For now let us ignore equations on these operations. The operation *throw* is for throwing an exception. The operation *catch* is for catching an exception in a scope and handling it, which is an instance of [Equation 1](#) with $A = \text{Id} \times \text{Id}$. As we briefly mentioned earlier, the product $M \times M$ in the signature of *catch* is a pair of computations, one for the program p whose exceptions are caught, the other one for the program h handling the exception; the $\circ M$ after $M \times M$ in the signature of *catch* is an *explicit continuation* argument k . Thus *catch* $(\langle p, h \rangle, k)$ is understood as handling the exception in p using h , and then continues as k . The explicit continuation is exactly the trick to workaround the limitation of algebraic operations (we will say more about it in [section 6](#)).

All theories of *monads with some scoped operations* are collected as a category $\text{SCP}(\mathcal{E})$, which we call the *monoidal theory family of scoped operations* over $\mathcal{E}$, whose arrows are *translations* of theories. The category $\text{SCP}(\mathcal{E})$ has finite coproducts by taking the coproduct of the signatures and equations. Moreover, each theory in $\text{SCP}(\mathcal{E})$ has free algebras, in particular initial algebras. For example, the initial algebra of Ec is the initial one among all *monads with throw and catch*. The carrier of the initial algebra $\mu\text{Ec} : \text{SET} \rightarrow \text{SET}$ can be characterised as the initial solution to

$$X \cong \text{Id} + 1 + (X \times X) \circ X \in \text{ENDO}_f(\text{SET}).$$

The monad μEc models *syntactic programs* with exception throwing and catching.

A (strict) *modular model* of the theory Ec is a family of functors $M_{\tilde{\Sigma}} : \tilde{\Sigma}\text{-ALG} \rightarrow (\tilde{\Sigma} + \text{Ec})\text{-ALG}$, natural in $\tilde{\Sigma} \in \text{SCP}(\mathcal{E})$. Here $(-)\text{-ALG}$ is the functor $\text{SCP}(\mathcal{E}) \rightarrow \text{CAT}$ sending each theory to the category of its models. One possible modular model of Ec is to send every $\tilde{\Sigma}$ -algebra carried by A to $A \circ (1 + \text{Id})$ equipped with operations in $\tilde{\Sigma}$ and Ec . In the same way that there may be many handlers of the same algebraic operation, we also have choices over how *catch* and *throw* act on $A \circ (1 + \text{Id})$. Besides the ‘standard’ semantics (detailed in [Example 8.4*6](#)), we may also have non-standard semantics such as re-trying the program after the handling program is executed, or the semantics that the exceptions thrown by the handling program are recursively handled.

The advantage of a modular model M of Ec over an ordinary model of Ec is that M allows us to interpret syntactic programs $\mu(\tilde{\Sigma} + \text{Ec})$ involving throwing and catching mixed with *any other operations* $\tilde{\Sigma}$ in $\text{SCP}(\mathcal{E})$. By the initiality of $\mu(\tilde{\Sigma} + \text{Ec})$, there is a morphism

$$h : \mu(\tilde{\Sigma} + \text{Ec}) \rightarrow M_{\tilde{\Sigma}}(\mu\tilde{\Sigma}) \quad \text{in} \quad (\tilde{\Sigma} + \text{Ec})\text{-ALG},$$

which interprets Ec but leaves $\tilde{\Sigma}$ -operations uninterpreted. In this way, we achieve syntactic and semantic modularity in the style of effect handlers of algebraic operations, but for the non-algebraic operation *catch*.

Paper organisation. For the first part of the paper,

- in [section 2](#), we review the concept of monoids in monoidal categories and their applications in computational effects;

- in [section 3](#), we develop [Fiore and Hur \(2009\)](#)’s *equational systems*, a convenient framework for presenting algebraic theories. In particular, we study morphisms between equational systems and their colimits, allowing us to construct equational systems by ‘gluing’ smaller ones;
- in [section 4](#), we develop a type theory for describing constructions and equational systems over monoidal categories;
- in [section 5](#), we come back to theories of monoids with operations and see some concrete examples;
- in [section 6](#), we classify operations on monoids into finer families and study the connections between these families.

For the second part of the paper,

- in [section 7](#), we introduce *modular models* of a theory and the more general concept of *model transformers*;
- in [section 8](#), we show general constructions and examples of modular models.

Along these lines, the contribution that we make is a unified general framework for defining both *theories* and *models* of effectful operations in a modular way.

Changes from the conference version. This paper is the extended version of the paper *Modular Models of Monoids with Operations* ([Yang and Wu 2023](#)) presented at the 28th ACM SIGPLAN International Conference on Functional Programming (ICFP 2023). The main differences between this version and the conference version are as follows:

- (1) Proofs omitted in the conference version are included, along with some new theorems, remarks, and examples throughout the paper.
- (2) Definitions and theorems in Sections [2](#) to [6](#) are revised to assume only the existence of the free algebras of the relevant equational systems, instead of assuming chain-cocontinuity of signature functors globally as in the conference version. Consequently, these results apply to small-complete small category, which is useful when working in type theories with impredicative universes.
- (3) The introduction to the metalanguage of monoidal categories is refactored to a separate section, where it is treated more carefully under the name *monoidal algebraic theories*. The main technical change is that in [Section 4.3](#) we identify a class of monoidal algebraic theories corresponding to equational systems, while in the conference version, we used the metalanguage to present the components of equational systems in an ad-hoc way.
- (4) The development of modular models in Sections [7](#) to [8](#) is greatly revised and extended. The ad-hoc definition in the conference version is replaced by a principled definition based on lifting functors along fibrations. Also, morphisms between modular models are now discussed, which allows us to show several new constructions with universal properties and (co)limits of modular models.

Since this extended version has become rather long, we opt in a more fine-grained numbering system to help the reader navigate in the paper more easily: (1) all theorem-like environments share the same counter of the format ‘ABC*XYZ’ where ‘ABC’ is the current section number and ‘XYZ’ is the number of the environment within that section; (2) text paragraphs explaining one idea are grouped together and also numbered by an anonymous

environment; (3) the materials that are new or received significant changes from the conference version are numbered in green, such as Lemma 5*15 and Lemma 6*14 (although the colour of these hyperlinks are still the same as others for consistency).

1.1 Related work

1.1*1 (Effect handlers). The most closely related work is the line of research on *handlers of algebraic effects* introduced by Plotkin and Pretnar (2009, 2013). Semantically, handlers are models of first-order algebraic theories, and are used for interpreting free algebras. As a programming construct, handlers offer a composable approach to user-defined algebraic effects, essentially relying on the fact that algebraic operations can be lifted canonically. Many implementations of handlers have been developed, both as libraries (e.g. Kammar et al. (2013)) and languages (e.g. Bauer and Pretnar (2015)). Our work is inspired by Schrijvers et al. (2019)’s *modular handlers*, which are handlers polymorphic in the unhandled effects.

Handlers of algebraic effects have been generalised in several directions. Wu et al. (2014) observe that implementing *scoped operations* as handlers leads to non-modularity issues, and they propose modelling these operations with higher-order abstract syntax and generalising handlers to this setting. Later, the categorical foundation of handlers of scoped operations were studied by Piróg et al. (2018) and Yang et al. (2022). In another direction, Pieters et al. (2020) generalised handlers from algebraic operations on monads to monoids. In comparison to the present work, Piróg et al. (2018) and Yang et al. (2022) do not consider modularity of models, and Pieters et al. (2020) only consider algebraic operations. Hence our initial motivation of the present work was to develop a clear categorical formulation of equational theories of not necessarily algebraic operations and their modular models.

A notable deviation of our definition of modular models from the standard notion of effect handlers is that our modular models are required to have a monoid structure, rather than only modelling the operations. The distinction is analogous to Arkor and Fiore (2020)’s models of *simply typed syntax* versus *simple type theories*. We believe that our deviation is a reasonable choice since (i) many practical examples of modular handlers have monoid structures anyway; and (ii) many theories of non-algebraic operations inherently involve monoid operations in their equational laws, such as Example 5*17, so a handler for such a theory without a monoid structure makes little sense.

1.1*2 (Monad transformers). Moggi (1989b), Spivey (1990), and Wadler (1990) pioneered using *monads* to model computational effects in programming languages. To achieve modularity, Moggi (1989b) introduced *monad transformers*. Later, Jaskelioff (2009) showed how to lift $\hat{\Sigma}$ -operations along *functorial monad transformers*, and this result was generalised to *monoid transformers* by Jaskelioff and Moggi (2010). Our notion of *modular models* is a conceptual development of monoid transformers: besides transforming monoids into new ones, how to equip them with new operations and lift existing operations are also part of the definition. Thus philosophically, modular models are more like Plotkin and Pretnar (2013)’s handlers, since they are meant to interpret initial algebras of equational theories that model computational effects, rather than modelling the effects themselves.

Liang et al. (1995) introduced *monad transformers* in Haskell to overcome the problem that monads do not straightforwardly compose. Their implementation includes *type classes* of monads with certain operations, which can be seen as a Haskell realisation of algebraic theories, but they need to lift existing operations along monad transformers in ad-hoc ways.

1.1*3 (Theories and syntax). *Algebraic theories* and their connections to *Lawvere theories* and *monads* have been studied for decades (see e.g. (Adamek et al. 2010)). In this paper, we use Fiore and Hur (2009)’s *equational systems* to define equational theories for their generality and simplicity. *Abstract syntax* can be modelled as initial algebras of theories (Goguen et al. 1977), and Fiore et al. (1999) show that abstract syntax with variable bindings can be modelled as initial algebras in a presheaf category, and they introduce Σ -monoids. Building on their work, we have investigated connections between families of theories of Σ -monoids, and their modular models.

Their work leads to the line of research on *second-order algebraic theories* (Fiore 2008; Fiore and Hur 2010; Fiore and Mahmoud 2010; Fiore and Szamozvancev 2022). Ghani et al. (2006) also model the syntax of *explicit substitution* in a functorial category, which Piróg et al. (2018) base on to model scoped operations. In comparison, the focus of the present work is (modular) models of abstract syntax, rather than syntax itself.

Another line of work that is very closely related to Σ -monoids is the work on using *modules* of monads/monoids as the signatures of operations; see e.g. the recent exposition by Lamiaux and Ahrens (2024).

2 Notions of computation as monoids in monoidal categories

2*1. A *monoidal category* is a category $\mathcal{E}$ equipped with a functor $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$, called the *monoidal product* (and sometimes the *tensor product*), an object $I \in \mathcal{E}$, called the *monoidal unit*, and three natural isomorphisms

$$\alpha_{A,B,C} : A \square (B \square C) \cong (A \square B) \square C, \quad \lambda_A : I \square A \cong A, \quad \rho_A : A \square I \cong A$$

satisfying some coherence axioms (Mac Lane 1998, §VII.1) that guarantee there is a unique way using α , λ , and ρ to re-bracket an expression involving I and $\square$ to another (Mac Lane 1998, §VII.2). A monoidal category is *strict* if the isomorphisms are identity morphisms. A monoidal category is (*left*) *closed* if all functors $- \square A$ have right adjoints $- / A : \mathcal{E} \rightarrow \mathcal{E}$.

2*2. A *monoid* $\langle M, \mu, \eta \rangle$ in a monoidal category $\mathcal{E}$ is an object $M \in \mathcal{E}$ with two morphisms: a *multiplication* $\mu : M \square M \rightarrow M$ and a *unit* $\eta : I \rightarrow M$ making the following commute:

$$\begin{array}{ccc} (M \square M) \square M & \xrightarrow{\alpha_{M,M,M}} & M \square (M \square M) \\ \mu \square M \downarrow & & \downarrow M \square \mu \\ M \square M & \xrightarrow{\mu} & M \quad \longleftarrow \mu \quad M \square M \\ \\ I \square M & \xrightarrow{\eta \square M} & M \square M \\ \lambda_M \searrow & & \downarrow \mu \\ & & M \end{array} \qquad \begin{array}{ccc} M \square I & \xrightarrow{M \square \eta} & M \square M \\ \rho_M \downarrow & & \swarrow \mu \\ M & & \end{array}$$

2*3. In the rest of this section, we present a collection of examples of monoidal categories, in which monoids model different flavours of *notions of computations*, and will serve as the main application of the theory developed in this paper. The main messages are that

- (1) monoids in monoidal categories are an expressive abstraction that unifies different notions of computations;
- (2) by imposing suitable conditions, these monoidal categories for computations can be made closed and (co)complete, so are suitable for doing algebraic theories over them.

The purpose of this section is to organise some folklore knowledge about monoids in computing and thereby set the stage for the theory developed later. We give citations but no actual proofs of the claims in this section, and the reader does not need to understand all the details either to proceed to [Section 3](#).

2.1 Monads and size issues

2.1*1 (Monads). The category $\text{ENDO}(\mathcal{C})$ of endofunctors $\mathcal{C} \rightarrow \mathcal{C}$ on a category $\mathcal{C}$ can be turned into a monoidal category by equipping it with functor composition $F \circ G$ as the monoidal product and the identity functor $\text{Id} : \mathcal{C} \rightarrow \mathcal{C}$ as the unit. Monoids in this category are called *monads* on $\mathcal{C}$, and they are used to model *computational effects*, also called *notions of computation*, in programming languages ([Moggi 1989b, 1991](#)), where the unit $\eta : \text{Id} \rightarrow M$ is understood as embedding *pure values* into computations, and the multiplication $\mu : M \circ M \rightarrow M$ is understood as flattening *computations of computations* into computations by sequentially executing them. The understanding of μ as sequential composition is better exhibited by the following co-Yoneda isomorphism:

$$(F \circ G)A = F(GA) \cong \int^{\langle X, f : X \rightarrow GA \rangle \in \mathcal{C}/(GA)} FX$$

where $\int^{\langle X, f : X \rightarrow GA \rangle \in \mathcal{C}/(GA)}$ denotes a coend indexed by the slice category of $\mathcal{C}$ over GA (we do not need any assumption on $\mathcal{C}$ for this coend to exist – the coend is just given by FGA). The informal reading of the coend is that FX is the first computation, returning a value of type X , and the second computation is determined by the result of FX , given as a function $f : X \rightarrow GA$. So $\mu : M \circ M \rightarrow M$ is sequential composition of two computations in which the second is determined by the first one.

2.1*2. However, the category $\text{ENDO}(\mathcal{C})$ is usually not as well behaved as we would like for doing algebraic theories in it, even when $\mathcal{C}$ itself is a very nice category such as SET . In particular, $\text{ENDO}(\text{SET})$ is not closed with respect to either cartesian products or functor composition. Moreover, $\text{ENDO}(\text{SET})$ is not a nice category for doing algebraic theories; for example, some objects in $\text{ENDO}(\text{SET})$, such as the covariant powerset functor $\mathcal{P}$, do not have free monads over them.

These issues about $\text{ENDO}(\text{SET})$ are fundamentally related to *sizes* of sets. For instance, if $\mathcal{P} : \text{SET} \rightarrow \text{SET}$ were to have a free monad F , then F would also be *algebraically free* ([Kelly 1980](#), Proposition 22.4), which entails that $F\emptyset$ would carry the initial $\mathcal{P}$ -algebra. By Lambek’s lemma ([Lambek 1968](#)), we would then have a set $F\emptyset$ satisfying $F\emptyset \cong \mathcal{P}(F\emptyset)$, contradicting Cantor’s theorem ([Cantor 1891](#)).

2.1*3. Since we are dealing with size issues, we should be clear about the foundational assumptions used in this paper. In this paper, we use a traditional set-theoretic language to discuss category theory, so a category $\mathcal{C}$ means a *set* of objects $\text{Obj } \mathcal{C}$ and a family of *sets* $\mathcal{C}(a, b)$ of morphisms for all $a, b \in \text{Obj } \mathcal{C}$. We assume that we have two (arbitrary) Grothendieck universes U_1 and U_2 such that $U_1 \in U_2$. Moreover, U_1 -small sets will just be called *small sets*, and the category of small sets is denoted by SET ; U_1 -small categories will be called *small categories*, and their category will be denoted by CAT . Similarly, U_2 -small categories will be called *large categories*, and their category is denoted by CAT .

2.1*4. Now coming back to the size issues about $\text{ENDO}(\text{SET})$, we have two ways to rectify the size issues: (1) we can consider some ‘extremely nice’ categories $\mathcal{C}$, namely, (*small*-)

complete small categories, or (2) we can restrict our attention to ‘reasonably nice endofunctors’ on a ‘reasonably nice’ category $\mathcal{C}$, namely, κ -accessible functors on *locally κ -presentable categories*. We will describe both approaches below.

2.1*5. When $\mathcal{C}$ is a small category that is also small-complete, the monoidal structure $\langle \circ, \text{Id} \rangle$ on $\text{ENDO}(\mathcal{C})$ is closed, with the right adjoint $-/G$ to $- \circ G$ for every $G : \mathcal{C} \rightarrow \mathcal{C}$ given by *right Kan extension* (Mac Lane 1998, §X):

$$(F/G)A = \int_{B \in \mathcal{C}} \prod_{\text{SET}(A, GB)} FB, \quad (3)$$

which exists since the ‘bound of the end’ $B \in \mathcal{C}$ is small and $\mathcal{C}$ is small-complete. The unit of the adjunction $- \circ G \dashv -/G$ is given by the natural transformations $\eta_F : F \rightarrow (F \circ G)/G$ whose component at every $A \in \mathcal{C}$

$$\eta_{F,A} : FA \rightarrow \int_{B \in \mathcal{C}} \prod_{\text{SET}(A, GB)} F(GB)$$

is the mediating morphism induced by the wedge

$$\eta_{F,A,B} := \langle Fk \rangle_{k \in \text{SET}(A, GB)} : FA \rightarrow \prod_{\text{SET}(A, GB)} F(GB)$$

for all $B \in \mathcal{C}$. The counit ϵ of the adjunction $- \circ G \dashv -/G$ is given by the composite

$$((F/G) \circ G)A = \int_{B \in \mathcal{C}} \prod_{\text{SET}(GA, GB)} FB \xrightarrow{\pi_A} \prod_{\text{SET}(GA, GA)} FA \xrightarrow{\pi_{\text{id}, GA \rightarrow GA}} FA$$

for all $F \in \text{ENDO}(\mathcal{C})$ and $A \in \mathcal{C}$.

Moreover, $\text{ENDO}(\mathcal{C})$ is also a small-complete small category: it is small because its domain and codomain categories $\mathcal{C}$ are both small; it is small-complete because limits in functor categories are computed pointwise and $\mathcal{C}$ is small-complete. We will see later in Theorem 3.1*17 that it implies that all objects in $\text{ENDO}(\mathcal{C})$ have free monads (and many other free algebraic structures).

2.1*6. However, it is long known that in *classical logic* the only examples of small-complete small categories $\mathcal{C}$ are *complete preorders* (Freyd 1964, p. 78). Therefore $\mathcal{C}$ being small and small-complete is classically too strong for the purpose of modelling computational effects, since we certainly want to have more than one program for a type in general.

However, a stunning result in categorical logic is that if we carry out category theory internally in the *effective topos* EFF , or more generally realizability toposes (Hyland 1988; Oosten 2008), the category MSET of *modest sets*, also known as *partial equivalence relations* (PERs), is a non-trivial small-complete small-categories, internally. The effective topos and modest sets find many application in programming language theory: for one, they provide semantics for type theories with *impredicative* polymorphism, such as System F (Girard 1986) and the calculus of inductive constructions (Coquand and Huet 1988; Luo 1994) with an impredicative universe of sets, which is the type theory underlying the Rocq (previously known as Coq) proof assistant with the `-impredicative-set` option.

2.2 Finitary and accessible monads

2.2*1. We have seen the approach of rectifying the size issues in $\text{ENDO}(\mathcal{C})$ by assuming an ‘extremely nice’ $\mathcal{C}$ that exists only in non-classical settings. Now let us describe an alternative, classically valid, approach – restricting our attention to ‘reasonably nice’ endofunctors.

First we observe that the reason why the formula (3) does not work for arbitrary endofunctors $F, G : \text{SET} \rightarrow \text{SET}$ is that the bound of the end $B \in \text{SET}$ would be large while SET is only

small-complete. Therefore we can consider functors that allow us to cut $B \in \mathbf{SET}$ to a small bound. An endofunctor $F \in \mathbf{ENDO}(\mathbf{SET})$ is called *finitary* if it preserves *filtered colimits*. An informal description of finitariness is that *we lose no information if we restrict F to finite sets*. Precisely, F is finitary if we first restrict F to $F \circ V : \mathbf{FIN} \rightarrow \mathbf{SET}$ on the full subcategory of finite sets, where $V : \mathbf{FIN} \rightarrow \mathbf{SET}$ is the inclusion functor, and then take the left Kan extension of $F \circ V$ along V , the resulting functor is still isomorphic to F . In other words, we have the following equivalence, where $\mathbf{ENDO}_f(\mathbf{SET}) \subseteq \mathbf{ENDO}(\mathbf{SET})$ denotes the full subcategory of finitary endofunctors:

$$\mathbf{ENDO}_f(\mathbf{SET}) \begin{array}{c} \xleftarrow{\text{LAN}_V -} \\ \xrightarrow[-\circ V]{\cong} \end{array} \mathbf{SET}^{\mathbf{FIN}} \quad (4)$$

2.2*2. The category $\mathbf{ENDO}_f(\mathbf{SET})$ inherits the monoidal structure $\langle \circ, \text{Id} \rangle$ of $\mathbf{ENDO}(\mathbf{SET})$, which under the equivalence (4) corresponds to the monoidal structure $\langle \bullet, V \rangle$ on $\mathbf{SET}^{\mathbf{FIN}}$ (Fiore et al. 1999; Kelly and Power 1993) where

$$Vn = n \quad \text{and} \quad (F \bullet G)n = \int_{m \in \mathbf{FIN}} Fm \times (Gn)^m. \quad (5)$$

For every $G \in \mathbf{SET}^{\mathbf{FIN}}$, the functor $-\bullet G$ has a right adjoint

$$(F/G)n = \int_{m \in \mathbf{FIN}} \prod_{\mathbf{SET}(n, Gm)} Fm$$

with unit and counit similar to those in 2.1*5. The end $\int_{m \in \mathbf{FIN}}$ exists because the category $\mathbf{FIN}$ of finite sets is essentially small and $\mathbf{SET}$ is small complete.

Moreover, the functor $\circ : \mathbf{ENDO}_f(\mathbf{SET}) \times \mathbf{ENDO}_f(\mathbf{SET}) \rightarrow \mathbf{ENDO}_f(\mathbf{SET})$ is also finitary, i.e. preserving filtered colimits in $\mathbf{ENDO}_f(\mathbf{SET}) \times \mathbf{ENDO}_f(\mathbf{SET})$. This is equivalent to preserving filtered colimits in both of its arguments separately: functor composition preserves (all) colimits in the first argument

$$((\text{colim}_i F_i) \circ G)n = (\text{colim}_i F_i)(Gn) \cong \text{colim}_i (F_i(Gn))$$

because colimits of functors can be computed pointwise. It preserves filtered colimits in the second argument by the finitariness of its first argument:

$$(F \circ (\text{colim}_i G_i))n = F((\text{colim}_i G_i)n) \cong F(\text{colim}_i G_i n) \cong \text{colim}_i F(G_i n)$$

In particular, the diagram of an ω -chain is filtered, so functor composition preserves colimits of ω -chains of functors; we will use this property to construct free monads over finitary endofunctors in Section 3.1.

2.2*3. Monoids in $\langle \mathbf{ENDO}_f(\mathbf{SET}), \circ, \text{Id} \rangle$ are called *finitary monads* on $\mathbf{SET}$, and they are equivalent to (finitary) *Lawvere theories* [Power 1999, Theorem 4.2; Adamek et al. 2010, Theorem A.38] and the closely related *abstract clones* [Cohn 1981, page 132; Fiore et al. 1999, Section 3]. Computational effects modelled by Lawvere theories are usually called *algebraic effects* (Plotkin and Power 2002, 2004).

Apart from modelling computational effects, a related but slightly different application of monoids in $\mathbf{ENDO}_f(\mathbf{SET})$, or equivalently $\mathbf{SET}^{\mathbf{FIN}}$, is modelling *abstract syntax with variable binding* (Fiore et al. 1999; Fiore and Szamozvancev 2022). In this case, a monoid $M \in \mathbf{SET}^{\mathbf{FIN}}$ is understood as a family of sets of terms indexed by the number of *variables* in the context. The monoid unit $V \rightarrow M$ is then embedding *variables* as M -terms, and the monoid multiplication $M \bullet M \rightarrow M$ is *simultaneous substitution* of terms for variables.

Yet another interesting reading of monoids of $\langle \bullet, V \rangle$ due to [Fiore and Staton \(2014\)](#) is computations supporting (i) binding a piece of code to a code pointer and (ii) jumping to a code pointer. Based on this reading, the monoidal category $\langle \text{SET}^{\text{FIN}}, \bullet, V \rangle$ provides an adequate denotational semantics of a calculus of substitution/jumps, on which algebraic effects can be encoded.

2.2*4. The equivalence $\text{ENDO}_f(\text{SET}) \cong \text{SET}^{\text{FIN}}$ can be generalised to endofunctors on categories other than SET : we can replace (1) SET with any *locally κ -presentable* (lkp) category $\mathcal{C}$ for a regular cardinal κ , (2) FIN with the subcategory $\mathcal{C}_\kappa$ of *κ -presentable objects* in $\mathcal{C}$, and (3) finitary functors with *κ -accessible* functors $\text{ENDO}_\kappa(\mathcal{C})$ ([Adámek and Rosicky 1994](#)). This results in a cocomplete closed monoidal category $\langle \text{ENDO}_\kappa(\mathcal{C}), \circ, \text{Id} \rangle$, on which $\circ$ is κ -accessible. The κ -accessibility of $\circ$ implies that there is a sufficiently large limit ordinal β such that $\circ : \text{ENDO}_\kappa(\mathcal{C}) \times \text{ENDO}_\kappa(\mathcal{C}) \rightarrow \text{ENDO}_\kappa(\mathcal{C})$ preserves colimits of all β -chains, a property we will later use for constructing free monoids.

2.2*5. All lkp categories are necessarily (small-) complete and cocomplete, and they cover a wide range of categories that are used for modelling programming languages. Examples of lkp categories include:

- all presheaf categories $\text{SET}^{\mathcal{D}}$ for small categories $\mathcal{D}$ and $\kappa = \aleph_0$ (an lkp category is called locally *finitely* presentable when $\kappa = \aleph_0$), and more generally all Grothendieck toposes for some κ ([Borceux 1994b](#), 3.4.16);
- categories of models for *essentially algebraic theories* ([Adámek and Rosicky 1994](#), §3.D), which include the category CAT of small categories for $\kappa = \aleph_0$, the category ωCPO of ω -complete partial orders for $\kappa = \aleph_1$ (however, the category of *directed* complete partial orders is not lkp for any κ);
- functor categories $\mathcal{C}^{\mathcal{D}}$ for small categories $\mathcal{D}$ and lkp $\mathcal{C}$;
- moreover, when $\mathcal{C}$ is lkp, it is automatically λlp for any $\lambda > \kappa$.

Therefore, lkp categories provide a nice setting for algebraic theories in the context of programming language semantics.

2.3 Strong monads

2.3*1. The multiplication $\mu : M \circ M \rightarrow M$ of a monad $M : \mathcal{C} \rightarrow \mathcal{C}$ allows one to compose two effectful computations $f : A \rightarrow MB$ and $g : B \rightarrow MC$ by

$$A \xrightarrow{f} MB \xrightarrow{Mg} M(MC) \xrightarrow{\mu_C} MC.$$

However, to give semantics to effectful programming languages with a *structural* context of variables as done by [Moggi \(1989b, 1991\)](#), what we need is slightly stronger: for all objects $\Gamma \in \mathcal{C}$ (thought of as variable contexts) and morphisms $f : \Gamma \times A \rightarrow MB$ and $g : \Gamma \times B \rightarrow MC$ (two computations under a context Γ), we would like to have a morphism $\Gamma \times A \rightarrow MC$ (the sequential composition of f and g). The structure of monads $\langle M, \mu, \eta \rangle$ is not sufficient for doing this, and we need additionally a natural transformation

$$s_{\Gamma, B} : \Gamma \times MB \rightarrow M(\Gamma \times B),$$

with which we have the composite

$$\Gamma \times A \xrightarrow{\langle \pi_1, f \rangle} \Gamma \times MB \xrightarrow{s_{\Gamma, B}} M(\Gamma \times B) \xrightarrow{Mg} M(MC) \xrightarrow{\mu_C} MC.$$

To make this way of composing effectful computations associative and the pure computation $(\eta_A \cdot \pi_2) : \Gamma \times A \rightarrow MA$ an identity, the natural transformation s must satisfy certain coherence conditions (see (Moggi 1991, Definition 3.2)). The natural transformation s is called a *strength* for the monad M , and the tuple $\langle M, \mu, \eta, s \rangle$ is called a *strong monad*.

2.3*2. Strong monads are monoids in the monoidal category of *strong endofunctors*. A strong endofunctor $\langle F, s \rangle$ on a category $\mathcal{C}$ with finite products is a functor $F : \mathcal{C} \rightarrow \mathcal{C}$ with a natural transformation $s_{\Gamma, B} : \Gamma \times FB \rightarrow F(\Gamma \times B)$ making the following commute:

$$\begin{array}{ccc} 1 \times FB & & \\ s_{1, B} \downarrow & \searrow \lambda_{FB} & \\ F(1 \times B) & \xrightarrow{F\lambda_B} & FB \end{array} \quad (6)$$

$$\begin{array}{ccc} (\Gamma' \times \Gamma) \times FB & \xrightarrow{\alpha_{\Gamma', \Gamma, FB}} & \Gamma' \times (\Gamma \times FB) \xrightarrow{\Gamma' \times s_{\Gamma, B}} \Gamma' \times F(\Gamma \times B) \\ s_{\Gamma' \times \Gamma, B} \downarrow & & \downarrow s_{\Gamma', \Gamma \times B} \\ F((\Gamma' \times \Gamma) \times B) & \xrightarrow{F\alpha_{\Gamma', \Gamma, B}} & F(\Gamma' \times (\Gamma \times B)) \end{array} \quad (7)$$

where λ and α are the left unitor and associator for the cartesian monoidal structure $\langle \times, 1 \rangle$. Moreover, *strong natural transformations* between $\langle F, s^F \rangle$ and $\langle G, s^G \rangle$ are natural transformations $\tau : F \rightarrow G$ making the following commute:

$$\begin{array}{ccc} \Gamma \times FB & \xrightarrow{\Gamma \times \tau_B} & \Gamma \times GB \\ s_{\Gamma, B}^F \downarrow & & \downarrow s_{\Gamma, B}^G \\ F(\Gamma \times B) & \xrightarrow{\tau_{\Gamma \times B}} & G(\Gamma \times B) \end{array}$$

2.3*3. Strong endofunctors on $\mathcal{C}$ and strong natural transformations can be collected into a category $\text{ENDO}_s(\mathcal{C})$. The category $\text{ENDO}_s(\mathcal{C})$ has a monoidal structure $\langle \circ_s, \text{Id}_s \rangle$ where Id_s is the identity functor equipped with the identity strength, and $\circ_s$ is the composition of strong functors given by

$$\langle F, s^G \rangle \circ_s \langle G, s^F \rangle = \langle F \circ G, (Fs_{\Gamma, B}^G \cdot s_{\Gamma, GB}^F)_{\Gamma, B \in \mathcal{C}} \rangle.$$

Strong monads on $\mathcal{C}$ are precisely monoids in this monoidal category. When $\mathcal{C}$ is a cartesian *closed* category, strong functors and natural transformations are the same as $\mathcal{C}$ -*enriched* functors and natural transformations (Kock 1972; McDermott and Uustalu 2022b).

2.3*4. When $\mathcal{C}$ is SET , or slightly more generally a full subcategory of SET closed under finite products of SET , every $F : \mathcal{C} \rightarrow \mathcal{C}$ has a unique strength (McDermott and Uustalu 2022b):

$$\begin{aligned} s_{\Gamma, B} &: \Gamma \times FB \rightarrow F(\Gamma \times B) \\ s_{\Gamma, B} \langle \gamma, f \rangle &= F(\lambda b. \langle \gamma, b \rangle) f \end{aligned} \quad (8)$$

However, functors on a general category $\mathcal{C}$ may have no or non-unique strengths.

2.3*5. Similar to the setting of ordinary monads, to have a ‘nice’ monoidal category of strong monads, we need to either (1) consider small-complete small categories $\mathcal{C}$ or (2) κ -accessible strong monads. More precisely, denote by $\text{ENDO}_{s\kappa}(\mathcal{C})$ the full subcategory of $\text{ENDO}_s(\mathcal{C})$ that contains strong functors whose underlying functors are κ -accessible.

When $\mathcal{C}$ is *lcp as a cartesian closed category*, which means that $\mathcal{C}$ is lcp and cartesian closed, and that the κ -presentable objects of $\mathcal{C}$ are closed under finite products, the category $\text{ENDO}_{s\kappa}(\mathcal{C})$ is also lcp and has a closed monoidal structure of functor composition and the identity functor. The primary example of such setting is $\mathcal{C} = \omega\text{Cpo}$ for modelling general recursion. We refer the reader to [Kelly and Power \(1993, §4\)](#) and [Kelly \(1982\)](#) for details.

2.4 Graded monads

2.4*1. Another generalisation of monads is to index the monad with some *grades* that track quantitative information about the effects performed by a computation ([Katsumata 2014](#); [Katsumata et al. 2022](#); [McDermott and Uustalu 2022a](#)). Precisely, let $\langle \mathcal{G}, \cdot, 1 \rangle$ be any small monoidal category, whose objects we call grades. A $\mathcal{G}$ -*graded monad* on a category $\mathcal{C}$ is a functor $M : \mathcal{G} \rightarrow \text{ENDO}(\mathcal{C})$ equipped with natural transformations

$$\eta : \text{Id} \rightarrow M1 \quad \mu_{a,b} : (Ma) \circ (Mb) \rightarrow M(a \cdot b) \quad (9)$$

natural in $a, b \in \mathcal{G}$, satisfying laws similar to those of monads ([Katsumata 2014](#)).

2.4*2. For example, for tracking operations performed by a computation, $\mathcal{G}$ can be the poset of sets of operation names, ordered by inclusion, with the monoidal structure $1 = \emptyset$ and $a \cdot b = a \cup b$. And for tracking the number of nondeterministic choices made by a computation, $\mathcal{G}$ can be the poset $\langle \mathbb{N}, \leq \rangle$ with monoidal structure $\langle 0, + \rangle$.

2.4*3. Again, to avoid the size issues when doing algebraic theories ([2.1*2](#)), we can either require $\mathcal{C}$ to be small and small-complete or consider only κ -*accessible $\mathcal{G}$ -graded monads* $M : \mathcal{G} \rightarrow \text{ENDO}_{\kappa}(\mathcal{C})$ for lcp $\mathcal{C}$.

2.4*4. Similar to the ungraded situation, κ -accessible graded monads are equivalent to monoids in the functor category $\text{ENDO}_{\kappa}(\mathcal{C})^{\mathcal{G}}$ equipped with the following monoidal structure similar to the Day tensor:

$$I = \coprod_{\mathcal{G}(1, -)} \text{Id} \quad F * G = \int^{a, b \in \mathcal{G}} \coprod_{\mathcal{G}(a \cdot b, -)} (Fa \circ Gb). \quad (10)$$

This monoidal product has right adjoints given by

$$(G \multimap F)a = \int_{b \in \mathcal{G}} \int_{m \in \mathcal{C}_{\kappa}} \prod_{\mathcal{C}(-, (Gb)m)} F(a \cdot b)m.$$

2.4*5. We have seen monads and several variations – realizable, accessible, strong, graded – all formulated as monoids in monoidal categories. Despite their differences, they all model notions computations that support sequential compositions in a sense. In the following, let us have a look at some other monoids that are conceptually quite different from monads.

2.5 Cartesian monoids

2.5*1. Every cartesian category $\mathcal{C}$, i.e. a category with finite products, can be equipped with the product $\times$ as the monoidal product and the terminal object $1 \in \mathcal{C}$ as the monoidal unit. When $\mathcal{C}$ has all exponentials B^A , $\mathcal{C}$ is then a cartesian closed category. Particularly, the category SET is a closed monoidal category in this way. Monoids in SET are precisely the usual notion of monoids, such as the set of lists with concatenation and empty list.

2.5*2. For lcp and cartesian closed $\mathcal{C}$, the category $\text{ENDO}_{\kappa}(\mathcal{C})$ is cartesian closed. The cartesian unit and product in $\text{ENDO}_{\kappa}(\mathcal{C})$ are defined pointwise:

$$1n = 1_{\mathcal{C}} \quad (F \times G)n = Fn \times Gn$$

The exponential is given by

$$(F^G)n = \int_{m \in \mathcal{C}_k} \prod_{\mathcal{C}(n,m)} (Fm)^{Gm}.$$

A computational interpretation of cartesian monoids in $\text{ENDO}_k(\mathcal{C})$ is that they model *notions of independent computations*: cartesian multiplication $M \times M \rightarrow M$ composes two computations that have no dependency, whereas monadic multiplication $M \circ M \rightarrow M$ composes a computation with another that depends on the result of the former.

2.6 Applicative functors

2.6*1. Between the two extremes of $M \circ M$ and $M \times M$, there are monoidal structures on $\text{ENDO}_k(\mathcal{C})$ that allow computations to have restricted dependency. One of them is the Day convolution (Day 1970) induced by cartesian products: the Day monoidal structure on $\text{ENDO}_k(\text{SET})$ has as unit the identity functor, and the monoidal product is given by the following coend formula:

$$(F * G)n = \int^{m,k \in \text{SET}_k \times \text{SET}_k} Fm \times Gk \times n^{(m \times k)}. \quad (11)$$

Informally, $F * G$ models two computations Fm and Gm that are *almost independent* except that their return values are combined by a pure function $n^{(m \times k)}$. This structure is symmetric and closed, with the right adjoint to $- * G$ given by

$$G \multimap F = \int_{n \in \text{SET}_k} (F(- \times n))^{Gn}.$$

More generally, we can replace SET with any *lcp as a cartesian closed category* $\mathcal{V}$ (Kelly 1982) and also the coend in (11) with a $\mathcal{V}$ -enriched coend, which allows us to give a more accurate formulation of applicatives in functional languages with general recursion by setting $\mathcal{V} = \omega\text{CPO}$ (the (co)ends above continue to exist after we generalise from SET to $\mathcal{V}$, because an lcp category is always small-complete and small-cocomplete). Alternatively, we can consider small-complete small $\mathcal{C}$ to work around the size issues as in 2.1*5.

2.6*2. Monoids for $\langle *, \text{Id} \rangle$ are called *applicative functors* or simply *applicatives*. For the case of $\langle \text{SET}, *, \text{Id} \rangle$, applicative functors are equivalent to *lax monoidal functors* $\langle \text{SET}, \times, 1 \rangle \rightarrow \langle \text{SET}, \times, 1 \rangle$ (McBride and Paterson 2008; Paterson 2012). A practical application of them is *build systems* (Mokhov et al. 2018), since usually the result of a building task does not affect what the next building task is.

2.6*3. Applicatives are a weaker notion than strong monads, as intuitively independence is a special case of dependence. Precisely, for any two functors $F, G \in \text{ENDO}_k(\text{SET})$, there are canonical strengths (8):

$$s_{XY}^F : FX \times Y \rightarrow F(X \times Y), \quad s_{XY}^G : GX \times Y \rightarrow G(X \times Y),$$

and then there is a natural transformation $e : F * G \rightarrow F \circ G$ as follows:

$$\begin{aligned} (F * G)n &= \int^{m,k} Fm \times Gk \times n^{(m \times k)} \rightarrow \int^{m,k} F(G(m \times k \times n^{(m \times k)})) \\ &\rightarrow \int^{m,k} F(Gn) \cong F(Gn) \end{aligned}$$

where the first arrow is repeated uses of the strengths of F and G , and the second arrow is function evaluation. Consequently, for any monad $\langle M, \mu, \eta \rangle$ on SET , it induces an applicative functor with unit η and multiplication

$$M * M \xrightarrow{e} M \circ M \xrightarrow{\mu} M.$$

However, there are many applicative functors that are not obtained from monads in this way (McBride and Paterson 2008; Paterson 2012).

3 Equational systems and translations

3*1. We have seen monoids in various monoidal categories. In practice, monoids usually come with more operations in addition to multiplication and the unit. For example, the state monad $(- \times S)^S$ comes with operations for reading and writing the state; the exception monad $- + E$ has operations for throwing and catching exceptions; the ordinary monoid in $\mathbf{SET}$ of lists with concatenation has the operation of appending an element to a list.

Therefore, we shall have a way to talk about algebraic theories in monoidal categories systematically. In this paper, we will use Fiore and Hur's [2007; 2009] *equational systems*, a simple but powerful framework subsuming (enriched) algebraic theories. Importantly, the model theory of equational systems is well developed: Fiore and Hur established conditions for the existence of free algebras, cocompleteness of the category of models, and monadicity.

3*2. In the following, we first briefly review equational systems and Fiore and Hur's theorem for the existence of free algebras of equational systems. The new contributions of this paper are the following: we show a constructive proof of the theorem for small-complete small categories (Section 3.1). We then introduce *functorial translations* between equational systems, making them a category, and discuss some basic properties of this category (Sections 3.2 and 3.3).

3.1 Equational systems

3.1*1. Generally speaking, an algebraic theory consists of the *signature* and *equations* of its operations. A concise way to specify a signature over a category $\mathcal{C}$ is just a functor $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$, and then a Σ -*algebra* is an object $A \in \mathcal{C}$, called the *carrier*, together with a *structure map* $\alpha : \Sigma A \rightarrow A$. For example, the signature functor of the *theory of monoids* in a monoidal category $\mathcal{E}$ with binary coproducts is $\Sigma_{\mathbf{MON}} = (- \sqcup -) + I$. A $\Sigma_{\mathbf{MON}}$ -algebra $\langle A, \alpha \rangle$ is an object A with a morphism $\alpha : (A \sqcup A) + I \rightarrow A$, or equivalently two morphisms $A \sqcup A \rightarrow A$ and $I \rightarrow A$.

3.1*2. We denote the category of Σ -algebras by $\Sigma\text{-ALG}$, whose morphisms from $\langle A, \alpha \rangle$ to $\langle B, \beta \rangle$ are *algebra homomorphisms*, i.e. morphisms $h : A \rightarrow B$ in $\mathcal{C}$ such that $h \cdot \alpha = \beta \cdot \Sigma h : \Sigma A \rightarrow B$. The forgetful functor dropping the structure map is denoted by $U_\Sigma : \Sigma\text{-ALG} \rightarrow \mathcal{C}$ or just U when it is not ambiguous.

3.1*3. Equations on a signature $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$ are typically presented as commutative diagrams, such as the diagrams in 2*2 for monoids. Pictorially, a diagram looks like the following:

$$\begin{array}{ccc} \Gamma A & \xrightarrow{\quad R\alpha \quad} & A \\ & \searrow L\alpha & \downarrow \\ & & A \end{array}$$

which contains a pair of paths $L\alpha$ and $R\alpha$ from some formal object ΓA to some formal object A , parameterised by a formal morphism $\alpha : \Sigma A \rightarrow A$ of the operation. The starting node ΓA can be called the *context* of the diagram.

A categorical way to make precise such as a diagram is a functor $\Gamma : \mathcal{C} \rightarrow \mathcal{C}$ and a pair of functors $L, R : \Sigma\text{-ALG} \rightarrow \Gamma\text{-ALG}$. For example, let $\langle \mathcal{E}, \sqcup, I \rangle$ be a monoidal category and $\Sigma = (- \sqcup -) : \mathcal{E} \rightarrow \mathcal{E}$ be the signature of a binary operation. To encode the associativity

diagram in 2*2 for Σ -algebras, we let the functor $\Gamma : \mathcal{E} \rightarrow \mathcal{E}$ be $(-\square -) \square -$, and the two paths are represented by two functors $L, R : (-\square -)\text{-ALG} \rightarrow ((-\square -) \square -)\text{-ALG}$:

$$\begin{aligned} L\langle A, \mu : A \square A \rightarrow A \rangle &= \langle A, \mu \cdot (\mu \square A) \rangle \\ R\langle A, \mu : A \square A \rightarrow A \rangle &= \langle A, \mu \cdot (A \square \mu) \cdot \alpha_{A,A,A} \rangle \end{aligned}$$

The functors $L, R : \Sigma\text{-ALG} \rightarrow \Gamma\text{-ALG}$ must satisfy $U_\Gamma \circ L = U_\Sigma$ and $U_\Gamma \circ R = U_\Sigma$.

3.1*4 Definition (Fiore and Hur (2009)). An *equational system* on a category $\mathcal{C}$

$$\dot{\Sigma} := (\Sigma \triangleright \Gamma \vdash L = R)$$

consists of four functors: a (functorial) *signature* $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$, a (functorial) *context* $\Gamma : \mathcal{C} \rightarrow \mathcal{C}$, and a pair of two (functorial) *terms* $L, R : \Sigma\text{-ALG} \rightarrow \Gamma\text{-ALG}$ making the following diagrams commute:

$$\begin{array}{ccc} \Sigma\text{-ALG} & \xrightarrow{L} & \Gamma\text{-ALG} \\ & \searrow U_\Sigma & \swarrow U_\Gamma \\ & \mathcal{C} & \end{array} \quad \begin{array}{ccc} \Sigma\text{-ALG} & \xrightarrow{R} & \Gamma\text{-ALG} \\ & \searrow U_\Sigma & \swarrow U_\Gamma \\ & \mathcal{C} & \end{array} \quad (12)$$

An *algebra*, or a *model*, of $\dot{\Sigma}$ is a Σ -algebra $\langle A \in \mathcal{C}, \alpha : \Sigma A \rightarrow A \rangle$ such that

$$L\langle A, \alpha \rangle = R\langle A, \alpha \rangle \in \Gamma\text{-ALG}.$$

We denote by $\dot{\Sigma}\text{-ALG}$ the full subcategory of $\Sigma\text{-ALG}$ containing $\dot{\Sigma}$ -algebras.

3.1*5 Notation. In the definition above, the functors $L, R : \Sigma\text{-ALG} \rightarrow \Gamma\text{-ALG}$ always send objects $\langle A, \alpha : \Sigma A \rightarrow A \rangle$ to $\langle A, \beta : \Gamma A \rightarrow A \rangle$, keeping carriers A unchanged, so we will simply write $L\alpha : \Gamma A \rightarrow A$ to mean $\pi_2(L\langle A, \alpha \rangle)$.

3.1*6 Example. Let $\mathcal{E}$ be a monoidal category with binary coproducts. The concept of monoids in $\mathcal{E}$ (2*2) can be defined as an equational system on $\mathcal{E}$

$$\text{MON} := (\Sigma_{\text{MON}} \triangleright \Gamma_{\text{MON}} \vdash L_{\text{MON}} = R_{\text{MON}}) \quad (13)$$

with functorial signature and contexts

$$\begin{aligned} \Sigma_{\text{MON}} M &= (M \square M) + I \\ \Gamma_{\text{MON}} M &= ((M \square M) \square M) + (I \square M) + (M \square I), \end{aligned}$$

and $L_{\text{MON}}, R_{\text{MON}} : \Sigma_{\text{MON}}\text{-ALG} \rightarrow \Gamma_{\text{MON}}\text{-ALG}$ given by

$$L_{\text{MON}} \beta = [L_1, L_2, L_3] \quad R_{\text{MON}} \beta = [R_1, R_2, R_3],$$

where we define $\mu := (\beta \cdot \iota_1) : M \square M \rightarrow M$, $\eta := (\beta \cdot \iota_2) : I \rightarrow M$ and

$$\begin{aligned} L_1 &:= \mu \cdot (\mu \square M) & R_1 &:= \mu \cdot (M \square \mu) \cdot \alpha_{M,M,M} \\ L_2 &:= \mu \cdot (\eta \square M) & R_2 &:= \lambda_M \\ L_3 &:= \mu \cdot (M \square \eta) & R_3 &:= \rho_M \end{aligned}$$

Each pair L_i and R_i encodes a commutative diagram for monoids in 2*2. An algebra of this equational system is precisely a monoid in $\mathcal{E}$.

3.1*7 Notation. From the example above we see that although the definition of equational systems has exactly one operation and one equation, multiple operations/equations can be encoded via coproducts. We will say *an equational system with a set of operations* $\{\Sigma_o : \mathcal{C} \rightarrow \mathcal{C}\}_{o \in O}$ and *a set of equations* $\{\Gamma_e \vdash L_e = R_e\}_{e \in E}$, where $\Gamma_e : \mathcal{C} \rightarrow \mathcal{C}$ and $L_e, R_e : (\coprod_{o \in O} \Sigma_o)\text{-ALG} \rightarrow \Gamma_e\text{-ALG}$, provided that $\mathcal{C}$ has the required indexed coproducts.

Given an equational system $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$ on a category $\mathcal{C}$ with binary coproducts, we denote by $\dot{\Sigma} \dot{\vdash} \Sigma'$ the extension of $\dot{\Sigma}$ with a new operation of signature $\Sigma' : \mathcal{C} \rightarrow \mathcal{C}$:

$$\dot{\Sigma} \dot{\vdash} \Sigma' := (\Sigma + \Sigma' \triangleright \Gamma \vdash L \circ U = R \circ U)$$

where $U : (\Sigma + \Sigma')\text{-ALG} \rightarrow \Sigma\text{-ALG}$ is the forgetful functor dropping Σ' operations. Similarly, we denote by $\dot{\Sigma} \dot{\vdash} (\Gamma' \vdash L' = R')$ the extension of $\dot{\Sigma}$ with a new equation:

$$\dot{\Sigma} \dot{\vdash} (\Gamma' \vdash L' = R') := (\Sigma \triangleright \Gamma + \Gamma' \vdash [L, L'] = [R, R']),$$

where $[L, L'] : \Sigma\text{-ALG} \rightarrow (\Gamma + \Gamma')\text{-ALG}$ is the functor mapping $\alpha : \Sigma X \rightarrow X$ to $[L\alpha, L'\alpha] : (\Gamma + \Gamma')X \rightarrow X$, and $[R, R']$ is similar.

3.1*8 Example. The concept of Eilenberg-Moore algebras of a monad can be defined as an equational system. Let M be a monad on some $\mathcal{C}$ with coproducts. The equational system $M\text{-ALG}$ has as signature M itself and two equations: $\text{Id}_{\mathcal{C}} \vdash L_1 = R_1$ and $M \circ M \vdash L_2 = R_2$ where for all $\langle X \in \mathcal{C}, \alpha : MX \rightarrow X \rangle$,

$$\begin{aligned} L_1\alpha &= (X \xrightarrow{\eta_X} MX \xrightarrow{\alpha} X) & R_1\alpha &= \text{id}_X \\ L_2\alpha &= (M(MX) \xrightarrow{\mu_X} MX \xrightarrow{\alpha} X) & R_2\alpha &= (M(MX) \xrightarrow{M\alpha} MX \xrightarrow{\alpha} X) \end{aligned}$$

The category of algebras for this equational system is precisely the Eilenberg-Moore category of the monad M .

3.1*9. Among the algebras of an equational system $\dot{\Sigma}$ over $\mathcal{C}$, the *free algebras* are particularly useful since they represent *abstract syntax* – quotients of terms built from variables and operations of the theory. The abstract syntax can be *interpreted* with another model using the free-forgetful adjunction:

$$\phi : \mathcal{C}(X, U\langle A, \alpha \rangle) \cong \dot{\Sigma}\text{-ALG}(F X, \langle A, \alpha \rangle)$$

Given any model $\langle A, \alpha \rangle$ of $\dot{\Sigma}$ and $g : X \rightarrow A = U\langle A, \alpha \rangle$, the morphism $\phi(g) : F X \rightarrow \langle A, \alpha \rangle$ interprets the free algebra with the semantic model $\langle A, \alpha \rangle$, with generators X interpreted by $g : X \rightarrow A$. [Fiore and Hur \(2009\)](#) showed several sufficient conditions for the existence of free algebras, which we record below.

3.1*10 Theorem (Fiore and Hur (2009)). *Let $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$ be an equational system over $\mathcal{C}$. If $\mathcal{C}$ is (small-) cocomplete and one of the following holds:*

- Σ and Γ preserve colimits of β -chains for a limit ordinal β ;
- Σ preserves colimits of β -chains for a limit ordinal β , and both Σ and Γ preserve epimorphisms in $\mathcal{C}$;
- Σ preserves colimits of β -chains for a limit ordinal β , and Σ preserves epimorphisms, and $\mathcal{C}$ has no transfinite chains of proper epimorphisms,

then there are left adjoints to the inclusion functor $\dot{\Sigma}\text{-ALG} \hookrightarrow \Sigma\text{-ALG}$ and the forgetful functor $\Sigma\text{-ALG} \rightarrow \mathcal{C}$:

$$\dot{\Sigma}\text{-ALG} \xrightleftharpoons[\perp]{\perp} \Sigma\text{-ALG} \xrightleftharpoons[\perp]{\perp} \mathcal{C} \quad (14)$$

Moreover, $\dot{\Sigma}\text{-ALG}$ is cocomplete and the composite of the adjunction is monadic.

3.1*11 Notation. We denote the composite adjunction of (14) by $F_{\dot{\Sigma}} \dashv U_{\dot{\Sigma}}$, or $F \dashv U$ when $\dot{\Sigma}$ is clear from context. Moreover, the initial $\dot{\Sigma}$ -algebra is denoted by $\langle \mu\dot{\Sigma}, \alpha^{\dot{\Sigma}} : \Sigma\mu\dot{\Sigma} \rightarrow \mu\dot{\Sigma} \rangle$.

3.1*12. Fiore and Hur's proof of this result is quite technical, but we will not rely on the specifics of their construction. For concreteness, we provide some basic intuition here: the free Σ -algebra on some $A \in \mathcal{C}$ is first constructed by a transfinite iteration of $A + \Sigma-$ on 0

$$0 \xrightarrow{!} A + \Sigma 0 \xrightarrow{A+\Sigma!} A + \Sigma(A + \Sigma 0) \longrightarrow \dots$$

and taking colimits for limit ordinals. The iteration will stop at some $X \cong A + \Sigma X$ in β steps, giving the carrier of the free Σ -algebra (Adámek 1974). Then it is quotiented by the equation $L = R$ and the congruence rule, using Fiore and Hur's *algebraic coequalisers*. The quotienting may also need to be repeated β times when Σ or Γ does not preserve epimorphisms. The result of quotienting is the free $\dot{\Sigma}$ -algebra.

3.1*13 Example. Let $\mathcal{E}$ be a small-cocomplete monoidal category such that $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$ preserves β -chains for some limit ordinal β . For example, $\mathcal{E}$ can be $\langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$ for an lkp $\mathcal{C}$ from Section 2.2 or $\langle \text{ENDO}_{\kappa}(\text{SET}), *, \text{Id} \rangle$ from Section 2.6. Then the first condition of Theorem 3.1*10 is applicable to the equational system MON from Example 3.1*6, so every $A \in \mathcal{E}$ has a free monoid.

Moreover, Fiore and Hur (2009) showed that when $\mathcal{E}$ is left closed (i.e. for all $A \in \mathcal{E}$, $- \square A$ has a right adjoint), there is a simple formula for free monoids: for every $A \in \mathcal{E}$, the free monoid over A is the initial algebra $\mu X. I + A \square X$ (sometimes called the *list object* for A) equipped with appropriate monoid operations. This formula is useful in practice: when $\mathcal{E}$ is $\langle \text{SET}, \times, 1 \rangle$, it is exactly the usual definition $\mu X. 1 + A \times X$ of lists of A -elements; and when $\mathcal{E}$ is $\langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$ or $\langle \text{ENDO}_{\kappa}(\text{SET}), *, \text{Id} \rangle$, this gives formulas for free monads and free applicatives that are suitable for implementation (Rivas and Jaskielioff 2017). Note that this formula needs closedness to work: to define the monoid multiplication

$$(\mu X. I + A \square X) \square (\mu X. I + A \square X) \rightarrow (\mu X. I + A \square X),$$

we need closedness to shift one $(\mu X. I + A \square X)$ to the right-hand side

$$(\mu X. I + A \square X) \rightarrow (\mu X. I + A \square X) / (\mu X. I + A \square X)$$

and then we can use the universal property of the initial algebra.

3.1*14 Remark. When $\mathcal{E}$ is not closed, the initial algebra $\mu X. I + A \square X$ may not be the free monoid over A . For a counterexample, let $\mathcal{E}$ be the cartesian monoidal category $\langle \text{MON}, \times, 1 \rangle$ of monoids in $\langle \text{SET}, \times, 1 \rangle$. By the Eckmann-Hilton argument, a monoid object in the monoidal category $\langle \text{MON}, \times, 1 \rangle$ is precisely an ordinary commutative monoid in sets (so the *structure* of a monoid object degenerates into a *property* in this case). The free monoid object over some $M \in \text{MON}$ is then obtained by quotienting M with commutativity.

On the other hand, the initial algebra $\mu X. I + M \times X$ in MON is the monoid whose elements are finite lists $\langle m_1, m_2, \dots, m_n \rangle$ of M -elements, considered up to trailing zeros, by which we mean $\langle m_1, m_2, \dots, m_n \rangle$ and $\langle m_1, m_2, \dots, m_n, e, \dots, e \rangle$ are considered the same, where e is

the unit element of M . Its unit element is the empty sequence $\langle \rangle$, and its multiplication is the *pairwise* multiplication using the multiplication of M (after padding with enough trailing zeros):

$$\langle m_1, m_2, \dots, m_n \rangle \cdot \langle m'_1, m'_2, \dots, m'_n \rangle = \langle m_1 \cdot_M m'_1, \dots, m_n \cdot_M m'_n \rangle. \quad (15)$$

The associated $(1 + M \times -)$ -algebra is still given by the empty list and *cons* as usual. Now we observe that the multiplication (15) is not commutative when M is not commutative, so the initial algebra $\mu X. 1 + M \times X$ may not be a monoid object in $\langle \mathbf{Mon}, \times, 1 \rangle$, let alone the free monoid object over M .

3.1*15. An appealing aspect of Theorem 3.1*10 is that the base category $\mathcal{C}$ is only required to be cocomplete rather than locally κ -presentable. Therefore the theorem applies to the category $\mathbf{dCPO}$ of *directed complete partial orders*, generalising the construction of free dcpo-algebras for operations of finite arities and (in)equations (Abramsky and Jung 1995, Theorem 6.1.2). Allowing an endofunctor $\Sigma : \mathbf{dCPO} \rightarrow \mathbf{dCPO}$ as the signature comes in handy: in particular, it allows us to express operations that take an ascending chain $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \dots$ as arguments by choosing $\Sigma := (-)^\omega$ where $\omega := \{0 \sqsubseteq 1 \sqsubseteq 2 \sqsubseteq \dots\}$. This functor preserves colimits of all Ω -chains, where Ω is the first uncountable ordinal.

3.1*16. We do not know if any of the sufficient conditions for the existence of free algebras given by Theorem 3.1*10 (or possibly some of their classically equivalent conditions) is constructively true, and in particular, whether they are applicable to small-complete small categories in realizability toposes (2.1*6). Anyway, we have an alternative constructive proof for small-complete small categories using *impredicative encodings* (Awodey et al. 2018), a refinement of the well known *Church encodings* of inductive datatypes.

Remarkably, the theorem below imposes no constraints on the functorial signature Σ and context Γ , generalising the result that every endofunctor on a small-complete small category has an initial algebra (Hyland 1988, §3.1).

3.1*17 Theorem. *Let $\mathcal{C}$ be a small-complete small category. For every equational system $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$ over $\mathcal{C}$, there is a monadic adjunction $F_{\dot{\Sigma}} \dashv U_{\dot{\Sigma}}$. Moreover, the category $\dot{\Sigma}\text{-ALG}$ is a small-complete-and-cocomplete small category.*

Proof. An easy proof goes by first noticing that $\dot{\Sigma}\text{-ALG}$ is also a small-complete small category for all equational systems $\dot{\Sigma}\text{-ALG}$ over $\mathcal{C}$, and then the free algebra of $\dot{\Sigma}$ over an object $X \in \mathcal{C}$ can be constructed from the initial algebra of $\dot{\Sigma} \upharpoonright K_X$, where $K_X : \mathcal{C} \rightarrow \mathcal{C}$ is the constant functor mapping to X . The initial algebra of $\dot{\Sigma} \upharpoonright K_X$ can then be constructed as the limit of the identity functor $\text{Id} : (\dot{\Sigma} \upharpoonright K_X)\text{-ALG} \rightarrow (\dot{\Sigma} \upharpoonright K_X)\text{-ALG}$ (Mac Lane 1998, §X.1 Lemma 1), which exists since $(\dot{\Sigma} \upharpoonright K_X)\text{-ALG}$ is a small-complete small category. $\square$

3.1*18. Theorem 3.1*10 and Theorem 3.1*17 give different sufficient conditions for the existence of free algebras of equational systems, and there are some other constructive techniques such as the one by Fiore et al. (2022). In this paper, we only rely on the existence of free algebras but not those specific conditions guaranteeing the existence of free algebras. The following definition will be used for abstracting over those different conditions.

3.1*19 Definition. Let $\mathcal{C}$ be a category. A relation $\mathcal{A} \subseteq \mathbf{Endo}(\mathcal{C}) \times \mathbf{Endo}(\mathcal{C})$ of endofunctors is called a *freeness condition* if for all equational systems $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$ with $\langle \Sigma, \Gamma \rangle \in \mathcal{A}$, the forgetful functor $U_{\dot{\Sigma}} : \dot{\Sigma}\text{-ALG} \rightarrow \mathcal{C}$ has a left adjoint.

3.1*20 Example. (1) If $\mathcal{C}$ is lkp then the relation $\text{ENDO}_\kappa(\mathcal{C}) \times \text{ENDO}_\kappa(\mathcal{C})$ containing all pairs of κ -accessible endofunctors (Section 2.2) is a freeness condition by the first item of Theorem 3.1*10. (2) If $\mathcal{C}$ is small-complete and small, the entire relation $\text{ENDO}(\mathcal{C}) \times \text{ENDO}(\mathcal{C})$ is a freeness condition (2.1*5) by Theorem 3.1*17. (3) For every $\mathcal{C}$, there is always the largest freeness condition $\mathcal{F}$ containing all pairs $\langle \Sigma, \Gamma \rangle$ such that all equational systems with signature Σ and context Γ have the free-forgetful adjunction. However, $\mathcal{F}$ is less useful than it may appear: we may not know whether $\mathcal{F}$ has good closure properties, for example, whether $\langle \Sigma + \Sigma', \Gamma + \Gamma' \rangle$ is in $\mathcal{F}$ when $\langle \Sigma, \Gamma \rangle$ and $\langle \Sigma', \Gamma' \rangle$ are in $\mathcal{F}$.

3.2 Functorial translations

3.2*1. Morphisms between equational systems are not studied by Fiore and Hur (2007, 2009), but we need them later for talking about combinations of equational systems. A natural idea for morphisms from an equational system $\dot{\Sigma}$ to another $\dot{\Psi}$ is *translations*, which map operations in $\dot{\Sigma}$ to *terms* of $\dot{\Psi}$, preserving equations in a suitable sense. However, a technical difficulty is that equational systems $\dot{\Psi}$ may not have terms, i.e. free algebras.

In the following, we avoid this by introducing a more indirect definition which we call *functorial translations* between equational systems. For some motivation, consider Lawvere theories: every morphism $T : L \rightarrow L'$ between Lawvere theories induces a functor $- \circ T : L'\text{-MOD} \rightarrow L\text{-MOD}$ between their category of models from the opposite direction. Moreover this functor commutes with the forgetful functors from models of L and L' to sets:

$$\begin{array}{ccc} L\text{-MOD} & \xleftarrow{- \circ T} & L'\text{-MOD} \\ & \searrow U_L \quad \swarrow U_{L'} & \\ & \text{SET} & \end{array}$$

This mapping from morphisms T between Lawvere theories to functors $- \circ T$ between the categories of models (from the opposite direction) that commute with U_L and $U_{L'}$ is in fact an equivalence of categories (Adamek et al. 2010, 11.38). Mimicking this situation, we will define morphisms between equational systems also as functors between their categories of models from the opposite direction that commute with the forgetful functors.

From a more informal viewpoint, this is justified by the fact that if we can ‘translate’ operations of an equational system Σ to terms of operations of another equational system Σ' , then given a model of Σ' , we can construct a model of Σ .

3.2*2. In the rest of this section, we fix a category $\mathcal{C}$ and a freeness condition $\mathcal{A} \subseteq \text{ENDO}(\mathcal{C}) \times \text{ENDO}(\mathcal{C})$ of endofunctors (Definition 3.1*19).

3.2*3 Definition. A *functorial translation* of equational systems on $\mathcal{C}$ from $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$ to $\dot{\Sigma}' = (\Sigma' \triangleright \Gamma' \vdash L' = R')$ is a functor $T : \dot{\Sigma}'\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ such that $U_{\dot{\Sigma}} \circ T = U_{\dot{\Sigma}'}$, where $U_{\dot{\Sigma}} : \dot{\Sigma}\text{-ALG} \rightarrow \mathcal{C}$ and $U_{\dot{\Sigma}'} : \dot{\Sigma}'\text{-ALG} \rightarrow \mathcal{C}$ are the forgetful functors.

Equational systems on $\mathcal{C}$ and translations form a category $\text{Eqs}(\mathcal{C})$ whose identity morphisms are the identity functors $\dot{\Sigma}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ and composition of translations is functor composition. Moreover, we denote by $\text{Eqs}_{\mathcal{A}}(\mathcal{C})$ the full subcategory of $\text{Eqs}(\mathcal{C})$ (Definition 3.2*3) containing equational systems whose functorial signature and context satisfy a freeness condition $\mathcal{A}$ (Definition 3.1*19).

3.2*4 Example. Let $\mathcal{C}$ be a category with finite coproducts and products. The theory GRP of groups over $\mathcal{C}$ is the theory MON of monoids in $\langle \mathcal{C}, \times, 1 \rangle$ from Example 3.1*6 extended

with a new operation i with signature $\text{Id} : \mathcal{C} \rightarrow \mathcal{C}$ and two new equations $\text{Id} \vdash L_1 = R_1$ and $\text{Id} \vdash L_2 = R_2$ where

$$\begin{aligned} L_1 \langle M, \eta, \mu, i \rangle &= \langle M, M \xrightarrow{\langle \text{id}_M, i \rangle} M \times M \xrightarrow{\mu} M \rangle \\ R_1 \langle M, \eta, \mu, i \rangle &= \langle M, M \xrightarrow{!} 1 \xrightarrow{\eta} M \rangle \\ L_2 \langle M, \eta, \mu, i \rangle &= \langle M, M \xrightarrow{\langle i, \text{id}_M \rangle} M \times M \xrightarrow{\mu} M \rangle \\ R_2 \langle M, \eta, \mu, i \rangle &= \langle M, M \xrightarrow{!} 1 \xrightarrow{\eta} M \rangle \end{aligned}$$

Then there is a translation $T : \text{MON} \rightarrow \text{GRP}$ that maps every $\langle M, \eta, \mu, i \rangle$ in GRP-ALG to an object $\langle M, \eta, \mu \rangle$ in MON-ALG by forgetting the newly added operation. We call translations like $T : \text{MON} \rightarrow \text{GRP}$ that simply forget some operations and equations *inclusion translations*.

3.2*5. For equational systems with free-algebras, the following lemma shows that functorial translations coincide with the expected notion of translations: maps between monads.

3.2*6 Lemma. Let $\dot{\Sigma}, \dot{\Psi} \in \text{Eqs}_{\mathcal{A}}(\mathcal{C})$. There is a bijection between functorial translations $T : \dot{\Sigma} \rightarrow \dot{\Psi}$ and monad morphisms $m : U_{\dot{\Sigma}}F_{\dot{\Sigma}} \rightarrow U_{\dot{\Psi}}F_{\dot{\Psi}}$.

Proof. By [Fiore and Hur \(2009, Proposition 6.4\)](#), the free-forgetful adjunction for an equational system is always monadic, so functorial translations $T : \dot{\Sigma} \rightarrow \dot{\Psi}$, i.e. functors $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ such that $U_{\dot{\Sigma}} \circ T = U_{\dot{\Psi}}$, are in bijection with functors $S : \mathcal{C}^{U_{\dot{\Psi}}F_{\dot{\Psi}}} \rightarrow \mathcal{C}^{U_{\dot{\Sigma}}F_{\dot{\Sigma}}}$ between the corresponding Eilenberg-Moore categories that commute with the forgetful functors. By [Borceux \(1994a, Proposition 4.5.9\)](#), those functors S are in bijection with monad morphisms $m : U_{\dot{\Sigma}}F_{\dot{\Sigma}} \rightarrow U_{\dot{\Psi}}F_{\dot{\Psi}}$. $\square$

3.2*7 Corollary. The two functorial terms $L, R : \Sigma\text{-ALG} \rightarrow \Gamma\text{-ALG}$ of an equational system can also be viewed as translations between the equational systems of signatures Γ and Σ without equations. Therefore when Σ and Γ have free algebras, L and R are in bijection with two monad morphisms $U_{\Gamma}F_{\Gamma} \rightarrow U_{\Sigma}F_{\Sigma}$. The monad $U_{\Gamma}F_{\Gamma}$ for free Γ -algebras is also the free monad over the endofunctor Γ ([Kelly 1980, Proposition 22.2](#)). In this case, monad morphisms $U_{\Gamma}F_{\Gamma} \rightarrow U_{\Sigma}F_{\Sigma}$ are in bijection with natural transformations $\Gamma \rightarrow U_{\Sigma}F_{\Sigma}$. Moreover, it can be checked that a Σ -algebra $\alpha : \Sigma A \rightarrow A$ satisfies a functorial equation $\Gamma \vdash L = R$ if and only if the following commutes:

$$\Gamma A \begin{array}{c} \xrightarrow{\tilde{L}_A} \\ \xrightarrow{\tilde{R}_A} \end{array} U_{\Sigma}F_{\Sigma}A \xrightarrow{U_{\Sigma}\epsilon_{\langle A, \alpha \rangle}} A$$

where $\tilde{L}$ and $\tilde{R}$ are the natural transformations corresponding to L and R .

3.2*8 Theorem. Let $\mathcal{C}$ be a category with binary coproducts and $\text{Eqs}_f(\mathcal{C}) \subseteq \text{Eqs}(\mathcal{C})$ be the full subcategory containing all equational systems admitting the free-forgetful adjunction. Then we have an equivalence of categories

$$\text{Eqs}_f(\mathcal{C}) \cong \text{MND}(\mathcal{C}),$$

where $\text{MND}(\mathcal{C})$ is the category of monads over $\mathcal{C}$ and monad morphisms.

Proof. Example 3.1*8 shows that every monad M induces an equational system $M\text{-ALG}$ whose category of algebras is precisely the Eilenberg-Moore category of M . Thus the equational system $M\text{-ALG}$ is in $\text{Eqs}_f(\mathcal{C})$ since free Eilenberg-Moore algebras always exist

(which are simply $\langle X, \mu_X : M(MX) \rightarrow MX \rangle$ for all $X \in \mathcal{C}$). By Lemma 3.2*6 above, this construction extends to a fully faithful functor $\mathbf{MND}(\mathcal{C}) \rightarrow \mathbf{EQS}_f(\mathcal{C})$. Moreover, the forgetful functor for every equational system $\dot{\Sigma} \in \mathbf{EQS}_f(\mathcal{C})$ is monadic (Fiore and Hur 2009, Proposition 6.4), so $\dot{\Sigma}$ is isomorphic to the equational system $(F_{\dot{\Sigma}} U_{\dot{\Sigma}})\text{-ALG}$. Therefore we have an essentially surjective fully faithful functor $\mathbf{MND}(\mathcal{C}) \rightarrow \mathbf{EQS}_f(\mathcal{C})$, and thus an equivalence $\mathbf{MND}(\mathcal{C}) \cong \mathbf{EQS}_f(\mathcal{C})$. $\square$

3.2*9. Below, we turn our attention to the property of translations $T : \dot{\Sigma} \rightarrow \dot{\Psi}$ as functors $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$, and show a condition for T to have left adjoints. We start with an interesting lemma saying that (when $\dot{\Psi}$ has free algebras) translations $T : \dot{\Sigma} \rightarrow \dot{\Psi}$ can be extended in a canonical way to translations $\tilde{T} : \Sigma \rightarrow \Psi$ between the respective equational systems *without equations*. Note, however, such an extension may not be unique: when the equation of $\dot{\Psi}$ is inconsistent in the sense that $\dot{\Psi}$ only has the trivial model 1, there is a unique trivial T , but there can still be different translations $\Sigma \rightarrow \Psi$.

3.2*10 Lemma. Let $\mathcal{C}$ be a category and $\dot{\Sigma}, \dot{\Psi} \in \mathbf{EQS}(\mathcal{C})$ such that $\dot{\Psi}$ has the free-forgetful adjunction. Every functorial translation $\dot{\Sigma} \rightarrow \dot{\Psi}$, i.e. a functor $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ such that $U_{\dot{\Sigma}} \circ T = U_{\dot{\Psi}}$ can be extended to a functor $\tilde{T} : \Psi\text{-ALG} \rightarrow \Sigma\text{-ALG}$ such that $U_{\Sigma} \circ \tilde{T} = U_{\Psi}$, and $\tilde{T}$ coincides with T on $\dot{\Psi}\text{-ALG}$.

Proof. Given any Ψ -algebra $\langle A, \alpha : \Psi A \rightarrow A \rangle$, there is a free $\dot{\Psi}$ -algebra over A with structure map $o : \Psi(F_{\dot{\Psi}} A) \rightarrow F_{\dot{\Psi}} A$. It is mapped by T to a $\dot{\Sigma}$ -algebra $To : \Sigma(F_{\dot{\Psi}} A) \rightarrow F_{\dot{\Psi}} A$. We now define the functor $\tilde{T} : \Psi\text{-ALG} \rightarrow \Sigma\text{-ALG}$ by

$$\tilde{T}\langle A, \alpha \rangle = \langle A, \Sigma A \xrightarrow{\Sigma \eta_A} \Sigma(F_{\dot{\Psi}} A) \xrightarrow{To} F_{\dot{\Psi}} A \xrightarrow{\epsilon_{\langle A, \alpha \rangle}} A \rangle.$$

To see that $\tilde{T}$ restricts to T on $\dot{\Psi}\text{-ALG}$, supposing $\langle A, \alpha \rangle \in \dot{\Psi}$, T maps the counit $\epsilon_{\langle A, \alpha \rangle} : \langle F_{\dot{\Psi}} A, o \rangle \rightarrow \langle A, \alpha \rangle$ to the following commutative diagram:

$$\begin{array}{ccc} \Sigma(F_{\dot{\Psi}} A) & \xrightarrow{To} & F_{\dot{\Psi}} A \\ \Sigma \epsilon_{\langle A, \alpha \rangle} \downarrow & & \downarrow \epsilon_{\langle A, \alpha \rangle} \\ \Sigma A & \xrightarrow{T\alpha} & A \end{array}$$

Therefore $\tilde{T}\alpha = \epsilon_{\langle A, \alpha \rangle} \cdot To \cdot \Sigma \eta_A = T\alpha \cdot \Sigma \epsilon \cdot \Sigma \eta_A = T\alpha$ since $\epsilon_{\langle A, \alpha \rangle} \cdot \eta_A = id$. $\square$

3.2*11. In the adjunction $F_{\dot{\Sigma}} \dashv U_{\dot{\Sigma}} : \dot{\Sigma}\text{-ALG} \rightarrow \mathcal{C}$ for free algebras, the category $\mathcal{C}$ can be viewed as the category $\emptyset\text{-ALG}$ for the empty theory $\emptyset$ with no operations or equations, and $U_{\dot{\Sigma}} : \dot{\Sigma}\text{-ALG} \rightarrow \emptyset\text{-ALG}$ is the unique translation from $\emptyset$ to $\dot{\Sigma}$. The fact that $U_{\dot{\Sigma}}$ always has a left adjoint can be generalised to any functorial translations, giving us *relative free algebras*.

3.2*12 Definition. Letting J be a set, we say that the freeness condition $\mathcal{A}$ is *closed under J -indexed coproducts* if whenever $\langle \Sigma_j, \Gamma_j \rangle \in \mathcal{A}$ for all $j \in J$ then $\coprod_j \Sigma_j$ and $\coprod_j \Gamma_j$ exist and are related by $\mathcal{A}$. Similarly, $\mathcal{A}$ is said to be *closed under constant functors* if $\langle K_A, K_B \rangle \in \mathcal{A}$ for all $A, B \in \mathcal{C}$.

3.2*13 Theorem. Assuming that $\mathcal{A}$ is closed under binary coproducts and constant functors, every functorial translation $T : \dot{\Sigma} \rightarrow \dot{\Psi}$ in $\mathbf{EQS}_{\mathcal{A}}(\mathcal{C})$ as a functor $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$

has a left adjoint F and the adjunction is monadic:

$$\begin{array}{ccc}
 \dot{\Sigma}\text{-ALG} & \xrightleftharpoons[\quad]{\quad} & \dot{\Psi}\text{-ALG} \\
 \uparrow \scriptstyle U_{\dot{\Sigma}} \quad \downarrow \scriptstyle F_{\dot{\Sigma}} & \begin{array}{c} \xrightarrow{F} \\ \perp \\ \xleftarrow{T} \end{array} & \uparrow \scriptstyle F_{\dot{\Psi}} \quad \downarrow \scriptstyle U_{\dot{\Psi}} \\
 \mathcal{C} & \xlongequal{\quad} & \mathcal{C}
 \end{array}$$

Proof. The idea of the proof is to construct the left adjoint F via the initial algebra of some other equational system, similarly to how the free algebra for a functor $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$ over an object $A \in \mathcal{C}$ can be constructed via the initial algebra of the functor $A + \Sigma -$, except that we need to take equations into account.

Let $\dot{\Sigma} = (\Sigma \triangleright \Gamma \vdash L = R)$. To construct the free $\dot{\Psi}$ -algebra over a $\dot{\Sigma}$ -algebra $\langle A, \alpha : \Sigma A \rightarrow A \rangle$, we consider the equational system

$$\dot{\Psi}_A := \dot{\Psi} \upharpoonright K_A \upharpoonright (K_{\Sigma A} \vdash L' = R') \quad (16)$$

where K_X is the constant endofunctor mapping to $X \in \mathcal{C}$, and the two functorial terms $L', R' : (\Psi + K_A)\text{-ALG} \rightarrow K_{\Sigma A}\text{-ALG}$ are

$$\begin{aligned}
 L'\langle B, \beta : \Psi B \rightarrow B, i : A \rightarrow B \rangle &= \langle B, \Sigma A \xrightarrow{\Sigma i} \Sigma B \xrightarrow{\tilde{T}\beta} B \rangle, \\
 R'\langle B, \beta : \Psi B \rightarrow B, i : A \rightarrow B \rangle &= \langle B, \Sigma A \xrightarrow{\alpha} A \xrightarrow{i} B \rangle.
 \end{aligned}$$

where $\tilde{T} : \Psi\text{-ALG} \rightarrow \Sigma\text{-ALG}$ is obtained from T by Lemma 3.2*10. Since $\mathcal{A}$ is assumed to be closed under binary coproducts and constant functors, the equational system $\dot{\Psi}_A$ is in $\text{Eqs}_{\mathcal{A}}(\mathcal{C})$ and thus has an initial algebra

$$\langle B_0, \beta : \Psi B_0 \rightarrow B_0, i : A \rightarrow B_0 \rangle.$$

We note that the functorial equation $L' = R'$ encodes a Σ -algebra homomorphism:

$$\begin{array}{ccc}
 \Sigma A & \xrightarrow{\Sigma i} & \Sigma B \\
 \alpha \downarrow & & \downarrow \tilde{T}\beta \\
 A & \xrightarrow{i} & B
 \end{array} \quad (17)$$

Since $\tilde{T}$ and T coincide on $\dot{\Psi}$ algebras, $\tilde{T}\beta$ is the same as $T\beta$. Therefore $i : A \rightarrow B_0$ is a $\dot{\Sigma}$ -algebra homomorphism from $\langle A, \alpha \rangle$ to $\langle B_0, T\beta \rangle$.

Next we show that the arrow $i : \langle A, \alpha \rangle \rightarrow T\langle B_0, \beta \rangle$ is a universal arrow from $\langle A, \alpha \rangle$ to the functor $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$. For every $\langle C, \delta \rangle \in \dot{\Psi}\text{-ALG}$ and an arrow $f : \langle A, \alpha \rangle \rightarrow \langle C, T\delta \rangle$, we need to find a unique $\dot{\Psi}$ -homomorphism $h : \langle B_0, \beta \rangle \rightarrow \langle C, \delta \rangle$ such that $Th \cdot i = f$:

$$\begin{array}{ccc}
 \langle A, \alpha \rangle & \xrightarrow{i} & T\langle B_0, \beta \rangle \\
 & \searrow f & \downarrow Th \\
 & & T\langle C, \delta \rangle
 \end{array}
 \qquad
 \begin{array}{ccc}
 \langle B_0, \beta \rangle & & \\
 \downarrow \exists! h & & \\
 \langle C, \delta \rangle & &
 \end{array}$$

We observe that $\langle C, \delta, f \rangle$ is a model of $\dot{\Psi}_A$ (16), so by the initiality of $\langle B_0, \beta, i \rangle$, there is an $h : \langle B_0, \beta, i \rangle \rightarrow \langle C, \delta, f \rangle$. Since h is a $\dot{\Psi}_A$ -homomorphism, we have $h \cdot i = f$. Hence $Th \cdot i = f$ as the translation T preserves homomorphisms.

It remains to show the uniqueness of such $h : \langle B_0, \beta \rangle \rightarrow \langle C, \delta \rangle \in \dot{\Psi}\text{-ALG}$ with $h \cdot i = f$. Assuming there is such an h' , then h' is also a $\dot{\Psi}_A$ -homomorphism from $\langle B_0, \beta, i \rangle$ to $\langle C, \delta, i \rangle$. Therefore $h' = h$ by the initiality of $\langle B_0, \beta, i \rangle$.

We have shown that for every $\dot{\Sigma}$ -algebra $\langle A, \alpha \rangle$, there is a universal arrow from $\langle A, \alpha \rangle$ to the functor $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$. This extends to an adjunction $F \dashv T$ uniquely by (Mac Lane 1998, §IV.1 Theorem 2).

For the monadicity of the adjunction $F \dashv T$, by Beck's monadicity theorem (Borceux 1994a, Theorem 4.4.4), we need to show that (1) the functor T reflects isomorphisms and (2) for a pair of morphisms $h, g : \langle A, \alpha \rangle \rightarrow \langle B, \beta \rangle \in \dot{\Psi}\text{-ALG}$ such that Th and Tg have a split coequaliser in $\dot{\Sigma}\text{-ALG}$, h and g have a coequaliser in $\dot{\Psi}\text{-ALG}$ and that is preserved by T .

For (1), the image of a morphism $h : \langle A, \alpha \rangle \rightarrow \langle B, \beta \rangle \in \dot{\Psi}\text{-ALG}$ under the translation functor T is still h (as a $\dot{\Sigma}$ -homomorphism). If h has an inverse h^{-1} in $\dot{\Sigma}\text{-ALG}$, then h^{-1} is a $\dot{\Psi}$ -homomorphism as well:

$$h \cdot \alpha = \beta \cdot \Psi h \iff h \cdot \alpha \cdot \Psi h^{-1} = \beta \iff \alpha \cdot \Psi h^{-1} = h^{-1} \cdot \beta$$

So h is also an isomorphism in $\dot{\Psi}\text{-ALG}$.

For (2), let $e : \langle B, T\beta \rangle \rightarrow \langle C, \gamma \rangle$ be the split coequaliser of Th and Tg . By the definition of split coequalisers, $U_{\dot{\Sigma}}e$ is a split coequaliser of $h, g : A \rightarrow B$ in $\mathcal{C}$. By the monadicity of $F_{\dot{\Psi}} \dashv U_{\dot{\Psi}}$, there is a coequaliser $e' : \langle B, \beta \rangle \rightarrow \langle C', \gamma' \rangle$ of h and g in $\dot{\Psi}\text{-ALG}$ that is preserved by $U_{\dot{\Psi}}$. Since $Te' \cdot Th = Tg \cdot Te'$, there is a morphism $i : \langle C, \gamma \rangle \rightarrow \langle C', T\gamma' \rangle$ such that $i \cdot e = Te'$ in $\dot{\Sigma}\text{-ALG}$. But in the category $\mathcal{C}$, both $U_{\dot{\Sigma}}e : B \rightarrow C$ and $U_{\dot{\Psi}}e' = U_{\dot{\Sigma}}Te' : B \rightarrow C'$ are coequalisers of $h, g : A \rightarrow B$, so $U_{\dot{\Sigma}}i$ is the unique isomorphism between C and C' . As a monadic functor, $U_{\dot{\Sigma}}$ reflects isomorphisms, so i is also an isomorphism in $\dot{\Sigma}\text{-ALG}$ too, and Te' is also a coequaliser of Th and Tg . $\square$

3.2*14. The two examples of freeness conditions $\mathcal{A}$ in Example 3.1*20 both satisfy the assumption in the theorem. In particular, the theorem applied to the translation $\text{MON} \rightarrow \text{GRP}$ in Example 3.2*4 constructs free groups over monoids. This theorem will later be used for constructing *free modular models*.

3.2*15. The construction of relative free algebras is a special case of the *adjoint lifting problem* (Borceux 1994a, §4.5), which asks given functors $Q \circ T = R \circ G$ such that R has a left adjoint, whether T has a left adjoint as well.

$$\begin{array}{ccc}
 \mathcal{A} & \begin{array}{c} \xleftarrow{T} \\ \xrightarrow{\tau} \\ \xrightarrow{\quad} \end{array} & \mathcal{B} \\
 \downarrow Q & & \downarrow G \\
 \mathcal{C} & \begin{array}{c} \xleftarrow{R} \\ \xrightarrow{\tau} \\ \xrightarrow{L} \end{array} & \mathcal{D}
 \end{array}$$

The situation of Theorem 3.2*13 is then the case where $L \dashv R$ is the identity adjunction $\text{Id} \dashv \text{Id} : \mathcal{C} \rightarrow \mathcal{C}$. An answer given by Borceux (1994a, Theorem 4.5.6) is that if G and Q are monadic and $\mathcal{B}$ has coequalisers, then T has a left adjoint. Therefore Theorem 3.2*13 can be generalised as follows.

3.2*16 Theorem. Let $L \dashv R : \mathcal{D} \rightarrow \mathcal{C}$ be an adjunction, $\mathcal{A}$ be a freeness condition on $\mathcal{D}$, $\dot{\Sigma} \in \text{Eqs}_f(\mathcal{C})$, and $\dot{\Psi} \in \text{Eqs}_{\mathcal{A}}(\mathcal{D})$. A functor $T : \dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ such that $U_{\dot{\Psi}} \circ T = R \circ U_{\dot{\Psi}}$ has a left adjoint $F : \dot{\Sigma}\text{-ALG} \rightarrow \dot{\Psi}\text{-ALG}$ if either (1) $\dot{\Psi}\text{-ALG}$ has coequalisers or (2) $\mathcal{A}$ is

closed under binary coproducts and constant functors.

$$\begin{array}{ccc}
 \dot{\Sigma}\text{-ALG} & \xleftarrow{T} & \dot{\Psi}\text{-ALG} \\
 \uparrow \scriptstyle U_{\dot{\Sigma}} \quad \vdash \quad \scriptstyle F_{\dot{\Sigma}} & \xrightarrow{F} & \uparrow \scriptstyle F_{\dot{\Psi}} \quad \dashv \quad \scriptstyle U_{\dot{\Psi}} \\
 \mathcal{C} & \xleftarrow{R} & \mathcal{D} \\
 & \xrightarrow{L} &
 \end{array}$$

Proof sketch. Free-forgetful adjunctions of equational systems are always monadic (Fiore and Hur 2009, Proposition 6.4), so if $\dot{\Psi}$ has coequalisers we have the left adjoint F by Borceux (1994a, Theorem 4.5.6). Alternatively, if $\mathcal{A}$ satisfies the required property, we can construct the left adjoint in the same way as in the proof of Theorem 3.2*13, inserting L and R in suitable places to go back and forth between $\mathcal{C}$ and $\mathcal{D}$. For example, the crucial diagram (17) in the proof of Theorem 3.2*13 should be modified to

$$\begin{array}{ccc}
 \Sigma A & \xrightarrow{\Sigma i} & \Sigma RB \\
 \alpha \downarrow & & \downarrow \tilde{T}\beta \\
 A & \xrightarrow{i} & RB
 \end{array}$$

where $\langle A \in \mathcal{C}, \alpha : \Sigma A \rightarrow A \rangle$ and $\langle B \in \mathcal{D}, \beta : \Psi B \rightarrow B \rangle$, and the functor $\tilde{T} : \Psi\text{-ALG} \rightarrow \Sigma\text{-ALG}$ from Lemma 3.2*10 is modified to

$$\tilde{T}\langle B, \beta \rangle = \langle RB, \Sigma RB \xrightarrow{\Sigma R \eta_B} \Sigma R(F_{\dot{\Psi}} B) \xrightarrow{To} RF_{\dot{\Psi}} B \xrightarrow{R \epsilon_{(B, \beta)}} RB \rangle.$$

where $o : \dot{\Psi}F_{\dot{\Psi}}B \rightarrow F_{\dot{\Psi}}B$ is the algebra structure on the free $\dot{\Psi}$ -algebra. $\square$

3.2*17 Example. Instantiate the adjunction $L \dashv R$ in Theorem 3.2*16 to be $+ \dashv \Delta : \text{SET} \rightarrow \text{SET} \times \text{SET}$. We have an equational system $\text{RING} \in \text{EQS}(\text{SET})$ of rings, and we can define an equational system $\text{ABMON} \in \text{EQS}(\text{SET} \times \text{SET})$ such that an algebra of ABMON is a pair $\langle A, M \rangle$ of sets with an abelian group on A and a monoid on M , so we have the following commutative triangle:

$$\begin{array}{ccc}
 \text{AB-ALG} \times \text{MON-ALG} & \xrightarrow{\cong} & \text{ABMON-ALG} \\
 \downarrow \scriptstyle U_{\text{AB}} \times U_{\text{MON}} & & \downarrow \scriptstyle U_{\text{ABMON}} \\
 \text{SET} \times \text{SET} & &
 \end{array}$$

Let $T : \text{RING-ALG} \rightarrow \text{AB-ALG} \times \text{MON-ALG}$ be the functor that projects out the additive group and multiplicative monoid structure of rings. It satisfies $U_{\text{AB}} \times U_{\text{MON}} \circ T = \Delta \circ U_{\text{RING}}$. By Theorem 3.2*16, T has a left adjoint F :

$$\begin{array}{ccc}
 \text{AB-ALG} \times \text{MON-ALG} & \xleftarrow{T} & \text{RING-ALG} \\
 \uparrow \scriptstyle U_{\text{AB}} \times U_{\text{MON}} \quad \vdash \quad \scriptstyle F_{\text{AB}} \times F_{\text{MON}} & \xrightarrow{F} & \uparrow \scriptstyle F_{\text{RING}} \quad \dashv \quad \scriptstyle U_{\text{RING}} \\
 \text{SET} \times \text{SET} & \xleftarrow{\Delta} & \text{SET} \\
 & \xrightarrow{+} &
 \end{array}$$

The functor F generates a free ring *simultaneously* over an abelian group A and a monoid M . Generally, whenever we have a cospan of equational systems $\dot{\Sigma} \rightarrow \dot{\Psi} \leftarrow \dot{\Phi}$, we can construct

an algebra of $\dot{\Psi}$ out of a $\dot{\Sigma}$ -algebra and a $\dot{\Phi}$ -algebra in a similar fashion. We will later use this technique to turn an ordinary model of an equational system to a modular model.

3.3 Colimits of equational systems

3.3*1. Colimits in $\text{EQS}(\mathcal{C})$ allow us to construct bigger equational systems by ‘gluing’ smaller ones. For example, the equational system for rings can be obtained by first taking the coproduct of GRP and MON and then taking a suitable coequaliser $L \rightrightarrows \text{GRP} + \text{MON}$ encoding the interaction of operations.

3.3*2 Theorem. *The category $\text{EQS}_{\mathcal{A}}(\mathcal{C})$ is small-cocomplete when the freeness condition $\mathcal{A}$ is closed under small-coproducts and if $\langle \Psi, \Theta \rangle \in \mathcal{A}$ then $\langle 0, \Psi \rangle \in \mathcal{A}$.*

Proof sketch. Arbitrary colimits can be constructed from coproducts and coequalisers, so it is sufficient to show that $\text{EQS}_{\mathcal{A}}(\mathcal{C})$ has small-set-indexed coproducts and coequalisers. We sketch the constructions here without proof.

(i) The coproduct of a (small) set of equational systems is obtained by taking the coproduct of signatures and equations. Precisely, if $\langle \Sigma_i \triangleright \Gamma_i \vdash L_i = R_i \rangle_{i \in I}$ is a set of equational systems with each of them in $\text{EQS}_{\mathcal{A}}(\mathcal{C})$, their coproduct is

$$\coprod_i \Sigma_i \triangleright \coprod_i \Gamma_i \vdash L = R$$

where $L, R : (\coprod_{i \in I} \Sigma_i)\text{-ALG} \rightarrow (\coprod_{i \in I} \Gamma_i)\text{-ALG}$ are

$$L\alpha = [L_i(\alpha \cdot \iota_i)]_{i \in I} \quad R\alpha = [R_i(\alpha \cdot \iota_i)]_{i \in I}.$$

(ii) Let $T_1, T_2 : \dot{\Psi} \rightarrow \dot{\Sigma}$ be a pair of translations. Let $\dot{\Sigma}'$ be

$$\dot{\Sigma} \upharpoonright (\Psi \vdash \tilde{T}_1 = \tilde{T}_2),$$

where $\tilde{T}_1, \tilde{T}_2 : \Sigma\text{-ALG} \rightarrow \Psi\text{-ALG}$ are obtained from T_1 and T_2 by Lemma 3.2*10. By the assumption on $\mathcal{A}$, $\dot{\Sigma}'$ is still in $\text{EQS}_{\mathcal{A}}(\mathcal{C})$. The inclusion translation $\dot{\Sigma} \rightarrow \dot{\Sigma}'$ is the coequaliser of $T_1, T_2 : \dot{\Psi} \rightarrow \dot{\Sigma}$. $\square$

3.3*3 Corollary. For every lkp category $\mathcal{C}$, the freeness condition $\mathcal{A} = \text{ENDO}_{\kappa}(\mathcal{C}) \times \text{ENDO}_{\kappa}(\mathcal{C})$ satisfies the assumption of the theorem, so $\text{EQS}_{\mathcal{A}}(\mathcal{C})$ is cocomplete. For a small-complete small category $\mathcal{C}$, the freeness condition $\mathcal{A} = \text{ENDO}(\mathcal{C}) \times \text{ENDO}(\mathcal{C})$ is a freeness condition so $\text{EQS}(\mathcal{C})$ is cocomplete.

3.3*4. Lastly, the following description of a special case of pushouts of equational system will be convenient in the future.

3.3*5 Lemma. Let $\dot{\Sigma} \in \text{EQS}(\mathcal{C})$ for a category $\mathcal{C}$ with finite coproducts, $\Theta_i : \mathcal{C} \rightarrow \mathcal{C}$ be an endofunctor, and $E_i = (\Theta_i \vdash L_i = R_i)$ an equation for $i \in \{1, 2\}$. Let T_1 and T_2 in the diagram below be the inclusion translations, then the following is a pushout diagram of T_1 and T_2 :

$$\begin{array}{ccc} \dot{\Sigma} & \xrightarrow{T_2} & \dot{\Sigma} \upharpoonright \Phi_2 \upharpoonright E_2 \\ T_1 \downarrow & & \downarrow \Gamma \\ \dot{\Sigma} \upharpoonright \Phi_1 \upharpoonright E_1 & \longrightarrow & \dot{\Sigma} \upharpoonright (\Phi_1 + \Phi_2) \upharpoonright E \end{array}$$

where $E = (\Theta_1 + \Theta_2 \vdash [L_1 \circ \alpha_1, L_2 \circ \alpha_2] = [R_1 \circ \alpha_1, R_2 \circ \alpha_2])$ and

$$\alpha_i : (\Sigma + (\Phi_1 + \Phi_2))\text{-ALG} \rightarrow (\Sigma + \Phi_i)\text{-ALG}$$

are the evident forgetful functors.

Proof. First of all, there are evident inclusion translations P_i (which are the unlabelled arrows in the pushout diagram):

$$P_i : (\dot{\Sigma} \upharpoonright (\Phi_1 + \Phi_2) \upharpoonright E)\text{-ALG} \rightarrow (\dot{\Sigma} \upharpoonright \Phi_i \upharpoonright E_i)\text{-ALG}$$

such that $T_1 \circ P_1 = T_2 \circ P_2$. Now for every equational system $\Psi \in \text{Eqs}(\mathcal{C})$ with translations functors $Q_i : \Psi\text{-ALG} \rightarrow (\dot{\Sigma} \upharpoonright \Phi_i \upharpoonright E_i)\text{-ALG}$ such that $T_1 \circ Q_1 = T_2 \circ Q_2$, we can define a translation functor

$$U : \Psi\text{-ALG} \rightarrow (\dot{\Sigma} \upharpoonright (\Phi_1 + \Phi_2) \upharpoonright E')\text{-ALG}$$

by sending every Ψ -algebra $\langle A, \alpha \rangle$ to the algebra on A with structure map:

$$[T_1(Q_1\dot{A}), \quad Q_1\alpha \cdot \iota_2, \quad Q_2\alpha \cdot \iota_2] : (\Sigma + \Phi_1 + \Phi_2) A \rightarrow A$$

It can be checked that such U is the unique one making $P_i \circ U = Q_i$. $\square$

4 A type theory for monoidal categories

4*1. When categorical constructions get complex, it is beneficial to use type theoretic *internal languages* to describe those constructions in a more intuitive manner (Crole 1994; Jacobs 1999; Lambek and Scott 1986). This technique will be prevalent in this paper. In this section, we introduce *monoidal algebraic theories*, which allows us to present equational systems over monoidal categories and their algebras in a convenient syntactic manner.

4*2. Monoidal algebraic theories are the *linear* counterpart of multi-sorted universal algebra, and the *syntactic* counterpart of *coloured PROs* (strict monoidal categories whose objects are products of a set of base objects) (MacLane 1965). Our presentation of monoidal algebraic theories below is largely based on the calculus of Jaskelioff and Moggi (2010) with the following new contributions: (1) the rules of judgemental equality are strengthened so that they are complete for monoidal categories; (2) rules for products and coproducts are included; (3) the notion of monoidal algebraic theories is formally introduced, so that one can be precise about what exactly the axioms and constants are assumed; in particular, one can formally define the internal language of a particular monoidal category; (4) we explicitly carve out a class of monoidal algebraic theories that give rise to equational systems (Section 4.3).

4.1 Monoidal algebraic theories

4.1*1. A *monoidal algebraic theory* $\mathcal{L}$ is specified by three pieces of data $\langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ as follows. Firstly, $\mathcal{B}$ is a set, and every element $\alpha \in \mathcal{B}$ is called a *base type*. The *types* of $\mathcal{L}$ are inductively generated by the rules

$$\frac{\alpha \in \mathcal{B}}{\vdash \alpha \text{ type}} \quad \frac{}{\vdash I \text{ type}} \quad \frac{\vdash A \text{ type} \quad \vdash B \text{ type}}{\vdash A \sqcup B \text{ type}}$$

Then $\mathcal{P}$ is a family of sets indexed by pairs of types A and B . Every element $f \in \mathcal{P}_{A,B}$ is called a *primitive operation*, and we will write $f : A \rightarrow B$ for $f \in \mathcal{P}_{A,B}$.

A *context* Γ is a finite list of variables and types $(x_1 : A_1, \dots, x_n : A_n)$. The empty context ‘()’ is omitted as usual when it appears before the symbol ‘ $\vdash$ ’. The concatenation of two contexts is written as Γ_l, Γ_r .

The (well typed) *terms* under contexts Γ are generated by the following rules:

$$\begin{array}{c}
\frac{}{x : A \vdash x : A} \quad \frac{f : A \rightarrow B \in \mathcal{P} \quad \Gamma \vdash t : A}{\Gamma \vdash f \ t : B} \quad \frac{}{\vdash * : I} \quad \frac{\Gamma_1 \vdash t_1 : A \quad \Gamma_2 \vdash t_2 : B}{\Gamma_1, \Gamma_2 \vdash (t_1, t_2) : A \square B} \\
\\
\frac{\Gamma \vdash t_1 : I \quad \Gamma_l, \Gamma_r \vdash t_2 : A}{\Gamma_l, \Gamma, \Gamma_r \vdash \mathbf{let} \ * = t_1 \ \mathbf{in} \ t_2 : A} \quad \frac{\Gamma \vdash t_1 : A_1 \square A_2 \quad \Gamma_l, x_1 : A_1, x_2 : A_2, \Gamma_r \vdash t_2 : B}{\Gamma_l, \Gamma, \Gamma_r \vdash \mathbf{let} \ (x_1, x_2) = t_1 \ \mathbf{in} \ t_2 : B}
\end{array}$$

Note that the type system is substructural, since the language is to be interpreted in monoidal categories rather than cartesian categories.

Lastly, $\mathcal{A}$ is a set of pairs $\langle \Gamma \vdash t_l : A, \Gamma \vdash t_r : A \rangle$ of terms of the same type and under the same context. Every element of $\mathcal{A}$ is called an *axiom* of $\mathcal{L}$. We will write $(\Gamma \vdash t_l = t_r : A) \in \mathcal{A}$ when a pair $\langle \Gamma \vdash t_l : A, \Gamma \vdash t_r : A \rangle$ is in $\mathcal{A}$.

4.1*2 Example. The monoidal algebraic theory of monoids has one base type M , two primitive operations $\mu : M \square M \rightarrow M$ and $\eta : I \rightarrow M$ and the following axioms, which correspond to the laws of monoids (2*2):

$$\begin{array}{l}
x : M \vdash \mu(\eta(*), x) = x : M \qquad x : M \vdash \mu(x, \eta(*)) = x : M \\
x : M, y : M, z : M \vdash \mu(\mu(x, y), z) = \mu(x, \mu(y, z)) : M
\end{array} \tag{18}$$

Although the axioms above look the same as the usual laws of monoids in $\mathbf{SET}$, we will see below they can actually be interpreted in all monoidal categories, and a model of this theory will be exactly a monoid in a monoidal category.

4.1*3. The typing rules above all have contexts Γ_l, Γ , and Γ_r as general as possible to make substitution admissible. For example, if we had given $\mathbf{let} \ * = t_1 \ \mathbf{in} \ t_2$ the following simpler typing rule (which is also semantically valid):

$$\frac{\Gamma_l, \Gamma_r \vdash t_2 : A}{\Gamma_l, x : I, \Gamma_r \vdash \mathbf{let} \ * = x \ \mathbf{in} \ t_2 : A} \tag{19}$$

then this rule could only be used when the context literally contains the linear unit as a variable. If we would like to substitute some term $\Gamma \vdash t_1 : I$ for the variable x in the term $\mathbf{let} \ * = x \ \mathbf{in} \ t_2$, the context would become $(\Gamma_l, \Gamma, \Gamma_r)$, and the rule (19) may not be applicable anymore because Γ can be more complex than just $x : I$. Therefore to make substitution admissible, all typing rules should have ‘contexts as general as possible’, anticipating substitution.

4.1*4 Lemma. The following *substitution rule* (or the ‘cut rule’) is admissible:

$$\frac{\Gamma_l, x : A, \Gamma_r \vdash t : B \quad \Delta \vdash u : A}{\Gamma_l, \Delta, \Gamma_r \vdash t[u/x] : B} \text{CUT}$$

where $t[u/x]$ is substituting u for x in t .

Proof sketch. As we explained above, the typing rules of terms above all have substitution ‘built-in’ by having contexts as general as possible. Thus the substitution rule can be shown by a straightforward induction on t . $\square$

4.1*5 Notation. In this paper, when we simultaneously substitute terms $u_1, \dots, u_n$ for all the variables in the context of a term $x_1 : A_1, \dots, x_n : A_n \vdash t : B$, we usually just write $t[u_1, \dots, u_n]$ instead of $t[u_1/x_1, \dots, u_n/x_n]$.

4.1*6. The *judgemental equality* $\Gamma \vdash t_1 \equiv t_2 : A$ of terms of a theory $\mathcal{L}$ is generated by the following rules plus the usual rules for reflexivity, symmetry, transitivity, and congruence under all term formers *and substitution*:

$$\begin{array}{c}
\frac{(\Gamma \vdash t_l = t_r : A) \in \mathcal{A}}{\Gamma \vdash t_l \equiv t_r : A} \text{Ax} \qquad \frac{\Gamma \vdash t : A}{\Gamma \vdash (\mathbf{let} \ * = * \ \mathbf{in} \ t : A) \equiv t : A} I\text{-}\beta \\
\\
\frac{\Gamma \vdash t_1 : I \quad \Gamma_l, x : I, \Gamma_r \vdash t_2 : A}{\Gamma_l, \Gamma, \Gamma_r \vdash (\mathbf{let} \ * = t_1 \ \mathbf{in} \ t_2[* / x]) \equiv t_2[t_1 / x] : A} I\text{-}\eta \\
\\
\frac{\Gamma_1 \vdash t_1 : A_1 \quad \Gamma_2 \vdash t_2 : A_2 \quad \Gamma_l, x_1 : A_1, x_2 : A_2, \Gamma_r \vdash t_3 : B}{\Gamma_l, \Gamma_1, \Gamma_2, \Gamma_r \vdash (\mathbf{let} \ (x_1, x_2) = (t_1, t_2) \ \mathbf{in} \ t_3) \equiv t_3[t_1 / x_1, t_2 / x_2] : B} \square\text{-}\beta \\
\\
\frac{\Gamma \vdash t_1 : A_1 \square A_2 \quad \Gamma_l, x : A_1 \square A_2, \Gamma_r \vdash t_2 : B}{\Gamma_l, \Gamma, \Gamma_r \vdash (\mathbf{let} \ (x_1, x_2) = t_1 \ \mathbf{in} \ t_2[(x_1, x_2) / x]) \equiv t_2[t_1 / x] : B} \square\text{-}\eta
\end{array}$$

The congruence rule under substitution is

$$\frac{\Gamma_l, x : A, \Gamma_r \vdash t_1 \equiv t_2 : B \quad \Delta \vdash u : A}{\Gamma_l, \Delta, \Gamma_r \vdash t_1[u / x] \equiv t_2[u / x] : B}$$

4.1*7. A *model* of a monoidal algebraic theory $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ in a monoidal category $\mathcal{E}$ consists of (1) an assignment of $\mathcal{E}$ -objects $\llbracket \alpha \rrbracket \in \mathbf{Ob}(\mathcal{E})$ to each base type $\alpha \in \mathcal{B}$, which induces the interpretation of all types and contexts:

$$\begin{array}{ll}
\llbracket I \rrbracket = I_{\mathcal{E}} & \llbracket A \square B \rrbracket = \llbracket A \rrbracket \square_{\mathcal{E}} \llbracket B \rrbracket \\
\llbracket () \rrbracket = I_{\mathcal{E}} & \llbracket \Gamma, x : A \rrbracket = \llbracket \Gamma \rrbracket \square_{\mathcal{E}} \llbracket A \rrbracket
\end{array}$$

and (2) an assignment of $\mathcal{E}$ -morphisms $\llbracket f \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket B \rrbracket$ to each primitive operation $f : A \rightarrow B \in \mathcal{P}$, which determines the interpretation of all terms:

$$\begin{aligned}
\llbracket x \rrbracket &= id & \llbracket f \ t \rrbracket &= \llbracket f \rrbracket \cdot \llbracket t \rrbracket & \llbracket * \rrbracket &= id & \llbracket (t_1, t_2) \rrbracket &= (\llbracket t_1 \rrbracket \square_{\mathcal{E}} \llbracket t_2 \rrbracket) \cdot _ \\
\llbracket \mathbf{let} \ * = t_1 \ \mathbf{in} \ t_2 \rrbracket &= \llbracket t_2 \rrbracket \cdot _ \cdot (\llbracket \Gamma_l \rrbracket \square_{\mathcal{E}} \llbracket t_1 \rrbracket \square_{\mathcal{E}} \llbracket \Gamma_r \rrbracket) \cdot _ \\
\llbracket \mathbf{let} \ (x_1, x_2) = t_1 \ \mathbf{in} \ t_2 \rrbracket &= \llbracket t_2 \rrbracket \cdot _ \cdot (\llbracket \Gamma_l \rrbracket \square_{\mathcal{E}} \llbracket t_1 \rrbracket \square_{\mathcal{E}} \llbracket \Gamma_r \rrbracket) \cdot _
\end{aligned}$$

where the underscores stand for the unique isomorphisms built from associators α and unitors λ, ρ to make the domain and codomain match. Moreover, the assignments $\llbracket - \rrbracket$ of a model must make $\llbracket t_l \rrbracket = \llbracket t_r \rrbracket$ for all axioms $\langle t_l, t_r \rangle \in \mathcal{A}$.

4.1*8. Let M and N be two models of a theory $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ in a monoidal category $\mathcal{E}$. A family of $\mathcal{E}$ -morphisms $h_{\alpha} : \llbracket \alpha \rrbracket_M \rightarrow \llbracket \alpha \rrbracket_N$ for all base types $\alpha \in \mathcal{B}$ extends to a family of morphisms $\tilde{h}_A : \llbracket A \rrbracket_M \rightarrow \llbracket A \rrbracket_N$ for all $\mathcal{L}$ -types A :

$$\tilde{h}_I = id_I \qquad \tilde{h}_{A \square B} = \tilde{h}_A \square_{\mathcal{E}} \tilde{h}_B \qquad \tilde{h}_{\alpha} = h_{\alpha}.$$

Such a family of morphisms h is called a *homomorphism* from M to N if for every primitive operation $f : A \rightarrow B \in \mathcal{P}$, the following commutes:

$$\begin{array}{ccc} \llbracket A \rrbracket_M & \xrightarrow{\llbracket f \rrbracket_M} & \llbracket B \rrbracket_M \\ \tilde{h}_A \downarrow & & \downarrow \tilde{h}_B \\ \llbracket A \rrbracket_N & \xrightarrow{\llbracket f \rrbracket_N} & \llbracket B \rrbracket_N \end{array}$$

Models of $\mathcal{L}$ in $\mathcal{E}$ and their homomorphisms assemble to a category $\mathcal{L}\text{-MOD}(\mathcal{E})$.

4.1*9 Theorem (Soundness). *Let $\llbracket - \rrbracket$ be a model of a monoidal algebraic theory $\mathcal{L}$. If two $\mathcal{L}$ -terms are judgementally equal, $\Gamma \vdash t_1 \equiv t_2 : A$, then $\llbracket t_1 \rrbracket = \llbracket t_2 \rrbracket$.*

Proof sketch. It is straightforward verification that the rules of judgemental equalities in 4.1*6 is validated by the axioms of a monoidal category after we show the *substitution lemma*: for all $\Gamma_l, x : A, \Gamma_r \vdash t : B$ and $\Delta \vdash u : A$, we have

$$\llbracket t[u/x] \rrbracket = \llbracket t \rrbracket \cdot (\Gamma_l \sqcap \llbracket u \rrbracket \sqcap \Gamma_r) : \llbracket \Gamma_l, \Delta, \Gamma_r \rrbracket \rightarrow \llbracket B \rrbracket$$

which itself can be proven by induction on t . □

4.1*10. Given a monoidal category $\mathcal{E}$, its *internal language* $\mathcal{L}(\mathcal{E})$ is a monoidal algebraic theory defined as follows:

- The set of base types of $\mathcal{L}(\mathcal{E})$ is exactly $\text{Obj } \mathcal{E}$, so we have an interpretation of all types as objects in $\mathcal{E}$ by interpreting every base type as itself.
- The set of primitive operations between two types A and B is $\mathcal{E}(\llbracket A \rrbracket, \llbracket B \rrbracket)$. Again, by interpreting every primitive operation as itself in the category $\mathcal{E}$, we have an interpretation of all terms in $\mathcal{E}$ as in 4.1*7.
- The set of axioms of $\mathcal{L}(\mathcal{E})$ is the maximal one containing all pairs of terms $\langle t_1, t_2 \rangle$ such that $\llbracket t_1 \rrbracket = \llbracket t_2 \rrbracket$ in $\mathcal{E}$.

We have a canonical interpretation of the internal language $\mathcal{L}(\mathcal{E})$ in $\mathcal{E}$: every type or primitive operation is interpreted as itself. This is a model of $\mathcal{L}(\mathcal{E})$ by construction. The internal language $\mathcal{L}(\mathcal{E})$ of a monoidal category $\mathcal{E}$ is *sound and complete* for reasoning about $\mathcal{E}$: two terms in $\mathcal{L}(\mathcal{E})$ satisfy $t_1 \equiv t_2$ if and only if they are equal under the canonical interpretation in $\mathcal{E}$. Soundness follows from Theorem 4.1*9 and completeness is by the construction of $\mathcal{L}(\mathcal{E})$.

4.2 More type formers for monoidal algebraic theories

4.2*1. Sometimes we work in monoidal categories with additional structure, and in this case we extend monoidal algebraic theories with new syntax for the additional structure. For example, when we work in left closed monoidal categories, we extend the calculus with a new type former B/A and term formers:

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x : A. t : B/A} \qquad \frac{\Gamma_1 \vdash t_1 : B/A \quad \Gamma_2 \vdash t_2 : A}{\Gamma_1, \Gamma_2 \vdash t_1 t_2 : B}$$

whose interpretation is given by the corresponding closed monoidal structure:

$$\llbracket \lambda x : A. t : B/A \rrbracket = \text{abs}(\llbracket t \rrbracket) \qquad \llbracket t_1 t_2 \rrbracket = \text{ev} \cdot (\llbracket t_1 \rrbracket \sqcap_{\mathcal{E}} \llbracket t_2 \rrbracket)$$

where $abst : \mathcal{E}(C \sqcap_{\mathcal{E}} A, B) \rightarrow \mathcal{E}(C, B/\mathcal{E}A)$ is the isomorphism associated to the adjunction $(-\sqcap_{\mathcal{E}} A) \dashv (-/\mathcal{E}A)$ and $ev : (B/\mathcal{E}A) \sqcap_{\mathcal{E}} A \rightarrow B$ is its counit. The β and η rules characterising the universal property of B/A are added to the equational theory routinely.

4.2*2. Note that the type former B/A is *contravariant* in the position of A , so in the presence of $/$ -types, the way in 4.1*8 of extending a family of morphisms $h_{\alpha} : \llbracket \alpha \rrbracket_M \rightarrow \llbracket \alpha \rrbracket_N$ between interpretations of base types to a family of morphisms $\tilde{h}_A$ between interpretations of all types will not work, and we will not have a category of models and *model homomorphisms* for a theory $\mathcal{L}$ with $/$ -types. However, we can still have a category $\mathcal{L}\text{-MOD}_{\cong}(\mathcal{E})$ of models and *model isomorphisms* by demanding that every h_{α} must be an $\mathcal{E}$ -isomorphism.

4.2*3. As a comment on the notation, [Jaskelioff and Moggi \(2010\)](#) used B^A for the closed structure, while we use [Lambek's \[1958\]](#) notation B/A to avoid the confusion with exponentials (which are right adjoints to cartesian products).

4.2*4 Example. Consider the theory of monoids from Example 4.1*2 and the extension of the closed structure B/A as in 4.2*1. *Cayley's theorem* from elementary algebra adapted to this setting says that the monoid $\langle M, \mu, \eta \rangle$ embeds into the monoid M/M with unit $\vdash \lambda x. x : M/M$ and multiplication

$$f : M/M, g : M/M \vdash \lambda x. f (g x) : M/M. \quad (20)$$

The embedding is given by $e = (x : M \vdash \lambda y. \mu(x, y) : M/M)$, which is a monoid homomorphism in the following sense:

$$\begin{aligned} e[\eta *] &\equiv \lambda y. \mu(\eta *, y) \equiv \lambda y. y \\ e[\mu(x, y)] &\equiv \lambda z. \mu(\mu(x, y), z) \equiv \lambda z. \mu(x, \mu(y, z)) \equiv \lambda z. e[x] (e[y] z) \end{aligned}$$

where we write $t[-]$ for substitution when there is a unique variable in the context of t . The embedding e has a left inverse $r = (f : M/M \vdash f (\eta *) : M)$ such that

$$x : M \vdash r[e] \equiv (\lambda y. \mu(x, y)) (\eta *) \equiv x : M$$

However, the inverse r is in general *not* a monoid homomorphism: in the context $(f : M/M, g : M/M)$, we have

$$r[\lambda x. f (g x)] \equiv f (g (\eta *)) \not\equiv \mu(f (\eta *), g (\eta *)) \equiv \mu(r[f], r[g])$$

This elementary result has many applications in functional programming because the multiplication (20) is usually a kind of function composition with $O(1)$ time complexity, regardless of the possibly expensive multiplication μ . When M is a free monoid in $\langle \text{SET}, \times, 1 \rangle$, i.e. a list, this optimisation is known as *difference lists* ([Hughes 1986](#)). In $\langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$, this optimisation is known as *codensity transformation* ([Hinze 2012](#)).

4.2*5. Cartesian products and coproducts in monoidal categories can be internalised similarly: we add type formers $A \times B$ and $A + B$ with term formers:

$$\frac{\Gamma \vdash t_1 : A_1 \quad \Gamma \vdash t_2 : A_2}{\Gamma \vdash \langle t_1, t_2 \rangle : A_1 \times A_2}$$

$$\frac{\Gamma \vdash t_1 : A_1 \times A_2 \quad \Gamma_l, x : A_i, \Gamma_r \vdash t_2 : B}{\Gamma_l, \Gamma, \Gamma_r \vdash t_2[\pi_i t_1/x] : B} \quad i \in \{1, 2\}$$

$$\frac{\Gamma \vdash t : A_i}{\Gamma \vdash \iota_i t : A_1 + A_2} \quad i \in \{1, 2\}$$

$$\frac{\Gamma \vdash t : A_1 + A_2 \quad x_i : A_i \vdash t_i : C, i \in \{1, 2\}}{\Gamma \vdash \mathbf{case} \, t \, \mathbf{of} \, \{\iota_1 \, x_1 \mapsto t_1; \, \iota_2 \, x_2 \mapsto t_2\} : C}$$

as well as their β and η rules. Sometimes we also write $[t_1, t_2] \, t$ instead of $\mathbf{case} \, t \, \mathbf{of} \, \{\iota_1 \, x_1 \mapsto t_1; \, \iota_2 \, x_2 \mapsto t_2\}$ for brevity.

These rules straightforwardly generalise to the non-binary cases. Note that the first rule introduces non-linearity to the syntax as the variables in Γ may appear in both t_1 and t_2 .

4.3 Syntactic presentations of equational systems

4.3*1. In this subsection, we carve out a class of monoidal algebraic theories that can be defined as equational systems, therefore allowing us to apply the theorems of equational systems to those monoidal algebraic theories, or from the opposite perspective, allowing us to define equational systems that syntactically use those monoidal algebraic theories.

4.3*2 Definition. Let $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ be a monoidal algebraic theory. When all primitive operations $f : A \rightarrow B$ and equations $\Gamma \vdash t_l = t_r : B$ of a monoidal algebraic theory $\mathcal{L}$ satisfy that $B \in \mathcal{B}$, we say that $\mathcal{L}$ has *basic outputs*.

4.3*3. For example, the theory of monoids in Example 4.1*2 has basic outputs, but the internal language $\mathcal{L}(\mathcal{E})$ of a monoidal category $\mathcal{E}$ does not have basic outputs because not all axioms of $\mathcal{L}(\mathcal{E})$ have a base type B .

4.3*4 Theorem. For every monoidal algebraic theory $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ with basic outputs and every monoidal category $\mathcal{E}$ with small-coproducts, there is an equational system $\Sigma_{\mathcal{L}}$ over the product category $\mathcal{E}^{\mathcal{B}}$ such that $\mathcal{L}\text{-MOD}(\mathcal{E}) \cong \Sigma_{\mathcal{L}}\text{-ALG}$.

Proof. Recall that types A of $\mathcal{L}$ are generated by $I, \square$, and $\alpha \in \mathcal{B}$. Therefore the interpretation of every type A in $\mathcal{E}$ determines a functor $\llbracket A \rrbracket : \mathcal{E}^{\mathcal{B}} \rightarrow \mathcal{E}$:

$$\llbracket I \rrbracket X = I_{\mathcal{E}} \quad \llbracket A_1 \square A_2 \rrbracket X = \llbracket A_1 \rrbracket X \square \llbracket A_2 \rrbracket X \quad \llbracket \alpha \rrbracket X = X_{\alpha}$$

Similarly, every context Γ of $\mathcal{L}$ determines a functor $\llbracket \Gamma \rrbracket : \mathcal{E}^{\mathcal{B}} \rightarrow \mathcal{E}$.

We define an endofunctor $\llbracket \mathcal{P} \rrbracket : \mathcal{E}^{\mathcal{B}} \rightarrow \mathcal{E}^{\mathcal{B}}$ by

$$\llbracket \mathcal{P} \rrbracket X = \langle \coprod_A \coprod_{f:A \rightarrow \alpha \in \mathcal{P}} \llbracket A \rrbracket X \rangle_{\alpha \in \mathcal{B}}.$$

It can be seen that an algebra of the endofunctor $\llbracket \mathcal{P} \rrbracket$ is precisely an interpretation of base types $\mathcal{B}$ and primitive operations $\mathcal{P}$ in $\mathcal{E}$.

Let $\uparrow_{\alpha} : \mathcal{E} \rightarrow \mathcal{E}^{\mathcal{B}}$ be the functor mapping every $X \in \mathcal{E}$ and $\beta \in \mathcal{B}$ to X if $\alpha = \beta$, or to $0_{\mathcal{E}}$ if $\alpha \neq \beta$. We write $\llbracket \Gamma \vdash \alpha \rrbracket : \mathcal{E}^{\mathcal{B}} \rightarrow \mathcal{E}^{\mathcal{B}}$ for $\uparrow_{\alpha} \circ \llbracket \Gamma \rrbracket$. An algebra of the endofunctor $\llbracket \Gamma \vdash \alpha \rrbracket$ is then an object $X \in \mathcal{E}^{\mathcal{B}}$ with a morphism in $\mathcal{E}^{\mathcal{B}}$

$$\langle \uparrow_{\alpha} (\llbracket \Gamma \rrbracket X) \, \beta \rangle_{\beta \in \mathcal{B}} \longrightarrow \langle X_{\beta} \rangle_{\beta \in \mathcal{B}},$$

which amounts to just an $\mathcal{E}$ -morphism $\llbracket \Gamma \rrbracket X \rightarrow X_{\alpha}$, i.e. $\llbracket \Gamma \rrbracket X \rightarrow \llbracket \alpha \rrbracket X$, since all other components are the unique $0_{\mathcal{E}} \rightarrow X_{\beta}$. Therefore the interpretation of every term $\Gamma \vdash t : \alpha$ of $\mathcal{L}$ with $\alpha \in \mathcal{B}$ then determines a mapping from a $\llbracket \mathcal{P} \rrbracket$ -algebra to a $\llbracket \Gamma \vdash \alpha \rrbracket$ -algebra. By induction on the term t in the style of Reynolds's [1983] *abstraction theorem*, it can be

shown that this mapping extends to a functor $\llbracket t \rrbracket : \llbracket \mathcal{P} \rrbracket\text{-ALG} \rightarrow \llbracket \Gamma \vdash \alpha \rrbracket\text{-ALG}$ satisfying $U_{\llbracket \Gamma \vdash \alpha \rrbracket} \circ \llbracket t \rrbracket = U_{\llbracket \mathcal{P} \rrbracket}$.

Let $\Sigma_{\mathcal{L}}$ be the equational system over $\mathcal{E}^{\mathcal{B}}$ with signature $\llbracket \mathcal{P} \rrbracket$ and a set of functorial equations $\llbracket \Gamma \rrbracket \vdash \llbracket t_l \rrbracket = \llbracket t_r \rrbracket$ for every axiom $(\Gamma \vdash t_l = t_r : \alpha) \in \mathcal{A}$ (c.f. Notation 3.1*7). By construction, the category $\mathcal{L}\text{-MOD}(\mathcal{E})$ of $\mathcal{L}$ -models in $\mathcal{E}$ and the category $\Sigma_{\mathcal{L}}\text{-ALG}$ of $\Sigma_{\mathcal{L}}$ -algebras (in $\mathcal{E}^{\mathcal{B}}$) are isomorphic. $\square$

4.3*5 Corollary. Let $\mathcal{E}$ be a cocomplete monoidal category with $\square$ preserving colimits of β -chains for a limit ordinal β . By the theorem above and Theorem 3.1*10, the category of models $\mathcal{L}\text{-MOD}(\mathcal{E})$ of a monoidal algebraic theory $\mathcal{L}$ with basic outputs is cocomplete and monadic over the category $\mathcal{E}^{\mathcal{B}}$.

4.3*6 Example. The monoidal algebraic theory of monoids in Example 4.1*2 has basic outputs, and the corresponding equational system obtained using the theorem above is precisely the equational system in Example 3.1*6.

4.3*7. When considering monoidal algebraic theories with extra type formers that are not covariant, such as the linear function type B/A in 4.2*1, Theorem 4.3*4 must additionally require a theory $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ with basic outputs to have *positive inputs*: every primitive operation $f : A \rightarrow \alpha$ and equation $\Gamma \vdash t_l = t_r : \alpha$ of $\mathcal{L}$ must satisfy that every base type $\beta \in \mathcal{B}$ occurs *positively* in A and Γ , so that A and Γ can be interpreted as covariant functors $\mathcal{E}^{\mathcal{B}} \rightarrow \mathcal{E}$. The rule for types in which $\beta \in \mathcal{B}$ occurs positively P and negatively N is defined inductively by the following grammar as usual:

$$P := \beta \mid \alpha \mid P \square P \mid P/N \qquad N := \alpha \mid N \square N \mid N/P$$

where α ranges over $\mathcal{B} \setminus \{\beta\}$. A base type β occurs positively in a context Γ if it occurs positively in every item of the context Γ . Other covariant type formers such as $\times$ and $+$ in 4.2*5 should be treated similarly to $\square$.

4.3*8. As explained above, type expressions P in which base types occur positively denote functors $\llbracket P \rrbracket$. It will be convenient if we also have access in the term language to the corresponding action of $\llbracket P \rrbracket$ on morphisms. Let $\tau \in \mathcal{B}$ and $x : A \vdash f : B$. For all types P and N in which $\tau \in \mathcal{B}$ occurs positively and negatively respectively, we define terms

$$x : P[A/\tau] \vdash P_{\tau}f : P[B/\tau] \qquad x : N[B/\tau] \vdash N_{\tau}f : N[A/\tau]$$

by induction on the structure of P and N :

$$\begin{aligned} \tau_{\tau}f &:= f & \alpha_{\tau}f &:= x & (P/N)_{\tau}f &:= \lambda y. P_{\tau}f [x (N_{\tau}f [y])] \\ (P \square P')_{\tau}f &:= \text{let } (x_l, x_r) = x \text{ in } (P_{\tau}f [x_l], P'_{\tau}f [x_r]) \end{aligned}$$

and symmetrically for N . When the type expression P contains (at most) one base type τ , we will just write Pf for $P_{\tau}f$.

As a convention, we will write $P_{\tau}f t$ for $P_{\tau}f[t/x]$, so we have an admissible rule:

$$\frac{x : A \vdash f : B \quad \Gamma \vdash t : P[A/\tau]}{\Gamma \vdash P_{\tau}f t : P[B/\tau]}$$

As an example, if $\mathcal{B} = \{\tau\}$, then τ occurs positively in the type expression $T = \tau \square \tau$, which denotes the functor $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$. Moreover we have $\Gamma \vdash Tf t : B \square B$ for all terms $A \vdash f : B$ and $\Gamma \vdash t : A \square A$.

4.3*9. A notable difference between a monoidal algebraic theory $\mathcal{L}$ and an equational system $\dot{\Sigma}$ over a monoidal category $\mathcal{E}$ is that the definition of the former (4.1*1) makes no reference to any specific monoidal category, while the definition of the latter (3.1*4) is parameterised by the underlying category $\mathcal{E}$. Therefore the operations of $\mathcal{L}$ must make sense for all monoidal categories. Such a generality can also be a limitation: sometimes we may be interested in operations that only make sense for a specific monoidal category $\mathcal{E}$. In this case, we need a slight generalisation of Theorem 4.3*4 to use monoidal algebraic theories to denote equational systems over $\mathcal{E}$ syntactically.

4.3*10. Let $\mathcal{L} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A} \rangle$ be a monoidal algebraic theory and M be a model of it in some monoidal category $\mathcal{E}$. We say that another theory $\mathcal{L}' = \langle \mathcal{B}', \mathcal{P}', \mathcal{A}' \rangle$ is an *extension* of $\mathcal{L}$ if $\mathcal{B} \subseteq \mathcal{B}'$, $\mathcal{P} \subseteq \mathcal{P}'$, and $\mathcal{A} \subseteq \mathcal{A}'$, where we implicitly treat types/terms generated by $\mathcal{B}$ and $\mathcal{P}$ as types/terms generated by the superset $\mathcal{B}'$ and $\mathcal{P}'$. Moreover, a model M' of $\mathcal{L}'$ in $\mathcal{E}$ is said to be *over* the model M if $\llbracket A \rrbracket_{M'} = \llbracket A \rrbracket_M$ and $\llbracket t \rrbracket_{M'} = \llbracket t \rrbracket_M$ for all types A and terms t of $\mathcal{L}$.

For example, let $\mathcal{E}$ be a monoidal category and $A \in \mathcal{E}$ be an object of $\mathcal{E}$. Let $\mathcal{L}'$ be the extension of the internal language $\mathcal{L}(\mathcal{E})$ with a new base type τ and a new operation $f : A \rightarrow \tau$. A model of $\mathcal{L}'$ over the canonical model of $\mathcal{L}(\mathcal{E})$ in $\mathcal{E}$ is precisely an object $X \in \mathcal{E}$ with a morphism $f : A \rightarrow X$.

4.3*11. We say that an extension $\mathcal{L}'$ of $\mathcal{L}$ has basic outputs if every new operation $(f : A \rightarrow B) \in (\mathcal{P}' \setminus \mathcal{P})$ and every equation $(\Gamma \vdash t_l = t_r : B) \in (\mathcal{A}' \setminus \mathcal{A})$ satisfies that $B \in \mathcal{B}' \setminus \mathcal{B}$. Similarly, we say that the extension $\mathcal{L}'$ has positive inputs if every $\alpha \in \mathcal{B}' \setminus \mathcal{B}$ occurs positively in A and Γ .

4.3*12 Theorem. Let $\mathcal{L}$ be a monoidal algebraic theory (with possibly extra type formers such as $/$, $+$, $\times$ -types in Section 4.2), and let M be a model of $\mathcal{L}$ in a monoidal category $\mathcal{E}$ with small-coproducts. Every extension $\mathcal{L}'$ of $\mathcal{L}$ that has basic outputs (and positive inputs) determines an equational system $\Sigma_{\mathcal{L}', M}$ over $\mathcal{E}^{\mathcal{B}' \setminus \mathcal{B}}$ such that algebras of $\Sigma_{\mathcal{L}', M}$ are in bijection with models of $\mathcal{L}'$ in $\mathcal{E}$ over M .

Proof. The proof goes almost the same as Theorem 4.3*4, except that the base types and primitive operations of $\mathcal{L}$ are fixed by the model M . $\square$

4.3*13. Our main use case of Theorem 4.3*12 is when $\mathcal{L}$ is the internal language $\mathcal{L}(\mathcal{E})$ of a monoidal category with coproducts $\mathcal{E}$ (4.1*10), and M is the canonical model, and there is exactly one new base type: $\mathcal{B}' \setminus \mathcal{B} = \{\tau\}$ of $\mathcal{L}(\mathcal{E})$ in $\mathcal{E}$. This allows us to present an equational system over $\mathcal{E}$ syntactically by a set of operations $f : A \rightarrow \tau$ and axioms $\Gamma \vdash t_l = t_r : \tau$ where A, Γ, t_l and t_r can refer to the existing objects and morphisms in $\mathcal{E}$.

5 Equational systems for monoids with operations

5*1. After the detour to monoidal algebraic theories, now we come back to monoids with operations, by which we mean the equational system Mon extended with a new operation $op : \Sigma M \rightarrow M$ for some endofunctor $\Sigma : \mathcal{E} \rightarrow \mathcal{E}$ and possibly some equations. In this section, we will discuss a general notion known as Σ -monoids in the literature (Fiore et al. 1999), which demands that the operation op is compatible with the monoid multiplication in a sense. After this, we will see some concrete examples.

$$\begin{array}{ccc}
(\Sigma X) \sqcap I & \xrightarrow{\theta_{X, \langle I, id \rangle}} & \Sigma(X \sqcap I) \\
& \searrow \rho_X & \downarrow \Sigma \rho_X \\
& & \Sigma X
\end{array}
\qquad
\begin{array}{ccc}
((\Sigma X) \sqcap Y) \sqcap Z & \xrightarrow{\theta_{X, \langle Y, f \rangle} \sqcap Z} & \Sigma(X \sqcap Y) \sqcap Z \\
\downarrow \alpha_{\Sigma X, Y, Z} & & \downarrow \theta_{X \sqcap Y, \langle Z, g \rangle} \\
(\Sigma X) \sqcap (Y \sqcap Z) & \xrightarrow{\theta_{X, \langle Y \sqcap Z, h \rangle}} & \Sigma(X \sqcap (Y \sqcap Z))
\end{array}$$

Fig. 1. Commutative diagrams for pointed strengths.

5*2. Two new contributions in this section are: (1) the theories of state and exceptions are stated in the generality of monoids, so they can be instantiated to monoids other than monads; (2) the connection between operations that must commute with monoid multiplication and operations that are not subject to this condition is clarified (Lemma 5*15).

5*3. Let $\mathcal{E}$ be a monoidal category and $\Sigma : \mathcal{E} \rightarrow \mathcal{E}$ be an endofunctor. A *pointed strength* θ for Σ is a natural transformation $\theta_{X, \langle Y, f \rangle} : (\Sigma X) \sqcap Y \rightarrow \Sigma(X \sqcap Y)$ for all X in $\mathcal{E}$ and $\langle Y \in \mathcal{E}, f : I \rightarrow Y \rangle$ in the coslice category $I/\mathcal{E}$, making the two diagrams in Figure 1 commute for all $X, Y, Z \in \mathcal{E}$, $f : I \rightarrow Y$, $g : I \rightarrow Z$, and $h := (f \sqcap g) \cdot \rho_I^{-1} : I \rightarrow Y \sqcap Z$.

5*4. Pointed strengths were originally introduced in the study of abstract syntax and substitution (Fiore et al. 1999). In this context, the informal intuition of a pointed strength $\theta_{X, \langle Y, f \rangle} : (\Sigma X) \sqcap Y \rightarrow \Sigma(X \sqcap Y)$ is that Σ is some syntactic constructor; X and Y are ‘sets’ whose elements are abstract syntax trees (containing variables); an element of $X \sqcap Y$ (resp. $(\Sigma X) \sqcap Y$) is an X -element t (resp. a ΣX -element t) together with a substitution δ , i.e. a mapping from variables in t to Y -elements; then θ is an operation that pushes the substitution δ inside the syntactic constructor Σ . The condition of pointedness $f : I \rightarrow Y$ of Y means that variables can be embedded into abstract syntax trees of Y . This intuition will be clearer later when we come back to abstract syntax in 6*22 with more mathematical details.

5*5. To denote Σ and θ syntactically, we extend the internal language $\mathcal{L}(\mathcal{E})$ (4.1*10) with a new type constructor Σ and the following typing rules

$$\frac{x : A \vdash f : B \quad \Gamma \vdash t : \Sigma A}{\Gamma \vdash \Sigma f \ t : \Sigma B} \qquad \frac{\vdash f : Y \quad \Gamma \vdash t : (\Sigma X) \sqcap Y}{\Gamma \vdash \theta_{X, \langle Y, f \rangle} \ t : \Sigma(X \sqcap Y)}$$

which of course are interpreted in $\mathcal{E}$ by the functor Σ and the strength θ .

5*6. The equational system $\Sigma\text{-MON}$ of Σ -monoids (Fiore and Hur 2009; Fiore et al. 1999) extends the theory MON of monoids (3.1*6 and 4.1*2) with a new operation $op : \Sigma\tau \rightarrow \tau$ and a new equation $L_{\Sigma\text{-MON}} = R_{\Sigma\text{-MON}}$:

$$\Sigma\text{-MON} = \text{MON} \upharpoonright \Sigma \upharpoonright (\Sigma - \sqcap - \vdash L_{\Sigma\text{-MON}} = R_{\Sigma\text{-MON}}) \quad (21)$$

where the new equation $L_{\Sigma\text{-MON}} = R_{\Sigma\text{-MON}}$ is given in terms of an extension of the internal language of $\mathcal{E}$ by the following pair of terms:

$$x : \Sigma\tau, y : \tau \vdash \mu \ (op \ x, y) = op \ (\Sigma\mu \ (\theta_{\tau, \langle \tau, \eta \rangle} \ (x, y))) : \tau \quad (22)$$

which encodes the following commutative diagram:

$$\begin{array}{ccc}
 (\Sigma\tau) \sqcap \tau & \xrightarrow{\theta_{\tau,(\tau,\eta)}} & \Sigma(\tau \sqcap \tau) \xrightarrow{\Sigma\mu} \Sigma\tau \\
 \text{\scriptsize $op \sqcap \tau$} \downarrow & & \downarrow \text{\scriptsize op} \\
 \tau \sqcap \tau & \xrightarrow{\mu} & \tau
 \end{array}$$

Note that (22) refers to θ , so technically the equational system should be denoted by $\langle \Sigma, \theta \rangle$ -MON, but writing Σ -MON is unlikely to cause confusion.

5*7. Equation (22) expresses that the operation op commutes with monoid multiplication. When using $\langle \text{ENDO}_f(\text{SET}), \bullet, V \rangle$ for modelling higher-order abstract syntax, which was the original context where Fiore et al. (1999) introduced Σ -monoids, this equation expresses the sensible condition that syntactic operations must commute with substitution.

However, this equation might not be desirable in other contexts. For example, in the study of *modal type theories*, there are plenty of operations that do not commute with substitution (Gratzer 2023). Also, when using $\langle \text{ENDO}_\kappa(\mathcal{C}), \circ, \text{Id} \rangle$ to model computational effects, this equation expresses that effectful operations must commute with sequential composition, which is not true in general. Those effectful operations that do satisfy this condition and have signature $\Sigma = A \sqcap -$ for some $A \in \mathcal{C}$ are called *algebraic operations* (Jaskelioff and Moggi 2010; Plotkin and Power 2001). We will say more about equation (22) shortly in Lemma 5*15 and see that imposing it on Σ -monoids actually does not lose generality.

5*8. When the monoidal category $\mathcal{C}$ is cocomplete and functors $\Sigma, \sqcap$ both preserve colimits of β -chains for some limit ordinal β , Theorem 3.1*10 ensures the existence of free Σ -monoids. When $\mathcal{C}$ is additionally closed, such as $\langle \text{ENDO}_\kappa(\mathcal{C}), \circ, \text{Id} \rangle$ in Section 2, there is a simple description of the free Σ -monoid (Fiore and Hur 2007): it is carried by the initial algebra $\mu X. I + A \sqcap X + \Sigma X$. This formula has many applications in modelling abstract syntax: variable binding (Fiore and Szamozvancev 2022), explicit substitution (Ghani et al. 2006), and scoped operations (Piróg et al. 2018).

5*9 Remark. Given a strong monad T over $\mathcal{C}$, Piróg (2016), Fiore and Saville (2017) studied a notion akin to Σ -monoids in 5*6 (with $\Sigma = T$) but additionally asking the operation $op : T\tau \rightarrow \tau$ to be an Eilenberg-Moore algebra of T . This concept is called *Eilenberg-Moore monoids* by Piróg (2016) and *T-monoids* by Fiore and Saville (2017). When $\mathcal{C}$ is closed, there is again a simple formula $\mu X. T(I + A \sqcap X)$ for the free T -monoid/Eilenberg-Moore monoid over $A \in \mathcal{C}$. Thinking of a monad T as an algebraic theory, using T -monoids instead of Σ -monoids allows us to impose equational laws on the operation op . In this paper we have opted in to use equational systems to present equations, so in the following we will continue with using Σ -monoids (with possibly additional equations). This has an additional advantage that we can have equations that involve both the monoid operations η, μ and the operation op at the same time, rather than just equations involving op .

5*10. Now let us look at some concrete examples. In all the following examples, the monoidal category $\langle \mathcal{C}, \sqcap, I \rangle$ is assumed to have small-coproducts $\coprod_{i \in S} A_i$ and finite products $\prod_{i \in F} A_i$. Additionally, we assume the monoidal product distributes over coproducts from the right:

$$(\coprod_{i \in S} A_i) \sqcap B \cong \coprod_{i \in S} (A_i \sqcap B).$$

5*11 Example. Let S be a set. The theory ST_S of *monads with global S -state* (Plotkin and Power 2002) can be generally defined for monoids as follows. The theory ST_S is Σ_{ST_S} -MON with signature Σ_{ST_S} denoted by the type expression:

$$((\prod_S I) \square \tau) + ((\coprod_S I) \square \tau),$$

whose first component represents an operation $g : (\prod_S I) \square \tau \rightarrow \tau$ reading the state, and the second component $p : (\coprod_S I) \square \tau \rightarrow \tau$ writes an S -value into the state.

Plotkin and Power's [2002] equations of these two operations can also be specified at this level of generality. For example, the law saying that writing $s \in S$ to the state and reading it immediately gives back s is

$$k : \prod_S I \vdash p_s(g(k, \eta^\tau)) = p_s(\text{let } * = \pi_s k \text{ in } \eta^\tau) : \tau$$

where $p_s(x)$ abbreviates $p(\iota_s *, x)$.

5*12 Example (Exception throwing). Letting E be a set, the theory ET_E of *exception throwing* is the theory Σ_{ET_E} -MON where $\Sigma_{\text{ET}_E} = (\coprod_E 1) \square - : \mathcal{C} \rightarrow \mathcal{C}$ equipped with the associator $\alpha : ((\coprod_E 1) \square X) \square Y \rightarrow \coprod_E 1 \square (X \square Y)$ as the strength, where $\coprod_E 1$ is the E -fold coproduct of the terminal object (which may be different from the monoidal unit I).

For the case $\mathcal{C} = \langle \text{ENDO}_\kappa(\mathcal{C}), \circ, \text{Id} \rangle$, the equational system ET_E describes (κ -accessible) monads $M : \mathcal{C} \rightarrow \mathcal{C}$ equipped with a natural transformation

$$\text{throw} : (\coprod_E 1) \circ M = \coprod_E (1 \circ M) = \coprod_E 1 \longrightarrow M \quad (23)$$

whose component $1 \rightarrow M$ for each $e \in E$ represents a computation throwing an exception e . Working in the generality of monoids allows us to generalise exceptions to more settings: taking $\mathcal{C} = \langle \text{ENDO}_\kappa(\text{SET}), *, \text{Id} \rangle$, the theory describes applicatives F with exception throwing:

$$(\coprod_E 1) * F \cong \coprod_E (1 * F) = \coprod_E (\int^{a,b} 1a \times Fb \times -^{a \times b}) \cong \coprod_E (\int^b Fb) \longrightarrow F \quad (24)$$

Note that exception throwing for monads (23) and for applicatives (24) differ by the domain: 1 vs $\int^b Fb$. This reflects the nature of applicative functors that computations are independent, so the computation after exception throwing is not necessarily discarded.

5*13. Although exception *throwing* is an algebraic operation, exception *catching* is not: if we were to model it as an operation $\text{catch} : M \times M \rightarrow M$ on a monad M such that $\text{catch} \langle p, h \rangle$ means catching exceptions possibly thrown by p and handling exceptions using h , then equation (22) for $\Sigma M = M \times M$ with the isomorphism $(M \times M) \circ Y \cong (M \circ Y) \times (M \circ Y)$ as the strength implies that

$$ph : M \times M, k : M \vdash \mu(\text{catch } ph, k) = \text{catch} \langle \mu(\pi_1 ph, k), \mu(\pi_2 ph, k) \rangle : M \quad (25)$$

But this is undesirable because the scopes of catching on the two sides are different: the left-hand side does not catch exceptions in k but the right-hand side catches exceptions in k .

5*14. Plotkin and Pretnar's [2009; 2013] take on this problem is that catching is inherently different from throwing: throwing is the only *operation* of the algebraic theory of computation with exceptions, but catching is a *model* of the theory. This view leads to the fruitful line of research on *handlers of algebraic effects*.

Wu et al. (2014) proposed an alternative perspective: *catch* is still an operation in the theory of monads supporting the effect of exceptions, but its signature should be $(M \times M) \circ M \rightarrow M$ rather than $M \times M \rightarrow M$. Their perspective can be justified by the following observation.

5*15 Lemma. Let $\langle \mathcal{E}, \square, I \rangle$ be a monoidal category. For all functors $\Phi : \mathcal{E} \rightarrow \mathcal{E}$ and monoids $\langle M, \mu, \eta \rangle$ in $\mathcal{E}$, define $\Sigma = (\Phi -) \square -$ and a pointed strength:

$$(\Sigma X) \square Y \xrightarrow{\cong} \Phi(X \square I) \square (X \square Y) \xrightarrow{\Phi(X \square \eta^Y) \square id} \Phi(X \square Y) \square (X \square Y) = \Sigma(X \square Y).$$

Then morphisms $f : \Phi M \rightarrow M$ without any condition are in bijection with morphisms $g : \Sigma M \rightarrow M$ satisfying the compatibility equation (22) of Σ -monoids instantiated with Σ .

Proof. The bijection between f and g is given by

$$f \mapsto (\Phi M \square M \xrightarrow{f \square M} M \square M \xrightarrow{\mu} M) \quad g \mapsto (\Phi M \xrightarrow{\Phi M \square \eta} \Phi M \square M \xrightarrow{g} M).$$

The reader is encouraged to use the internal language of $\mathcal{E}$ to check the round-trip property of this bijection, and below is a proof using the traditional notation. If we start with $f : \Phi M \rightarrow M$, after a round-trip we get the morphism

$$\Phi M \xrightarrow{\Phi M \square \eta} \Phi M \square M \xrightarrow{f \square M} M \square M \xrightarrow{\mu} M,$$

which, by the naturality of η , is equal to $(\mu \cdot \eta) \cdot f = f$. If we start with a morphism $g : \Sigma M \rightarrow M$ satisfying (22), after a round-trip we get the morphism

$$\Phi M \square M \xrightarrow{\Phi M \square \eta \square M} \Phi M \square M \square M \xrightarrow{g \square M} M \square M \xrightarrow{\mu} M. \quad (26)$$

By the definition of the pointed strength of Σ above, equation (22) instantiates to the equation of the following two morphisms $\Phi M \square M \square M \rightarrow M$:

$$\mu \cdot (g \square M) = g \cdot (\Phi \mu \square \mu) \cdot (\Phi(M \square \eta) \square M \square M)$$

The right-hand side is equal to $g \cdot (\Phi M \square \mu)$, so (26) is equal to $g \cdot (\Phi M \square \mu) \cdot (\Phi M \square \eta \square M) = g \cdot (\Phi M \square (\mu \cdot (\eta \square M))) = g \cdot (\Phi M \square id) = g$. $\square$

5*16. Therefore, using Lemma 5*15, exception catching on a monad can still be formulated as a Σ -monoid with $\Sigma M := (M \times M) \circ M = (\text{Id} \times \text{Id}) \circ M \circ M$. More generally, operations $s : A \circ M \circ M \rightarrow M$ satisfying (22) for an endofunctor A , or equivalently $A \circ M \rightarrow M$ without (22), are called *scoped operations* by Wu et al. (2014) and Piróg et al. (2018).

The name ‘scoped operations’ refers to the intuition that the operation s genuinely takes computations as its argument, so there is a meaningful boundary between its argument and the future continuation, so its arguments are ‘in the scope of s ’. This is in contrast with algebraic operations $a : B \circ M \rightarrow M$ satisfying (22), which are equivalently $B \rightarrow M$ by 5*15, so they do not genuinely take computations as input although they are usually presented in the form of $B \circ M \rightarrow M$.

5*17 Example (Exception catching). Now let us come back to the example of exception catching. Let $\mathcal{C}$ be a category with finite products and coproducts. Exception throwing and catching can be modelled as the theory of Σ_{Ec} -monoids in $\langle \text{ENDO}(\mathcal{C}), \circ, \text{Id} \rangle$, where the signature functor $\Sigma_{\text{Ec}} : \mathcal{C} \rightarrow \mathcal{C}$ is

$$\Sigma_{\text{Ec}} = ((\text{Id} \times \text{Id}) \circ - \circ -) + (1 \circ -)$$

equipped with the following pointed strength for all $X \in \mathcal{C}$ and $\langle Y, f \rangle : I/\mathcal{C}$:

$$\begin{aligned} (\Sigma_{\text{Ec}} X) \circ Y &= (((\text{Id} \times \text{Id}) \circ X \circ X) + (1 \circ X)) \circ Y \\ &\cong ((\text{Id} \times \text{Id}) \circ X \circ X \circ Y) + (1 \circ X \circ Y) \\ &\rightarrow ((\text{Id} \times \text{Id}) \circ X \circ \boxed{Y} \circ X \circ Y) + (1 \circ X \circ Y) \cong \Sigma_{\text{Ec}}(X \circ Y) \end{aligned}$$

where the boxed Y is inserted using $f : I \rightarrow Y$.

The intuition for the signature Σ_{Ec} is that the first operation $1 \circ M \rightarrow M$ is *throwing* an exception as in Example 5*12, and the second operation

$$\text{catch} : (\text{Id} \times \text{Id}) \circ M \circ M \cong (M \times M) \circ M \longrightarrow M \quad (27)$$

is catching. As we explained above, the trick here to avoid the undesirable equation (25) is that *catch* has an additional $- \circ M$ after $M \times M$ that represents an *explicit continuation* after the scoped operation *catch* (Piróg et al. 2018): *catch* $(\langle p, h \rangle, k)$ is understood as handling the exception in p with h and then continuing as k . Then equation (22) instantiates to

$$ph : M \times M, k : M, k' : M \vdash \mu(\text{catch}(ph, k), k') = \text{catch}(ph, \mu(k, k')) : M. \quad (28)$$

Unlike (25), this equation is reasonable for the theory of exceptions: catching ph and then doing k and then k' should be the same as catching ph and then continuing as $\mu(k, k')$. The scope of *catch* is not confused.

5*18. Moreover, we can add equations to the theory $\Sigma_{\text{Ec-MON}}$ to characterise the interaction of *throw* and *catch*. The theory Ec is $\Sigma_{\text{Ec-MON}}$ extended with the following equations:

$$\begin{aligned} k : \tau \vdash \text{catch}(\langle \text{throw}, \eta \rangle, k) &= k : \tau & k : \tau \vdash \text{catch}(\langle \text{throw}, \text{throw} \rangle, k) &= \text{throw} : \tau \\ k : \tau \vdash \text{catch}(\langle \eta, \text{throw} \rangle, k) &= k : \tau & k : \tau \vdash \text{catch}(\langle \eta, \eta \rangle, k) &= k : \tau \end{aligned}$$

where $\eta : I \rightarrow \tau$, $\text{throw} : 1 \rightarrow \tau$, and $\text{catch} : (\tau \times \tau) \square \tau \rightarrow \tau$. These equations can be alternatively presented with an empty context by replacing all the k 's with η as in $\vdash \text{catch}(\langle \text{throw}, \eta \rangle, \eta) = \eta : \tau$, which is equivalent to the first equation above, since by (28), $\text{catch}(\langle x, y \rangle, \eta); k = \text{catch}(\langle x, y \rangle, k)$.

5*19 Remark. By modelling scoped operations like *catch* as genuine operations rather than handlers, they enjoy the usual benefits of algebraic effects: we can impose equational axioms on them, consider the free models, combine their theories with effects, etc. But the reader may still wonder – from a purely practical programming perspective, what do we gain by modelling them as scoped operations rather than as effect handlers? Towards this question, Wu et al. (2014) argued that there is a problem of *compositionality* if we model operations like *catch* as effect handlers. Wu et al. used a concrete example of non-deterministic parsing with effect handlers, and Yang et al. (2022) clarified this problem using exception catching. Here, we explain this problem again using a small example about parallel composition as either a handler or an operation in its own right; c.f. Castellano et al. (1987).

Implementing parallel composition as a handler is similar to *interleaving concurrency* in concurrency theory: the parallel composition $P \parallel_i Q$ of two processes P and Q is handled/reduced to the nondeterministic choice of all the ways of interleaving the actions of P and Q . For example, if $P := a.0$ is the process that performs an action a and stops, and $Q := b.0$ is the process that performs an action b and stops, then we have

$$P \parallel_i Q = (a.b.0) + (b.a.0).$$

A problem arises if we want to lower the level of abstraction by refining the action a into two actions a_1 and a_2 (e.g. we may want to refine the action of writing a memory location into several actions when we move down the level of abstractions from an abstract memory model to a more realistic memory model). We may define this refinement as a meta-operation

r on programs that replaces every action a with two actions a_1 and a_2 . The expected outcome of ‘parallel composition of P and Q ’ with actions refined is

$$r(P) \parallel_i r(Q) = (a_1.a_2.0) \parallel_i (b.0) = (a_1.a_2.b.0) + (a_1.b.a_2.0) + (b.a_1.a_2.0),$$

but refining $P \parallel_i Q$ above gives us

$$r(P \parallel_i Q) = r((a.b.0) + (b.a.0)) = (a_1.a_2.b.0) + (b.a_1.a_2.0),$$

which is not the expected outcome in this scenario.

The problem here is precisely that if we treat parallel composition as a handler, then the scope of parallel composition is lost after handling. On the other hand, if we treat parallel composition as an operation in its own right, parallel composition of P and Q is *just* $P \parallel Q$ (with perhaps some axioms), so

$$r(P \parallel Q) = (a_1.a_2.0) \parallel b.0 = r(P) \parallel r(Q).$$

Therefore, if we model parallel composition as a handler $\parallel_i$, we must carefully ensure that the refinement operation r is applied to the processes P and Q before applying $\parallel_i$ to them, whereas if we model parallel composition a scoped operation $\parallel$, we can apply the refinement operation r to processes built with $\parallel$, which is a more compositional approach.

5*20. There are many more examples of Σ -monoids that we cannot expand on here. Some interesting ones are lambda abstraction (Fiore et al. 1999), the algebraic operations of π -calculus (Stark 2008), and the non-algebraic operation of parallel composition (Piróg et al. 2018). A collection of interesting programming examples can be found in (van den Berg and Schrijvers 2024).

6 Families of operations

6*1. Given a monoidal category $\mathcal{E}$, the coslice category $\text{MON}/\text{EQS}(\mathcal{E})$ contains all equational systems that extend the theory of monoids with new operations/equations. However, this category is sometimes too general – we will see later that there are many constructions that only work for a certain kind of operations, such as algebraic operations or scoped operations. Therefore, we will need to consider subcategories of $\text{MON}/\text{EQS}(\mathcal{E})$ that contain equational systems that extend MON with a certain family of operations. In this section, let us have a look at some important operation families, and it turns out that there are quite a few interesting things to be said about them.

6*2. This section is a key new contribution of this paper: the concept of operation families (Definition 6*3) and two theorems about the family of algebraic operations (Theorem 6*11 and 6*15). Theorem 6*11 says that the category of theories of algebraic operations is equivalent to the category of monoids in a monoidal category $\mathcal{E}$, which can be viewed as a generalisation from the classical monad-theory correspondence to a monoid-theory correspondence. Theorem 6*15 roughly says that almost every theory can be co-reflected into a theory of algebraic operations in a way that the initial object as a monoid is preserved, which formally captures the idea that for the purpose of modelling syntax, algebraic operations are sufficient (but for other purposes we would still like more expressive forms of operations because the notion of models is different).

6*3 Definition. An *operation family* on monoids in a monoidal category $\mathcal{E}$ is a subcategory $\mathcal{F} \subseteq \text{MON}/\text{EQS}(\mathcal{E})$ of the coslice category under the theory of monoids.

6*4. An object $\dot{\Sigma}$ in an operation family $\mathcal{F}$ is a pair $\langle \dot{\Sigma}, T_{\dot{\Sigma}} : \text{MON} \rightarrow \dot{\Sigma} \rangle$, but we will colloquially say *something* of $\dot{\Sigma}$ to mean that thing of $\dot{\Sigma}$. For example, when we say $\dot{\Sigma}$ has the free-forgetful adjunction, we mean that $\dot{\Sigma}$ has it.

6*5 (Algebraic operations). The simplest example is the family $\text{ALG}(\mathcal{E})$ of *algebraic operations* on a monoidal category $\langle \mathcal{E}, \square, I \rangle$ with binary coproducts. The full subcategory $\text{ALG}(\mathcal{E}) \subseteq \text{MON}/\text{EQS}(\mathcal{E})$ contains objects of the following form

$$\langle (A \square -)\text{-MON} \uparrow (K_B \vdash L = R), \quad T \rangle \quad (29)$$

where $A, B \in \mathcal{E}$; the functor $A \square -$ is equipped with the strength

$$\alpha_{A,X,Y} : (A \square X) \square Y \cong A \square (X \square Y)$$

which does not need Y to be pointed; T is the inclusion translation from MON ; $K_B : \mathcal{E} \rightarrow \mathcal{E}$ is the constant functor mapping to B . In other words, $\text{ALG}(\mathcal{E})$ contains all equational systems that are of the form ‘ $\Sigma\text{-MON}$ (5*6) extended with an equation’, for some $\Sigma = A \square -$.

6*6. In particular, the theory of exceptions (Example 5*12) and state (Example 5*11) are in $\text{ALG}(\mathcal{E})$. When $\mathcal{E} = \langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$ for an lkp category $\mathcal{C}$, $\text{ALG}(\mathcal{E})$ consists of theories of algebraic operations $A \circ M \rightarrow M$ for $A \in \text{ENDO}_{\kappa}(\mathcal{C})$ on κ -accessible monads M . When $\mathcal{E}$ is $\langle \text{ENDO}_{\kappa}(\text{SET}), *, \text{Id} \rangle$, it then contains theories of applicatives F with ‘applicative-algebraic’ operations $A * F \rightarrow F$.

6*7. When the monoidal category $\mathcal{E}$ is *right distributive* for binary coproducts, which means that the canonical morphism

$$[\iota_1 \square C, \iota_2 \square C] : (A \square C + B \square C) \rightarrow (A + B) \square C$$

is an isomorphism, the category $\text{ALG}(\mathcal{E})$ has binary coproducts as well: binary coproducts in $\text{ALG}(\mathcal{E})$ are equivalently pushouts in $\text{EQS}(\mathcal{E})$, and by Lemma 3.3*5, such a pushout still has a constant context $K_B + K_{B'} = K_{B+B'}$ and a signature $(A \square -) + (A' \square -) \cong (A + A') \square -$, so it is still in $\text{ALG}(\mathcal{E})$.

6*8. Many of the monoidal categories in Section 2 are right distributive:

- $\langle \text{ENDO}(\mathcal{C}), \circ, \text{Id} \rangle$ for a small-complete small $\mathcal{C}$;
- $\langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$ for an lkp $\mathcal{C}$;
- $\langle \mathcal{C}, \times, 1 \rangle$ for a cocomplete cartesian closed $\mathcal{C}$;
- $\langle \text{ENDO}_{\kappa}(\text{SET}), *, \text{Id} \rangle$ underlying applicative functors;
- $\langle \text{ENDO}_{s\kappa}(\mathcal{C}), \circ_s, \text{Id}_s \rangle$ for an lkp as a cartesian closed category $\mathcal{C}$;
- $\langle \text{ENDO}_f(\text{SET})^{\mathcal{G}}, *, I \rangle$ for a small strict monoidal category $\mathcal{G}$.

Moreover, for these choices of $\mathcal{E}$, every object of $\text{ALG}(\mathcal{E})$ has the free-forgetful adjunction using the freeness conditions in Section 3.1.

6*9. The restriction in (29) that the context of the equation must be a constant functor K_B deserves some explanation. Let $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$ be an endofunctor on a category $\mathcal{C}$. We call a functorial equation $K_B \vdash L = R$ over the signature Σ with a constant functor as its context a *constant equation*. Assume that $\mathcal{C}$ is locally small and small-complete, and that the functor Σ has the free-forgetful adjunction $F_{\Sigma} \dashv U_{\Sigma}$. By Corollary 3.2*7, the equation $K_B \vdash L = R$ is equivalently a pair of natural transformations $K_B \rightrightarrows F_{\Sigma}U_{\Sigma}$. Moreover, when $\mathcal{C}$ has an initial

object 0, such a natural transformation is uniquely determined by its component at 0:

$$\begin{array}{ccc} K_B 0 = B & \xrightarrow{id_B} & K_B X = B \\ L_0 \downarrow & & L_X \downarrow \\ U_\Sigma F_\Sigma 0 & \xrightarrow{U_\Sigma F_\Sigma f} & U_\Sigma F_\Sigma X \end{array}$$

Therefore constant equations $K_B \vdash L = R$ over the signature $\Sigma : \mathcal{C} \rightarrow \mathcal{C}$ are in bijection with pairs of $\mathcal{C}$ -morphisms $L', R' : B \rightarrow U_\Sigma F_\Sigma 0$, whose codomain is precisely (the carrier of) the initial algebra $\mu\Sigma$ of Σ .

6*10. More specially, assume that a monoidal category $\mathcal{E}$ satisfies the conditions on $\mathcal{C}$ in the last paragraph, and that $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$ preserves colimits of β -chains for some limit ordinal β . In this case, let Σ be the signature functor of the equational system $(A \square -)\text{-MON}$, i.e. $\Sigma = I + - \square - + A \square -$. Then both Σ and the equational system $(A \square -)\text{-MON}$ have initial algebras, whose carriers we denote by X and Y respectively. For every algebra $\langle Z, \alpha \rangle$ of $(A \square -)\text{-MON}$, it satisfies $K_B \vdash L = R$ if and only if the following diagram commutes:

$$B \xrightarrow[r]{l} X \xrightarrow{!_\Sigma} Z$$

where the morphism $!_\Sigma : X \rightarrow Z$ is the unique Σ -homomorphism from X to Z . Since the algebras of $(A \square -)\text{-MON}$ is a full subcategory of Σ -algebras, the morphism $!_\Sigma$ factors via $!_{(A \square -)\text{-MON}} : Y \rightarrow Z$, the unique homomorphism from the initial algebra of $(A \square -)\text{-MON}$:

$$B \xrightarrow[r]{l} X \xrightarrow{!_\Sigma} Y \xrightarrow{!_{(A \square -)\text{-MON}}} Z$$

Therefore, equations $K_B \vdash L = R$ can be given as a pair of morphisms $B \rightrightarrows Y$ without loss of expressivity. In particular, if $\mathcal{E}$ is $\langle \text{END}_f(\text{SET}), \circ, \text{Id} \rangle$, Y is precisely the free monad A^* over A , a pair of morphisms $B \rightrightarrows A^*$ for $A, B \in \text{END}_f(\text{SET})$ is indeed the traditional way of presenting a finitary algebraic theory. In fact, we can show that the category $\text{ALG}(\mathcal{E})$ is equivalent to the category of presentations of finitary algebraic theories and translations (as defined by e.g. [Fiore and Mahmoud \(2014\)](#)). Therefore, although constant equations seem very restrictive, they remain useful when the monoidal category $\mathcal{E}$ itself is informative.

6*11 Theorem. *Let $\mathcal{E}$ be a monoidal category with binary coproducts such that every object in $\text{ALG}(\mathcal{E})$ has the free-forgetful adjunction. There is an equivalence between $\text{ALG}(\mathcal{E})$ and the category $\text{MON}(\mathcal{E})$ of monoids in $\mathcal{E}$.*

Proof. In sketch, every $\tilde{\Sigma} \in \text{ALG}(\mathcal{E})$ is mapped to its initial algebra treated as a monoid, forgetting the operation. On the other hand, every monoid M is mapped to the theory of M -actions on monoids.

In more detail, the direction $\text{ALG}(\mathcal{E}) \rightarrow \text{MON}(\mathcal{E})$ of the equivalence sends every object $\tilde{\Sigma} = \langle \dot{\Sigma}, T_\Sigma \rangle$ in $\text{ALG}(\mathcal{E})$ to the initial algebra $\langle \mu\dot{\Sigma}, \alpha^\Sigma \rangle$ regarded as a monoid $T_\Sigma \langle \mu\dot{\Sigma}, \alpha^\Sigma \rangle$. On morphisms, every translation $T : \tilde{\Sigma} \rightarrow \tilde{\Psi} \in \text{ALG}(\mathcal{E})$ induces a unique $\tilde{\Sigma}$ -homomorphism out of the initial algebra:

$$h : \langle \mu\dot{\Sigma}, \alpha^\Sigma \rangle \rightarrow T \langle \mu\dot{\Psi}, \alpha^\Psi \rangle.$$

Then the arrow mapping is $T \mapsto T_\Sigma h$ where

$$T_\Sigma h : T_\Sigma \langle \mu\dot{\Sigma}, \alpha^\Sigma \rangle \rightarrow T_\Sigma (T \langle \mu\dot{\Psi}, \alpha^\Psi \rangle) = T_\Psi \langle \mu\dot{\Psi}, \alpha^\Psi \rangle.$$

The equality $T_\Sigma \circ T = T_\Psi$ is by the definition of morphisms in $\text{ALG}(\mathcal{E})$.

For the other direction, every monoid $\dot{M} = \langle M, \mu^M, \eta^M \rangle$ in $\mathcal{E}$ is sent to the theory $\dot{M}\text{-ACT}$ of $\dot{M}$ -actions on monoids, which is the theory of $(M \square -)\text{-MON}$ extended with the following two equations (expressed as an extension of the internal language of $\mathcal{E}$ as in 4.3*13):

$$\begin{aligned} &\vdash op(\eta^M, \eta^\tau) = \eta^\tau : \tau \\ &x : M, y : M \vdash op(\mu^M(x, y), \eta^\tau) = op(x, op(y, \eta^\tau)) : \tau \end{aligned}$$

saying that $op : M \square \tau \rightarrow \tau$ is a monoid action on τ . Every monoid morphism $f : \dot{M} \rightarrow \dot{N}$ is mapped to the translation $\dot{M}\text{-ACT} \rightarrow \dot{N}\text{-ACT}$ sending $\dot{N}$ -actions

$$\langle A \in \mathcal{E}, \alpha : (N \square A) + \Sigma_{\text{MON}} A \rightarrow A \rangle$$

to $\dot{M}$ -actions $\langle A, [\alpha \cdot \iota_1 \cdot (f \square A), \alpha \cdot \iota_2] : (M \square A) + \Sigma_{\text{MON}} A \rightarrow A \rangle$.

It remains to show that the mappings above are a pair of equivalences. Starting from a monoid $\dot{M}$, it can be shown that the category $(\dot{M}\text{-ACT})\text{-ALG}$ is equivalent to the coslice category $\dot{M}/\text{MON}(\mathcal{E})$ (see e.g. Fiore and Saville (2017, Proposition 5.5)). Thus the initial algebra of $\dot{M}\text{-ACT}$ is $\dot{M}$ as required.

Starting from a theory $\langle \dot{\Sigma}, T_\Sigma \rangle \in \text{ALG}(\mathcal{E})$ where

$$\dot{\Sigma} = (S \square -)\text{-MON} \upharpoonright (K_B \vdash L = R),$$

it is mapped to the monoid $T_\Sigma \langle \mu^{\dot{\Sigma}}, \alpha^{\dot{\Sigma}} \rangle$, which is then mapped back to the theory $\mu^{\dot{\Sigma}}\text{-ACT}$ with the inclusion translation. We need to construct an isomorphism translation $T : \dot{\Sigma} \rightarrow \mu^{\dot{\Sigma}}\text{-ACT}$ that preserves monoid operations. Given a monoid A with $\alpha : \mu^{\dot{\Sigma}} \square A \rightarrow A$ satisfying the laws of $\mu^{\dot{\Sigma}}\text{-ACT}$, T maps it to the $\dot{\Sigma}$ -algebra on A with the following operation:

$$S \square A \xrightarrow{S \square \eta^{\mu^{\dot{\Sigma}} \square A}} S \square \mu^{\dot{\Sigma}} \square A \xrightarrow{\alpha^{\mu^{\dot{\Sigma}} \square A}} \mu^{\dot{\Sigma}} \square A \xrightarrow{\alpha} A$$

where $\alpha^{\mu^{\dot{\Sigma}}} : S \square \mu^{\dot{\Sigma}} \rightarrow \mu^{\dot{\Sigma}}$ is the structure map of the initial algebra.

For the inverse of T , every tuple $\langle A, \beta : S \square A \rightarrow A \rangle \in \dot{\Sigma}\text{-ALG}$ is mapped to the following $\mu^{\dot{\Sigma}}\text{-ACT}$ -algebra on A :

$$\mu^{\dot{\Sigma}} \square A \xrightarrow{(\beta) \square A} A \square A \xrightarrow{\mu^A} A$$

where (β) is the unique homomorphism from the initial $\dot{\Sigma}$ -algebra $\mu^{\dot{\Sigma}}$ to the $\dot{\Sigma}$ -algebra $\langle A, \beta \rangle$. This completes the proof. $\square$

6*12. Instantiating $\mathcal{E}$ with $\langle \text{ENDO}_f(\mathcal{C}), \circ, \text{Id} \rangle$ for an lfp $\mathcal{C}$, we obtain an equivalence between finitary monads over $\mathcal{C}$ and theories of algebraic operations on finitary monads. This is reminiscent of the classical *theory-monad correspondence* between finitary monads and (presentations of) first-order algebraic theories. What is new is that Theorem 6*15 is applicable to other monoidal categories, such as those in Section 2, giving us equivalences of cartesian monoids/applicative functors/graded monads and the corresponding categories of theories of algebraic operations.

6*13. Another interesting property of $\text{ALG}(\mathcal{E})$ is the following saying that (almost) all equational systems of operations on monoids can be turned into one in $\text{ALG}(\mathcal{E})$ by a coreflection, and the coreflection preserves initial algebras, i.e. the abstract syntax of terms of operations. Hence in principle, theories of algebraic operations alone are sufficient for the purpose of modelling syntax.

6*14 Lemma. Let $\mathcal{E}$ be a monoidal category with binary coproducts. For every $\check{\Psi} \in \text{ALG}(\mathcal{E})$ and $\check{\Sigma} \in \text{MON}/\text{EQS}(\mathcal{E})$ such that both of them have initial algebras, the set $\text{MON}/\text{EQS}(\check{\Psi}, \check{\Sigma})$ is in natural bijection to monoid morphisms $\mu^{\check{\Psi}} \rightarrow \mu^{\check{\Sigma}}$ between the initial algebras of $\check{\Psi}$ and $\check{\Sigma}$ viewed as monoids.

Proof sketch. For one direction of the bijection ϕ , every translation $T : \check{\Psi} \rightarrow \check{\Sigma}$ sends the initial algebra $\mu^{\check{\Sigma}}$ of $\check{\Sigma}$ to a $\check{\Psi}$ -algebra carried by $\mu^{\check{\Sigma}}$. Then by the initiality of $\mu^{\check{\Psi}}$, there is a $\check{\Psi}$ -homomorphism, which is also a monoid morphism, $u : \mu^{\check{\Psi}} \rightarrow \mu^{\check{\Sigma}}$. We set $\phi(T) = u$. For the backward direction of the bijection ϕ , given a monoid morphism $h : \mu^{\check{\Psi}} \rightarrow \mu^{\check{\Sigma}}$, we define a translation $T : \check{\Psi} \rightarrow \check{\Sigma}$, i.e. a functor $T : \check{\Sigma}\text{-ALG} \rightarrow \check{\Psi}\text{-ALG}$ as follows. Recall that $\check{\Psi} \in \text{ALG}(\mathcal{E})$ must be of the form $\check{\Psi} = (G \square -)\text{-MON} \upharpoonright (K_B \vdash L = R)$, for some $G \in \mathcal{E}$. The functor T maps every $\check{\Sigma}$ -algebra $\langle A, \alpha : \Sigma A \rightarrow A \rangle$ to the $\check{\Psi}$ -algebra carried by A with

$$G \square A \xrightarrow{G \square \eta^{\mu^{\check{\Psi}}}} G \square \mu^{\check{\Psi}} \square A \xrightarrow{\alpha^{\mu^{\check{\Psi}}}} \mu^{\check{\Psi}} \square A \xrightarrow{h} \mu^{\check{\Sigma}} \square A \xrightarrow{\langle \alpha \rangle} A \square A \xrightarrow{\mu^A} A$$

where $\alpha^{\mu^{\check{\Psi}}} : G \square \mu^{\check{\Psi}} \rightarrow \mu^{\check{\Psi}}$ is the structure map of the initial $\check{\Psi}$ -algebra, $\langle \alpha \rangle : \mu^{\check{\Sigma}} \rightarrow A$ is the unique $\check{\Sigma}$ -homomorphism from the initial algebra $\mu^{\check{\Sigma}}$ to $\langle A, \alpha \rangle$. It can be checked that ϕ is a natural bijection. $\square$

6*15 Theorem. Let $\mathcal{E}$ be a monoidal category with binary coproducts such that every object of $\text{ALG}(\mathcal{E})$ has the free-forgetful adjunction. The category $\text{ALG}(\mathcal{E})$ is a coreflective subcategory of $\text{MON}/\text{EQS}_f(\mathcal{E})$, where $\text{EQS}_f(\mathcal{E}) \subseteq \text{EQS}(\mathcal{E})$ is the full subcategory containing equational systems with the free-forgetful adjunction:

$$\text{ALG}(\mathcal{E}) \xrightleftharpoons{\tau} \text{MON}/\text{EQS}_f(\mathcal{E}).$$

Moreover, the coreflector $[-]$ preserves initial algebras: for every $\langle \check{\Sigma}, T \rangle$ in the category $\text{MON}/\text{EQS}_f(\mathcal{E})$, the initial $\check{\Sigma}$ -algebra (viewed as a monoid via T) is isomorphic to the initial algebra of $[\langle \check{\Sigma}, T \rangle]$ also viewed as a monoid.

Proof sketch. Every theory $\langle \check{\Sigma}, T_{\Sigma} \rangle \in \text{MON}/\text{EQS}_f(\mathcal{E})$ has an initial algebra $\mu^{\check{\Sigma}} \in \mathcal{E}$ by assumption, and $\mu^{\check{\Sigma}}$ carries a monoid structure by $T_{\Sigma} : \text{MON} \rightarrow \check{\Sigma}$. We define the coreflector $[-]$ to map every $\langle \check{\Sigma}, T_{\Sigma} \rangle$ to $\mu^{\check{\Sigma}}\text{-ACT} \in \text{ALG}(\mathcal{E})$ as in the proof of Theorem 6*11. For every theory $\langle \check{\Psi}, T_{\Psi} \rangle \in \text{ALG}(\mathcal{E})$, by Lemma 6*14, each translation in the hom-set $\text{MON}/\text{EQS}_f(\langle \check{\Psi}, T_{\Psi} \rangle, \langle \check{\Sigma}, T_{\Sigma} \rangle)$ is equivalently a monoid morphism $\mu^{\check{\Psi}} \rightarrow \mu^{\check{\Sigma}}$, which is also equivalently a translation in $\text{ALG}(\langle \check{\Psi}, T_{\Psi} \rangle, [\langle \check{\Sigma}, T_{\Sigma} \rangle])$ by Theorem 6*11.

The coreflector maps each $\langle \check{\Sigma}, T \rangle \in \text{MON}/\text{EQS}_f(\mathcal{E})$ to the theory $\mu^{\check{\Sigma}}\text{-ACT}$. It can be shown that the category of algebras of $\mu^{\check{\Sigma}}\text{-ACT}$ is equivalent to the coslice category $\mu^{\check{\Sigma}}/\text{MON}(\mathcal{E})$ of monoids under $T\langle \mu^{\check{\Sigma}}, \alpha^{\check{\Sigma}} \rangle$, so the initial algebra of $\mu^{\check{\Sigma}}\text{-ACT}$ is still the monoid $\mu^{\check{\Sigma}}$. $\square$

6*16. Although $\text{ALG}(\mathcal{E})$ is sufficient for modelling syntax, it is *not* enough when we also consider models. The counit of the coreflection gives us a translation $[\langle \check{\Sigma}, T \rangle] \rightarrow \langle \check{\Sigma}, T \rangle$, i.e. a functor $\check{\Sigma}\text{-ALG} \rightarrow [\langle \check{\Sigma}, T \rangle]\text{-ALG}$, but this does not have to be an equivalence of categories.

6*17 (Scoped operations). Our next example of operation families is the family $\text{SCP}(\mathcal{E})$ of *scoped (and algebraic) operations*, such as exception catching (Example 5*17). Let $\mathcal{E}$ be a monoidal category with right distributive binary coproducts (6*7). The family $\text{SCP}(\mathcal{E})$ is the full subcategory of $\text{MON}/\text{EQS}(\mathcal{E})$ containing objects

$$\langle ((A \square - \square -) + (B \square -))\text{-MON} \upharpoonright (K_C \vdash L = R), \quad T \rangle \quad (30)$$

where $A, B, C \in \mathcal{E}$ and T is the inclusion translation. Letting $\Sigma := (A \square - \square -) + (B \square -)$, the pointed strength $\theta_{X, \langle Y, f \rangle}$ for Σ is the following composite:

$$\begin{aligned} (\Sigma X) \square Y &= (A \square X \square X + B \square X) \square Y \\ &\cong (A \square X \square X \square Y) + (B \square X \square Y) \\ &\rightarrow (A \square X \square \boxed{Y} \square X \square Y) + (B \square X \square Y) \\ &\cong \Sigma(X \square Y) \end{aligned}$$

where the boxed Y is inserted using $f : I \rightarrow Y$.

In the example of exceptions in Example 5*17, the monoidal category $\mathcal{E}$ is $\langle \text{ENDO}(\mathcal{C}), \circ, \text{Id} \rangle$, and A is $\text{Id} \times \text{Id}$, encoding the scoped operation *catch* that takes in two operands, and B is 1 , encoding the algebraic operation *throw* that takes in no operands.

6*18. When every object of $\text{SCP}(\mathcal{E})$ has the free-forgetful adjunction, for example when $\mathcal{E}$ is cocomplete and $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$ preserves colimits of β -chains for some limit ordinal β , a corollary of Theorem 6*15 is that the initial-algebra preserving coreflection there restricts to

$$\text{ALG}(\mathcal{E}) \xrightarrow{\quad \tau \quad} \text{SCP}(\mathcal{E}).$$

Thus the abstract syntax of programs with scoped operations can be alternatively expressed with only algebraic ones, but as argued by Piróg et al. (2018) and Yang et al. (2022), the models of scoped operations are different from those of the coreflected algebraic operations.

6*19. The operation family $\text{SCP}(\mathcal{E})$ should be more accurately called the family of *scoped and algebraic operations with constant equations*. We can certainly relax the restriction of constant equations to obtain bigger operation families. For example, we may have a family $\text{SCP}_1(\mathcal{E})$ that is similar to (30) but permits *first-order equations*, meaning that the functorial context can be either a constant functor K_C or a functor $C \square -$ for some $C \in \mathcal{E}$.

For computational effects in practice, it seems constant equations in $\text{ENDO}_\kappa(\mathcal{C})$ are enough for algebraic operations, evidenced by the examples in (Plotkin and Power 2002), whereas scoped operations sometimes need first-order equations. For example, a reasonable equation for exception catching $\text{catch} : M \times M \rightarrow M$ is that it is associative for $\times$:

$$\begin{array}{ccc} M \times M \times M & \xrightarrow{\text{catch} \times M} & M \times M \\ \downarrow M \times \text{catch} & & \downarrow \text{catch} \\ M & \xrightarrow{\text{catch}} & M \end{array}$$

As an equation in the monoidal category $\langle \text{ENDO}_\kappa(\mathcal{C}), \circ, \text{Id} \rangle$, this equation is first-order, since the context is $(\text{Id}_{\mathcal{C}} \times \text{Id}_{\mathcal{C}} \times \text{Id}_{\mathcal{C}}) \circ - : \text{ENDO}(\mathcal{C}) \rightarrow \text{ENDO}(\mathcal{C})$.

6*20. We did *not* require operation families $\mathcal{F} \subseteq \text{MON}/\text{EQS}(\mathcal{E})$ to be full, so we may also restrict the translations. For example, we can consider only translations that map operations to *operations* (rather than terms in general). Such translations are sometimes called *transliterations* (Arkor 2022). In particular, we define $\text{SCP}_1(\mathcal{E}) \subseteq \text{SCP}(\mathcal{E})$ to be the subcategory containing translations

$$T : ((A \square - \square -) + (B \square -))\text{-MON} \rightarrow ((A' \square - \square -) + (B' \square -))\text{-MON}$$

that sends $\langle M, s : A' \square M \square M, a : B' \square M \rightarrow M \rangle$ to $\langle M, s \cdot (f \square M \square M), a \cdot (g \square M) \rangle$ for some morphisms $f : A \rightarrow A'$ and $g : B \rightarrow B'$ in $\mathcal{E}$.

For example, let $A = \text{Id} \times \text{Id} \times \text{Id}$ and $A' = \text{Id} \times \text{Id}$, corresponding to a ternary operation t and a binary operation b respectively. Then the transliteration given by $f\langle x, y, z \rangle = \langle x, y \rangle$ translates the operation $t(x, y, z)$ to $b(x, y)$, but a transliteration cannot translate $t(x, y, z)$ to $b(b(x, y), z)$ that uses the target operation more than once, nor can it use the monoid structure μ and η in the translation.

6*21 (Variable-binding operations). Algebraic theories of operations with *variable-bindings* are called *second-order algebraic theories* (Fiore and Hur 2010; Fiore and Mahmoud 2010, 2014; Fiore and Szamozvancev 2022), and they can be formulated as an operation family as follows. For simplicity, we work specially in the monoidal category $\langle \text{SET}^{\text{FIN}}, \bullet, V \rangle$ in 2.2*2, but it is possible to replace SET^{FIN} with $\text{END}_k(\text{SET})$ for infinitary syntax or with SET^{CTX} for simply typed syntax given a category CTX of contexts and renamings.

6*22. A *binding signature* $\langle O, a \rangle$ consists of a set O of operations and an arity assignment $a : O \rightarrow \mathbb{N}^*$ of a sequence of natural numbers to each operation. Each $o \in O$ with $a(o) = \langle n_i \rangle_{1 \leq i \leq k}$ stands for an operation taking k arguments, each binding n_i variables:

$$o((x_{1,1}x_{1,2} \cdots x_{1,n_1}). e_1, \cdots, (x_{k,1}x_{k,2} \cdots x_{k,n_k}). e_k)$$

For example, the binding signature for λ -calculus has two operations $\{app, abs\}$: function application $a(app) = \langle 0, 0 \rangle$ has two arguments binding no variables; λ -abstraction $a(abs) = \langle 1 \rangle$ has one argument that binds one variable.

A binding signature $\langle O, a \rangle$ determines an endofunctor $\llbracket O, a \rrbracket$ on SET^{FIN} :

$$\llbracket O, a \rrbracket = \coprod_{o \in O, a(o) = \langle n_i \rangle_{1 \leq i \leq k}} \prod_{1 \leq i \leq k} (-)^{V^{n_i}}$$

where $(-)^{V^{n_i}}$ is the exponential by n_i -fold product of the monoidal unit V . This functor has a pointed strength $\theta_{X, \langle Y, \eta^Y \rangle}$:

$$(\coprod_o \prod_i X^{V^{n_i}}) \bullet Y \cong \coprod_o \prod_i (X^{V^{n_i}} \bullet Y) \xrightarrow{\coprod_o \prod_i t_{o,i}} \coprod_o \prod_i (X \bullet Y)^{V^{n_i}}$$

where $t_{o,i}$ is the adjoint transpose of

$$(X^{V^{n_i}} \bullet Y) \times V^{n_i} \xrightarrow{id \times \eta^Y} (X^{V^{n_i}} \bullet Y) \times (V^{n_i} \bullet Y) \cong (X^{V^{n_i}} \times V^{n_i}) \bullet Y \rightarrow X \bullet Y.$$

Previously in 5*4 we said that the intuition of a pointed strength θ is that it pushes a substitution inside a syntactic constructor. This is exactly what happens in the definition of $\theta_{X, \langle Y, \eta^Y \rangle}$ – each argument i of each operation o binds n_i new variables, and $t_{o,i}$ essentially means that after pushing the substitution inside the variable bindings, the newly bound variables are going to be substituted by themselves (where we use the pointedness $\eta^Y : V \rightarrow Y$ of Y).

6*23. The operation family $\text{VAR}(\text{SET}^{\text{FIN}}) \subseteq \text{MON}/\text{EQS}(\text{SET}^{\text{FIN}})$ then contains all objects of the following form:

$$\langle \llbracket O, a \rrbracket \text{-MON} \curvearrowright (\llbracket P, b \rrbracket \vdash L = R), T \rangle$$

where $\langle O, a \rangle$ and $\langle P, b \rangle$ are two binding signatures and T is still the inclusion translation. The category $\text{VAR}(\text{SET}^{\text{FIN}})$ is closed under coproducts by Lemma 3.3*5 and every object of it has the free-forgetful adjunction because $\llbracket O, a \rrbracket$ and $\llbracket P, b \rrbracket$ are finitary, which is a consequence of $(-)^V$ being a left adjoint to the right Kan extension RAN_{V+1} , so $(-)^V$ preserves all colimits.

6*24. Again, the coreflector in Theorem 6*15 allows us to turn every theory in VAR into one with only algebraic operations but isomorphic initial algebras. For example, under the coreflection, the theory Λ of untyped λ -calculus is turned into a theory $[\Lambda]$ which has an

ordinary n -ary operation t for *every* λ -term t with n free variables, together with suitable equations. The equational systems Λ and $[\Lambda]$ have isomorphic initial algebras (as monoids).

* * *

6*25. To reflect what we have done so far in this paper:

- We have seen how to present theories of monoids with operations using equational systems and monoidal algebraic theories (Sections 3 to 5);
- We can construct (relative) free algebras on cocomplete categories: Theorem 3.1*10, Theorem 3.1*17, Theorem 3.2*13;
- We can combine such theories modularly using colimits (Section 3.3);
- Theories of monoids with operations can be classified into operation families (Section 6), with the family of algebraic operations playing a special role: Theorem 6*11 and 6*15.

These results achieve syntactic modularity for computational effects. In the rest of the paper, we will develop a framework of *modular models*, which will allow us to combine models of existing theories into a model of a combined theory, thus achieving modularity for both syntax and semantics.

7 Modular constructions of algebraic structures

7*1. In many frameworks of algebraic theories, we can combine smaller theories into bigger ones by taking colimits. This gives us a modular way to design programming languages: language features are defined individually as algebraic theories, which are then combined to form algebraic theories of bigger languages. Programming languages are not only about syntax/theories though. What is usually more interesting is the implementations/models, and it turns out that modularly combining models is significantly harder than modularly combining theories. In the rest of this paper, we propose a formal theory of modularity in algebraic structures, which is a key new contribution of this paper.

7*2. Let us begin with some high-level description of the concepts that we will define. Let $\mathcal{T}$ be a category of some notion of algebraic theories, together with a functor $(-)\text{-ALG} : \mathcal{T}^{\text{op}} \rightarrow \text{CAT}$ that associates a category of models to every theory in $\mathcal{T}$, such that $\mathcal{T}$ has finite coproducts and each category of models $\Gamma\text{-ALG}$ has an initial object $\mu\Gamma$. For example, $\mathcal{T}$ may be the category $\text{ALG}(\mathcal{E})$ or $\text{SCP}(\mathcal{E})$ of theories of monoids with algebraic or scoped operations that we saw in Section 6 or the category of (ordinary) algebraic theories.

The motivation for a *modular model* of $\Gamma \in \mathcal{T}$ is that we treat $\Gamma \in \mathcal{T}$ as a *language feature* that can be added into existing programming languages, rather than a complete language on its own. A modular model for Γ is then going to be a family of functors $M_\Sigma : \Sigma\text{-ALG} \rightarrow (\Sigma + \Gamma)\text{-ALG}$, (oplax-) natural in $\Sigma \in \mathcal{T}$. In this way, for an arbitrary language $\Sigma \in \mathcal{T}$ and a model $A \in \Sigma\text{-ALG}$ of it, we can add the language feature Γ to the language Σ , giving us the combined language $\Sigma + \Gamma$, and a new model $M_\Sigma A \in (\Sigma + \Gamma)\text{-ALG}$.

7*3. In the situation above, we may also want to relate the old model $A \in \Sigma\text{-ALG}$ and the new model $M_\Sigma A \in (\Sigma + \Gamma)\text{-ALG}$. For this, we will define an *updater* u for the modular model M to be a family of Σ -homomorphisms $u_{\langle \Sigma, A \rangle} : A \rightarrow M_\Sigma A$, (oplax-) natural in Σ and A .

7*4. Moreover, we notice that coproducts $\Sigma + \Gamma$ are just one particular way to combine algebraic theories, and we could also consider other ways of combining theories, such as the *commutative combination* $\Sigma \otimes \Gamma$, also known as the *tensor* (Hyland et al. 2006), which is the theory $\Sigma + \Gamma$ with additional equations stating that operations from Σ and operations from Γ are commutative. Motivated by this, we will generalise from $(- + \Gamma) : \mathcal{T} \rightarrow \mathcal{T}$ in the definition of modular models to an arbitrary functor $\mathcal{T} \rightarrow \mathcal{T}$, better still, a functor $T : \mathcal{T} \rightarrow \mathcal{T}'$ where the domain $\mathcal{T}$ and codomain $\mathcal{T}'$ can be different ($\mathcal{T}'$ is also equipped with a functor $(-)\text{-ALG} : \mathcal{T}'^{\text{op}} \rightarrow \text{CAT}$). We will then call an (oplax-) natural family of functors $M_\Sigma : \Sigma\text{-ALG} \rightarrow (T\Sigma)\text{-ALG}$ a *model transformer* for T .

Model transformers will be shown to be equivalent to *liftings of functors* along fibrations, i.e. functors M making the diagram commute strictly,

$$\begin{array}{ccc} \mathcal{A} & \xrightarrow{M} & \mathcal{A}' \\ p \downarrow & & \downarrow p' \\ \mathcal{T} & \xrightarrow{T} & \mathcal{T}' \end{array}$$

where p and p' are the fibrations corresponding to $(-)\text{-ALG} : \mathcal{T}^{\text{op}} \rightarrow \text{CAT}$ and $(-)\text{-ALG} : \mathcal{T}'^{\text{op}} \rightarrow \text{CAT}$ respectively via the Grothendieck construction. A toolbox of constructions of such model transformers will be developed in 8*1.

7*5. The structure of this section is as follows: In Section 7.1, we define modular models of monoids with operations, and establish the correspondence between two formulations based on *indexed categories* and *fibrations* respectively. In Section 7.2, motivated by more scenarios of algebraic structures, we generalise modular models to *model transformers*, which are exactly liftings of functors along fibrations.

7.1 Modular models of monoids

7.1*1 Notation. We will work with indexed categories $\mathcal{C} \rightarrow \text{CAT}$ a lot in this subsection, where CAT is the category of large categories, so for convenience when we say ‘a category’ in this subsection, by default we mean a large category unless otherwise specified. We fix a monoidal category $\mathcal{E}$ and an operation family $\mathcal{F} \subseteq \text{MON}/\text{EQS}(\mathcal{E})$ such that $\mathcal{F}$ is closed under finite coproducts in $\text{MON}/\text{EQS}(\mathcal{E})$. Every object $\dot{\Sigma} \in \mathcal{F}$ is a pair $\langle \check{\Sigma} \in \text{EQS}(\mathcal{E}), T : \text{MON} \rightarrow \check{\Sigma} \rangle$. We will write $\dot{\Sigma}\text{-ALG}$ to mean the category $\check{\Sigma}\text{-ALG}$ of algebras for $\check{\Sigma}$.

7.1*2. In this subsection, we will make precise the idea of *modular models*, as motivated in 7*2, in two equivalent formulations, one based on *indexed categories* (Definition 7.1*5), and another based on *fibrations* (Theorem 7.1*17). The former is more explicit while the latter is more convenient to work with.

7.1*3. Recall that a morphism $T : \dot{\Sigma} \rightarrow \dot{\Psi}$ in $\text{EQS}(\mathcal{E})$ is a functor $\dot{\Psi}\text{-ALG} \rightarrow \dot{\Sigma}\text{-ALG}$ such that $U_{\dot{\Sigma}} \circ T = U_{\dot{\Psi}} : \dot{\Psi}\text{-ALG} \rightarrow \mathcal{E}$, so we can treat $(-)\text{-ALG}$ as a functor $\text{EQS}(\mathcal{E})^{\text{op}} \rightarrow \text{CAT}$. Similarly, we have a functor $(-)\text{-ALG} : \mathcal{F}^{\text{op}} \rightarrow \text{CAT}$.

7.1*4 Definition (Johnson and Yau (2020)). For a category $\mathcal{C}$ and two functors $F, G : \mathcal{C} \rightarrow \text{CAT}$, a *lax transformation* $\alpha : F \rightarrow G$ consists of a family of functors $\alpha_X : FX \rightarrow GX$ for all

$X \in \mathcal{C}$ and a family of natural transformations $\alpha_f : Gf \circ \alpha_X \rightarrow \alpha_Y \circ Ff$ for all $f : X \rightarrow Y$:

$$\begin{array}{ccc} FX & \xrightarrow{Ff} & FY \\ \alpha_X \downarrow & \nearrow \alpha_f & \downarrow \alpha_Y \\ GX & \xrightarrow{Gf} & GY \end{array}$$

Additionally, α must satisfy that $\alpha_{id_X} = id : \alpha_X \rightarrow \alpha_X$ for all $X \in \mathcal{C}$, and for all $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ in $\mathcal{C}$, $\alpha_{g \cdot f}$ is exactly the pasting of α_f and α_g :

$$\begin{array}{ccccc} FX & \xrightarrow{Ff} & FY & \xrightarrow{Gg} & FZ \\ \alpha_X \downarrow & \nearrow \alpha_f & \downarrow \alpha_Y & \nearrow \alpha_g & \downarrow \alpha_Z \\ GX & \xrightarrow{Gf} & GY & \xrightarrow{Gg} & GZ \end{array} = \begin{array}{ccccc} FX & \xrightarrow{Ff} & FY & \xrightarrow{Gg} & FZ \\ \alpha_X \downarrow & \nearrow \alpha_{g \cdot f} & & & \downarrow \alpha_Z \\ GX & \xrightarrow{Gf} & GY & \xrightarrow{Gg} & GZ \end{array}$$

An *oplax* transformation from F to G is similar to a lax transformation except that the direction of the 2-cells α_f becomes $\alpha_Y \circ Ff \rightarrow Gf \circ \alpha_X$. An (op)lax transformation α is called *strong* if α_f is a natural isomorphism for every $f : X \rightarrow Y$ in $\mathcal{C}$, and it is called *strict* if α_f is exactly the identity for every f (i.e. α is a natural transformation).

7.1*5 Definition. Given $\tilde{\Psi} \in \mathcal{F}$, a (strong/strict) *modular model* M of $\tilde{\Psi}$ is a (strong/strict) oplax transformation from $(-)\text{-ALG}$ to $(- + \tilde{\Psi})\text{-ALG} : \mathcal{F}^{\text{op}} \rightarrow \text{CAT}$.

7.1*6. Unpacking the definition, a modular model M of $\tilde{\Psi} \in \mathcal{F}$ consists of a family of functors $M_{\tilde{\Sigma}} : \tilde{\Sigma}\text{-ALG} \rightarrow (\tilde{\Sigma} + \tilde{\Psi})\text{-ALG}$ for all $\tilde{\Sigma} \in \mathcal{F}$ and a family of natural transformations $M_T : M_{\tilde{\Sigma}} \circ T \rightarrow (T + \tilde{\Psi}) \circ M_{\tilde{\Sigma}'}$ for all $T : \tilde{\Sigma} \rightarrow \tilde{\Sigma}'$ in $\mathcal{F}$:

$$\begin{array}{ccc} \tilde{\Sigma}\text{-ALG} & \xleftarrow{T} & \tilde{\Sigma}'\text{-ALG} \\ M_{\tilde{\Sigma}} \downarrow & \searrow M_T & \downarrow M_{\tilde{\Sigma}'} \\ (\tilde{\Sigma} + \tilde{\Psi})\text{-ALG} & \xleftarrow{T + \tilde{\Psi}} & (\tilde{\Sigma}' + \tilde{\Psi})\text{-ALG} \end{array}$$

such that M_{id} is the identity transformation, and for a pair of morphisms $T : \tilde{\Sigma} \rightarrow \tilde{\Sigma}'$ and $T' : \tilde{\Sigma}' \rightarrow \tilde{\Sigma}''$, $M_{T' \cdot T}$ is exactly the pasting of M_T and $M_{T'}$:

$$\begin{array}{ccccc} \tilde{\Sigma}\text{-ALG} & \xleftarrow{T} & \tilde{\Sigma}'\text{-ALG} & \xleftarrow{T'} & \tilde{\Sigma}''\text{-ALG} \\ M_{\tilde{\Sigma}} \downarrow & \searrow M_T & \downarrow M_{\tilde{\Sigma}'} & \searrow M_{T'} & \downarrow M_{\tilde{\Sigma}''} \\ (\tilde{\Sigma} + \tilde{\Psi})\text{-ALG} & \xleftarrow{T + \tilde{\Psi}} & (\tilde{\Sigma}' + \tilde{\Psi})\text{-ALG} & \xleftarrow{T' + \tilde{\Psi}} & (\tilde{\Sigma}'' + \tilde{\Psi})\text{-ALG} \end{array}$$

Specially, the data of a *strict* modular model M of $\tilde{\Psi} \in \mathcal{F}$ is only a family of functors $M_{\tilde{\Sigma}}$ such that the following diagram in CAT commutes:

$$\begin{array}{ccc} \tilde{\Sigma}\text{-ALG} & \xleftarrow{T} & \tilde{\Sigma}'\text{-ALG} \\ M_{\tilde{\Sigma}} \downarrow & & \downarrow M_{\tilde{\Sigma}'} \\ (\tilde{\Sigma} + \tilde{\Psi})\text{-ALG} & \xleftarrow{T + \tilde{\Psi}} & (\tilde{\Sigma}' + \tilde{\Psi})\text{-ALG} \end{array} \quad (31)$$

Therefore a strict modular model for $\tilde{\Psi}$ is exactly an ordinary natural transformation between the functors $(-)\text{-ALG}$ and $(- + \tilde{\Psi})\text{-ALG} : \mathcal{F}^{\text{op}} \rightarrow \text{CAT}$.

7.1*7 Example. For a trivial example, let $\mathcal{F}$ be the operation family containing only the initial object $\langle \text{MON}, \text{Id} : \text{MON} \rightarrow \text{MON} \rangle$ of $\text{MON}/\text{EQS}(\mathcal{E})$. In this case, $\langle \text{MON}, \text{Id} \rangle + \langle \text{MON}, \text{Id} \rangle$ is still $\langle \text{MON}, \text{Id} \rangle$, so a modular model of $\langle \text{MON}, \text{Id} \rangle$ is exactly an endofunctor $\text{MON}(\mathcal{E}) \rightarrow \text{MON}(\mathcal{E})$ over the category of monoids in $\mathcal{E}$.

7.1*8 Example. Let $\mathcal{E}$ be $\langle \text{ENDO}_K(\mathcal{C}), \circ, \text{Id} \rangle$ for lkp $\mathcal{C}$. A strict modular model M for the theory ET_E of *exception throwing* (Example 5*12) in the family $\text{ALG}(\mathcal{E})$ of algebraic operations is given by a family of functors

$$M_{\tilde{\Sigma}} : \tilde{\Sigma}\text{-ALG} \rightarrow (\tilde{\Sigma} + \text{ET}_E)\text{-ALG}$$

natural in $\tilde{\Sigma} \in \text{ALG}(\mathcal{E})$. Recall that each $\langle \tilde{\Sigma}, T_{\tilde{\Sigma}} \rangle \in \text{ALG}(\mathcal{E})$ is of the form

$$(S \square -)\text{-MON} \uparrow (K_G \vdash L = R)$$

for some S and $G \in \text{ENDO}_K(\mathcal{C})$, so objects of $\tilde{\Sigma}\text{-ALG}$ are tuples

$$\langle A \in \text{ENDO}_K(\mathcal{C}), \alpha : S \circ A \rightarrow A, \eta^A : \text{Id} \rightarrow A, \mu^A : A \circ A \rightarrow A \rangle$$

that are mapped by L and R to the same algebra. We define $M_{\tilde{\Sigma}}$ to send each of them to a $(\tilde{\Sigma} + \text{ET}_E)$ -algebra carried by $C_A := A \circ (\bar{E} + \text{Id})$, i.e. the *exception monad transformer* applied to A , where $\bar{E}$ is the E -fold coproduct of the terminal object 1 in $\text{ENDO}_K(\mathcal{C})$. Using the internal language of $\text{ENDO}_K(\mathcal{C})$, the operations $[\alpha^\sharp, \beta] : (S \circ C_A) + \bar{E} \rightarrow C_A$ are

$$\begin{aligned} \alpha^\sharp &= \llbracket s : S, a : A, e : \bar{E} + \text{Id} \vdash (\alpha(s, a), e) : C_A \rrbracket \\ \beta &= \llbracket e : \bar{E} \vdash (\eta^A, \iota_1 e) : C_A \rrbracket \end{aligned}$$

and C_A has the following monad structure:

$$\begin{aligned} \eta^C &= \llbracket \vdash (\eta^A, \iota_2(*)) : C_A \rrbracket \\ \mu^C &= \llbracket a : A, e : \bar{E} + \text{Id}, a' : A, e' : \bar{E} + \text{Id} \vdash \\ &\quad \text{let } (a'', e'') = d(e, a') \text{ in } (\mu^A(a, a''), \mu^{\bar{E} + \text{Id}}(e'', e')) \rrbracket \end{aligned}$$

where $d : (\bar{E} + \text{Id}) \circ A \rightarrow A \circ (\bar{E} + \text{Id})$ is the following distributive law:

$$e : \bar{E} + \text{Id}, a : A \vdash \text{case } e \text{ of } \{ \iota_1 e' \mapsto (\eta^A, \iota_1 e'); \iota_2 * \mapsto (a, \iota_2 *) : C_A \},$$

and $\mu^{\bar{E} + \text{Id}}$ is the multiplication of the exception monad $\bar{E} + \text{Id}$:

$$x : \bar{E} + \text{Id}, y : \bar{E} + \text{Id} \vdash \text{case } x \text{ of } \{ \iota_1 e \mapsto \iota_1 e; \iota_2 * \mapsto y \} : \bar{E} + \text{Id}.$$

On morphisms, $M_{\tilde{\Sigma}}$ sends a $\tilde{\Sigma}$ -homomorphism $h : A \rightarrow B$ to $h \circ (\bar{E} + \text{Id})$.

We need to check that the family of functors $M_{\tilde{\Sigma}}$ satisfies the naturality square (31): for all $T : \tilde{\Sigma} \rightarrow \tilde{\Sigma}'$, every object $\langle A, \alpha, \eta^A, \mu^A \rangle$ of $\tilde{\Sigma}'\text{-ALG}$ is mapped by functors $M_{\tilde{\Sigma}} \circ T$ and $(T + \text{ET}_E) \circ M_{\tilde{\Sigma}'}$ to two objects in $(\tilde{\Sigma} + \text{ET}_E)\text{-ALG}$ with the same carrier C_A , the same ET_E -algebra structure, and the same monad structure by construction. We need to show that the respective $\tilde{\Sigma}$ -algebras on C_A are also the same: the $\tilde{\Sigma}$ -algebra on C_A from $M_{\tilde{\Sigma}} \circ T$ is

$$T\alpha \circ (\bar{E} + \text{Id}) : S \circ A \circ (\bar{E} + \text{Id}) \rightarrow A \circ (\bar{E} + \text{Id}),$$

and the one from $(T + \text{ET}_E) \circ M_{\tilde{\Sigma}'}$ is $T(\alpha \circ (\bar{E} + \text{Id}))$. By Lemma 5*15, it is sufficient to show that the following diagram commutes

$$S \xrightarrow{S \circ \eta^C} S \circ C_A \xrightleftharpoons[T\alpha \circ (\bar{E} + \text{Id})]{T(\alpha \circ (\bar{E} + \text{Id}))} C_A \quad (32)$$

To see this, we first observe that we have the following $\check{\Sigma}'$ -homomorphism square:

$$\begin{array}{ccc} S' \circ A & \xrightarrow{\alpha} & A \\ S' \circ A \circ \iota_2 \downarrow & & \downarrow A \circ \iota_2 \\ S' \circ A \circ (\bar{E} + \text{Id}) & \xrightarrow{\alpha \circ (\bar{E} + \text{Id})} & A \circ (\bar{E} + \text{Id}) \end{array}$$

and this square is mapped by the translation T to a commuting square

$$\begin{array}{ccc} S \circ A & \xrightarrow{T\alpha} & A \\ S \circ A \circ \iota_2 \downarrow & & \downarrow A \circ \iota_2 \\ S \circ A \circ (\bar{E} + \text{Id}) & \xrightarrow{T(\alpha \circ (\bar{E} + \text{Id}))} & A \circ (\bar{E} + \text{Id}) \end{array}$$

This implies (32) because

$$\begin{aligned} & T(\alpha \circ (\bar{E} + \text{Id})) \cdot (S \circ \eta^C) \\ = & \{ \text{definition of } \eta^C \} \\ & T(\alpha \circ (\bar{E} + \text{Id})) \cdot (S \circ A \circ \iota_2) \cdot (S \circ \eta^A) \\ = & \{ \text{by the last commutativity square above} \} \\ & (A \circ \iota_2) \cdot T\alpha \cdot (S \circ \eta^A) \\ = & \{ \text{by naturality} \} \\ & (T\alpha \circ (\bar{E} + \text{Id})) \cdot (A \circ \iota_2) \cdot (S \circ \eta^A) \\ = & (T\alpha \circ (\bar{E} + \text{Id})) \cdot (S \circ \eta^C) \end{aligned}$$

7.1*9. CAT-valued functors also known as (strict) *indexed categories*, which are equivalent to *split fibrations* via the *Grothendieck construction*, so we can alternatively formulate modular models based on fibrations. The fibrational formulation is usually easier to work with, especially when we talk about morphisms between modular models later, which would be a 3-categorical concept in the indexed-category formulation. Also, the fibrational formulation slightly simplifies some ‘dependently typed’ constructions such as mapping each $\check{\Sigma}$ in $\mathcal{F}$ to the initial algebra in $\check{\Sigma}$ -ALG. We will only need the very basics about fibrations (see e.g. Borceux (1994a); Jacobs (1999); Streicher (2023)).

7.1*10. Applying the Grothendieck construction to $(-)\text{-ALG} : \mathcal{F}^{\text{op}} \rightarrow \text{CAT}$, we obtain a (split) fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$, which we explicitly describe below. The intuition is that $\mathcal{F}\text{-ALG}$ is the category of *all models of all equational systems* in $\mathcal{F}$. The objects of the category $\mathcal{F}\text{-ALG}$ are tuples

$$\langle \check{\Sigma} \in \text{Eqs}(\mathcal{E}), \quad T_{\check{\Sigma}} : \text{MON} \rightarrow \check{\Sigma}, \quad A \in \mathcal{E}, \quad \alpha : \Sigma A \rightarrow A \rangle$$

such that $\langle \check{\Sigma}, T_{\check{\Sigma}} \rangle \in \mathcal{F}$ and $\langle A, \alpha \rangle \in \check{\Sigma}\text{-ALG}$. Morphisms between two objects $\langle \check{\Sigma}, T_{\check{\Sigma}}, A, \alpha \rangle$ and $\langle \check{\Psi}, T_{\check{\Psi}}, B, \beta \rangle$ are pairs $\langle T, h \rangle$ where $T : \check{\Sigma} \rightarrow \check{\Psi}$ is a functorial translation in $\mathcal{F}$, and the other component $h : A \rightarrow B \in \mathcal{E}$ is a $\check{\Sigma}$ -algebra homomorphism from $\langle A, \alpha \rangle$ to $T\langle B, \beta \rangle$:

$$\begin{array}{ccc} \Sigma B \xleftarrow{\Sigma h} \Sigma A & & \Psi B \\ T\beta \downarrow & \downarrow \alpha & \downarrow \beta \\ B \xleftarrow{h} A & & B \end{array} \quad \text{in } \check{\Sigma}\text{-ALG} \quad \xleftarrow{T} \quad \text{in } \check{\Psi}\text{-ALG}$$

The identities in $\mathcal{F}\text{-ALG}$ are pairs $\langle \text{Id} : \dot{\Sigma} \rightarrow \dot{\Sigma}, \text{id} : A \rightarrow A \rangle$, and the composition of two morphisms $\langle T, h \rangle$ and $\langle T', h' \rangle$ is $\langle T \circ T', h \cdot h' \rangle$.

The fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ is the projection: $P\langle \dot{\Sigma}, T_{\Sigma}, A, \alpha \rangle = \langle \dot{\Sigma}, T_{\Sigma} \rangle$ and $P\langle T, h \rangle = T$. It has a split cleavage assigning to every pair of a morphism $T : \langle \dot{\Sigma}, T_{\Sigma} \rangle \rightarrow \langle \dot{\Psi}, T_{\Psi} \rangle \in \mathcal{F}$ and an object $\langle \dot{\Psi}, T_{\Psi}, B, \beta \rangle \in \mathcal{F}\text{-ALG}$ a cartesian morphism $\langle T, \text{id} \rangle : \langle \dot{\Sigma}, T_{\Sigma}, B, T\beta \rangle \rightarrow \langle \dot{\Psi}, T_{\Psi}, B, \beta \rangle$ in $\mathcal{F}\text{-ALG}$.

7.1*11 Example. Given an object $\ddot{\Psi} \in \mathcal{F}$, the Grothendieck construction of the functor $(- + \ddot{\Psi})\text{-ALG} : \mathcal{F}^{\text{op}} \rightarrow \text{CAT}$ is a split fibration $Q : (\mathcal{F} + \ddot{\Psi})\text{-ALG} \rightarrow \mathcal{F}$. Explicitly, the objects of $(\mathcal{F} + \ddot{\Psi})\text{-ALG}$ are tuples:

$$\langle \dot{\Sigma} \in \text{EQS}(\mathcal{E}), \quad T_{\Sigma} : \text{MON} \rightarrow \dot{\Sigma}, \quad A \in \mathcal{E}, \quad \alpha : \Sigma A \rightarrow A, \quad \beta : \Psi A \rightarrow A \rangle$$

such that $\langle \dot{\Sigma}, T_{\Sigma} \rangle \in \mathcal{F}$, $\langle A, \alpha \rangle \in \dot{\Sigma}\text{-ALG}$, $\langle A, \beta \rangle \in \dot{\Psi}\text{-ALG}$, and $T_{\Psi}\alpha = T_{\Sigma}\beta$. Morphisms in the category $(\mathcal{F} + \ddot{\Psi})\text{-ALG}$ are similar to those $\langle T, h \rangle$ in $\mathcal{F}\text{-ALG}$, but require h also to be a $\ddot{\Psi}$ -homomorphism. Therefore, objects of $(\mathcal{F} + \ddot{\Psi})\text{-ALG}$ are models of some equational systems in $\mathcal{F}$ that are additionally equipped with a $\ddot{\Psi}$ -algebra. The functor Q is the projection $\langle \dot{\Sigma}, T, A, \alpha, \beta \rangle \mapsto \langle \dot{\Sigma}, T \rangle$.

The split fibration $Q : (\mathcal{F} + \ddot{\Psi})\text{-ALG} \rightarrow \mathcal{F}$ can be alternatively given as the change of base of the fibration $P : \mathcal{F}\text{-MON} \rightarrow \mathcal{F}$ along the functor $(- + \ddot{\Psi}) : \mathcal{F} \rightarrow \mathcal{F}$, which is the following pullback in the category CAT of large categories:

$$\begin{array}{ccc} (\mathcal{F} + \ddot{\Psi})\text{-ALG} & \xrightarrow{K} & \mathcal{F}\text{-ALG} \\ Q \downarrow \lrcorner & & \downarrow P \\ \mathcal{F} & \xrightarrow{- + \ddot{\Psi}} & \mathcal{F} \end{array} \quad (33)$$

The functor K maps objects $\langle \dot{\Sigma}, T_{\Sigma}, A, \alpha, \beta \rangle$ to $\langle \langle \dot{\Sigma}, T_{\Sigma} \rangle + \langle \dot{\Psi}, T_{\Psi} \rangle, A, [\alpha, \beta] \rangle$. The pair $\langle K, (- + \ddot{\Psi}) \rangle$ is a morphism of split fibrations $Q \rightarrow P$.

7.1*12 Definition. Given two fibrations $P : \mathcal{E} \rightarrow \mathcal{B}$ and $P' : \mathcal{E}' \rightarrow \mathcal{B}'$, a *lifting* of a functor $F : \mathcal{B} \rightarrow \mathcal{B}'$ along P and P' is a functor $G : \mathcal{E} \rightarrow \mathcal{E}'$ making the diagram commute:

$$\begin{array}{ccc} \mathcal{E} & \xrightarrow{G} & \mathcal{E}' \\ P \downarrow & & \downarrow P' \\ \mathcal{B} & \xrightarrow{F} & \mathcal{B}' \end{array}$$

The lifting is called *fibred* if $\langle F, G \rangle$ is a morphism of fibrations (i.e. G preserves all cartesian morphisms). When P and P' have split cleavages, the lifting G is called *split* if $\langle F, G \rangle$ is a morphism of split fibrations.

A morphism $G \rightarrow H$ between two liftings G and H of the same functor F is a natural transformation $\sigma : G \rightarrow H$ that is vertical, i.e. $P'\sigma = \text{id}_{F \circ P}$:

$$\begin{array}{ccc} \mathcal{E} & \xrightarrow{G} & \mathcal{E}' \\ & \Downarrow \sigma & \\ \mathcal{E} & \xrightarrow{H} & \mathcal{E}' \\ P \downarrow & & \downarrow P' \\ \mathcal{B} & \xrightarrow{F} & \mathcal{B}' \end{array}$$

Liftings of F along P and P' and morphisms between them form a category $\text{LIFT}_{P,P'}(F)$, whose identity morphisms are the identity natural transformations, and composition is vertical composition of natural transformations.

7.1*13 (Modular models as liftings). Using the language of fibrations, we now have a very concise alternative formulation of modular models: a (strong/strict) modular model M of $\ddot{\Psi} \in \mathcal{F}$ is just a (fibred/split) lifting of the endofunctor $(- + \ddot{\Psi}) : \mathcal{F} \rightarrow \mathcal{F}$ along the fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ from 7.1*10:

$$\begin{array}{ccc} \mathcal{F}\text{-ALG} & \xrightarrow{M} & \mathcal{F}\text{-ALG} \\ P \downarrow & & \downarrow P \\ \mathcal{F} & \xrightarrow{- + \ddot{\Psi}} & \mathcal{F} \end{array} \quad (34)$$

The commutativity of the square ensures that the functor M maps every object $\langle \dot{\Sigma}, A, \alpha \rangle$ to an object $\langle \dot{\Sigma} + \ddot{\Psi}, B, \beta \rangle$. Below, we first repeat Example 7.1*8 in the fibrational formulation, and then we show that the fibrational formulation of modular models is equivalent to the formulation based on indexed categories (Theorem 7.1*17).

7.1*14 Example. The modular model of exception throwing Et_E in Example 7.1*8 corresponds to a functor lifting as follows. The operation family $\text{ALG}(\mathcal{E})$ of algebraic operations (6*5) determines a fibration $P : \text{ALGMod}(\mathcal{E}) \rightarrow \text{ALG}(\mathcal{E})$; if we follow our notation $\mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ above, the total category here ought to be called ‘ $\text{ALG}(\mathcal{E})\text{-ALG}$ ’ but this is obviously a horrible name so we rename the total category to $\text{ALGMod}(\mathcal{E})$. Every object of $\text{ALGMod}(\mathcal{E})$ is a tuple $\langle \langle \dot{\Sigma}, T_\Sigma \rangle, \langle A, [\alpha, \eta^A, \mu^A] \rangle \rangle$, where $\dot{\Sigma}$ is an equational system of the form ‘monoids with an algebraic operation satisfying some equation’

$$(S \square -)\text{-MON} \uparrow (\text{K}_G \vdash L = R) \text{ for some } S, G, L, \text{ and } R;$$

$T_\Sigma : \text{MON} \rightarrow \dot{\Sigma}$ is the inclusion translation; and

$$\langle A, [\alpha : S \square A \rightarrow A, \eta^A : I \rightarrow A, \mu^A : A \square A \rightarrow A] \rangle$$

is an algebra of $\dot{\Sigma}$. The endofunctor $(- + \text{Et}_E) : \text{ALG}(\mathcal{E}) \rightarrow \text{ALG}(\mathcal{E})$ sends every object $\langle \dot{\Sigma}, T_\Sigma \rangle$ as above to $\langle \dot{\Sigma} \uparrow (\coprod_E 1), T' \rangle$ where T' is again the inclusion translation.

The lifting $M : \text{ALGMod}(\mathcal{E}) \rightarrow \text{ALGMod}(\mathcal{E})$ of $- + \text{Et}_E$ corresponding to Example 7.1*8 then maps every object $\langle \langle \dot{\Sigma}, T_\Sigma \rangle, \langle A, [\alpha, \eta^A, \mu^A] \rangle \rangle$ to the object

$$\langle \langle \dot{\Sigma} \uparrow (\coprod_E 1), T' \rangle, \langle C_A, [\alpha^\#, \beta, \eta^C, \mu^C] \rangle \rangle$$

where the new carrier C_A and the operations $[\alpha^\#, \beta, \eta^C, \mu^C]$ are defined in Example 7.1*8.

7.1*15 Lemma. Given two fibrations $P : \mathcal{T} \rightarrow \mathcal{B}$, $P' : \mathcal{T}' \rightarrow \mathcal{B}'$, and a functor $F : \mathcal{B} \rightarrow \mathcal{B}'$, let $F^*P' : F^*\mathcal{T}' \rightarrow \mathcal{B}$ be the change of base of P' along F .

- (1) Liftings of F along P and P' are in bijection with $\text{CAT}/\mathcal{B}(P, F^*P')$, the hom-set between P and $F^*P' : \mathcal{T} \rightarrow \mathcal{B}$ in the slice category of CAT over $\mathcal{B}$.
- (2) Fibred liftings of F along P and P' are in bijection with $\text{FIB}_{\mathcal{B}}(P, F^*P')$, the set of fibred functors from P to F^*P' over the base category $\mathcal{B}$.
- (3) If P and P' are equipped with split cleavages, split liftings of F along P and P' are in bijection with $\text{FIB}_{\mathcal{B}}^s(P, F^*P')$, the set of split fibred functors from P to F^*P' over the base category $\mathcal{B}$.

Proof. By definition, F^*P' is the pullback in the following diagram:

$$\begin{array}{ccccc}
 \mathcal{T} & & & G & \\
 \downarrow H & & & \searrow & \\
 F^*\mathcal{T} & \xrightarrow{K} & \mathcal{T}' & & \\
 \downarrow F^*P' & \lrcorner & \downarrow P' & & \\
 \mathcal{B} & \xrightarrow{F} & \mathcal{B}' & & \\
 \uparrow P & & & &
 \end{array}$$

By definition, liftings of F are functors $G : \mathcal{T} \rightarrow \mathcal{T}'$ such that $F \circ P = P' \circ G$, so by the universal property of the pullback, liftings G are in bijection with functors $H : \mathcal{T} \rightarrow F^*\mathcal{T}'$ such that $(F^*P') \circ H = P$, and the backward direction is given by $H \mapsto K \circ H$. This is the first item in the statement.

This bijection cuts down to fibred (resp. split) liftings. The functor K always preserves cartesian morphisms and the split cleavage. Hence from one direction, if H preserves cartesian morphisms (resp. split cleavage), then so does $K \circ H$. From the other direction, if G preserves cartesian morphisms (resp. split cleavage), then the functor H sends a cartesian morphism f in $\mathcal{T}$ to the pair $\langle Pf, Gf \rangle$, which is cartesian for the fibration F^*P' . $\square$

7.1*16 Lemma. For a category $\mathcal{C}$ and two functors $F, G : \mathcal{C}^{\text{op}} \rightarrow \text{CAT}$, denote the Grothendieck construction of F and G by $p : \int F \rightarrow \mathcal{C}$ and $q : \int G \rightarrow \mathcal{C}$. Oplax transformations $F \rightarrow G$ are in bijection with $\text{CAT}/\mathcal{C}(p, q)$.

Proof. For one direction, given an oplax transformation $\alpha : F \rightarrow G$, we define a functor $K : \int F \rightarrow \int G$ as follows. On objects, K sends every object $\langle I \in \mathcal{C}, a \in FI \rangle$ to $\langle I, \alpha_I a \in GI \rangle$. On morphisms, K sends every morphism

$$\langle f : I \rightarrow J, g : a \rightarrow (Ff)b \rangle : \langle I, a \rangle \rightarrow \langle J, b \rangle$$

in $\int F$ to $\langle f, g' \rangle$ where g' is the following morphism in the fiber category G_I :

$$\alpha_I a \xrightarrow{\alpha_I g} \alpha_I (Ff)b \xrightarrow{\alpha_{\langle f, b \rangle}} (Gf)\alpha_I b.$$

The functor preserves identities and composition following the unity and composition axioms of oplax transformations.

For the other direction, given a functor $K : \int F \rightarrow \int G$ with $q \circ K = p$, we define an oplax transformation $\alpha : F \rightarrow G$ as follows. For every object $I \in \mathcal{C}$, we define the functor $\alpha_I : FI \rightarrow GI$ to be K restricted to the fiber category of $\int F$ over I . For every $f : I \rightarrow J$ in $\mathcal{C}$, we need to define a natural transformation $\alpha_f : \alpha_I \circ Ff \rightarrow Gf \circ \alpha_J : FI \rightarrow GI$. For each object $b \in FJ$, there is a morphism $\langle f, id \rangle : \langle I, (Ff)b \rangle \rightarrow \langle J, b \rangle$ in the total category $\int F$, and this morphism is mapped by K to some $\langle f, g \rangle : \langle I, \alpha_I(Ffb) \rangle \rightarrow \langle J, \alpha_J b \rangle$, we define the natural transformation α_f to be

$$\alpha_{\langle f, b \rangle} := g : \alpha_I(Ffb) \rightarrow Gf(\alpha_J b)$$

in the fiber category FI . For every $h : b \rightarrow b'$ in FJ , the naturality of α_f follows from the fact that the diagram below left commutes in fF :

$$\begin{array}{ccccc}
 & & \langle I, \alpha_I(Ff)b \rangle & \xrightarrow{K\langle f, id \rangle} & \langle J, \alpha_J b \rangle \\
 & & \downarrow \langle id, \alpha_f b \rangle & \searrow \langle f, id \rangle & \downarrow \langle id, \alpha_J h \rangle \\
 \langle I, Ff b \rangle & \xrightarrow{\langle f, id \rangle} & \langle J, b \rangle & & \\
 \downarrow \langle id, Ff h \rangle & & \downarrow \langle id, h \rangle & & \\
 \langle I, Ff b' \rangle & \xrightarrow{\langle f, id \rangle} & \langle J, b' \rangle & & \\
 & & \downarrow \langle id, \alpha_f b' \rangle & \searrow \langle f, id \rangle & \\
 & & \langle I, \alpha_I(Ff)b' \rangle & \xrightarrow{\langle id, Gf(\alpha_J h) \rangle} & \langle J, \alpha_J b' \rangle \\
 & & \downarrow \langle id, \alpha_f b' \rangle & \searrow \langle f, id \rangle & \\
 & & \langle I, Gf \alpha_J b \rangle & \xrightarrow{K\langle f, id \rangle} & \langle J, \alpha_J b' \rangle \\
 & & \downarrow \langle id, \alpha_f b' \rangle & \searrow \langle f, id \rangle & \\
 & & \langle I, Gf \alpha_J b' \rangle & \xrightarrow{\langle f, id \rangle} & \langle J, \alpha_J b' \rangle
 \end{array}$$

and K maps this diagram to fG , which is the back square in the diagram above right. We observe that in the diagram above right the two triangles commute by the definition of $\alpha_f b$ and $\alpha_f b'$; the right square commutes by the definition of arrow composition in fG . Hence the following diagram commutes:

$$\begin{array}{ccccc}
 \langle I, \alpha_I(Ff)b \rangle & & \xrightarrow{\langle id, \alpha_f b \rangle} & & \langle I, Gf \alpha_J b \rangle \\
 \downarrow \langle id, \alpha_I(Ff)h \rangle & & & & \downarrow \langle id, Gf \alpha_J h \rangle \\
 \langle I, \alpha_I(Ff)b' \rangle & & \xrightarrow{\langle id, \alpha_f b' \rangle} & & \langle I, Gf \alpha_J b' \rangle \\
 & & & & \searrow \langle f, id \rangle \\
 & & & & \langle J, \alpha_J b' \rangle
 \end{array}$$

The square on the left is exactly the naturality square for α_f that we need, which also commutes because the morphism $\langle f, id \rangle$ on the right is cartesian. It can be checked that this natural transformations satisfies the axioms of oplax transformations and that the two directions define a bijection. $\square$

7.1*17 Theorem. For every $\ddot{\Psi} \in \mathcal{F}$, the following are in bijection with each other:

- (1) modular models (resp. strong or strict modular models) as in Definition 7.1*5,
- (2) functors in $\text{CAT}/\mathcal{F}(P, Q)$ (resp. morphisms of fibrations or split fibrations from P to Q) where the split fibrations P and Q are as in (33), and
- (3) liftings (resp. fibred or split liftings) of the endofunctor $(- + \ddot{\Psi}) : \mathcal{F} \rightarrow \mathcal{F}$ along the fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$.

Proof. The bijection between 2 and 3 is Lemma 7.1*15. The bijection between modular models and morphisms in $\text{CAT}/\mathcal{F}$ is Lemma 7.1*16. The bijection between strong (resp. strict) modular models – which are exactly strong transformations (resp. natural transformations) between CAT -valued functors – and morphisms of fibrations (resp. split fibrations) is standard (Jacobs 1999, §1.10). $\square$

7.1*18 Notation. In the future, we will leave implicit the conversion between modular models as oplax transformations and as liftings of functors, so we may say ‘a modular model $M : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}\text{-ALG}$ of $\ddot{\Psi}$ ’.

7.1*19. One advantage of the fibrational formulation is that we avoid the need of a category of categories CAT bigger than the base category. Also, it reduces the 2-categorical notion of oplax transformations to the 1-categorical notion of functors. Consequently, 3-categorical

concepts can be avoided when talking about transformations of modular models, such as homomorphisms between modular models and the concept of *updaters* below.

7.1*20 Definition. Let M be a modular model of $\check{\Psi} \in \mathcal{F}$ given in the lifting form. An *updater* u for M is a natural transformation $u : \text{Id} \rightarrow M$ such that

$$(P \circ u) = (\iota_1 \circ P) : P \rightarrow (P \circ M) = (- + \check{\Psi}) \circ P$$

where $\iota_1 : (-) \rightarrow (-) + \check{\Psi}$ is the coprojection in $\mathcal{F}$:

$$\begin{array}{ccc} & \text{Id} & \\ & \downarrow u & \\ \mathcal{F}\text{-ALG} & \xrightarrow{\quad} & M \xrightarrow{\quad} \mathcal{F}\text{-ALG} \\ P \downarrow & & \downarrow P \\ \mathcal{F} & \xrightarrow{\quad} & \mathcal{F} \\ & \uparrow \iota_1 & \\ & (-) & \end{array}$$

7.1*21. For an object $\langle \check{\Sigma}, A, \alpha \rangle \in \mathcal{F}\text{-ALG}$, the component of u at this object is a $\check{\Sigma}$ -homomorphism from $\langle A, \alpha \rangle$ to the algebra $M\langle \check{\Sigma}, A, \alpha \rangle$ forgetting the $\check{\Psi}$ -algebra. In the setting of Example 7.1*7, updaters correspond to the *lifting operation* $l : \text{Id} \rightarrow T : \text{MON}(\mathcal{E}) \rightarrow \text{MON}(\mathcal{E})$ of monad/monoid transformers (Jaskelioff and Moggi 2010). We have switched to the terminology *lifting* to avoid the confusion with liftings along fibrations (Definition 7.1*12).

7.1*22 Example. Continuing Example 7.1*14, the modular model defined there has an updater $\text{Id} \rightarrow M$ whose component at the object $\langle \check{\Sigma}, \langle A, [\alpha, \eta^A, \mu^A] \rangle \rangle$ is the morphism

$$\langle \iota_1 : \check{\Sigma} \rightarrow \check{\Sigma} + \text{Et}_E, \quad A \circ \iota_2 : A \rightarrow C_A \rangle \text{ in } \text{ALGMod}(\mathcal{E}),$$

where ι_1 is the inclusion translation and $A \circ \iota_2 : A = A \circ \text{Id} \rightarrow A \circ (\coprod_E + \text{Id}) = C_A$ is a $\check{\Sigma}$ -algebra homomorphism. The morphism $A \circ \iota_2$ is exactly the ‘lifting operation’ of the exception monad transformer, but a lifting operation of monad transformers is only required to be a monad homomorphism, while here the updater additionally needs to be a homomorphism for the operation α too, which can be seen by inspecting the definition of $\alpha^\#$ in Example 7.1*8.

7.1*23. Assuming objects of $\mathcal{F}$ have initial algebras, the ‘dependently typed’ mapping sending every $\langle \check{\Sigma}, T_\Sigma \rangle \in \mathcal{F}$ to its initial algebra $\mu\check{\Sigma}$ can be conveniently formulated as a functor $(-)^* : \mathcal{F} \rightarrow \mathcal{F}\text{-ALG}$ using the fibrational language:

$$\begin{aligned} \check{\Sigma}^* &= \langle \check{\Sigma}, T_\Sigma, \mu\check{\Sigma}, \alpha^\Sigma : \Sigma(\mu\check{\Sigma}) \rightarrow \mu\check{\Sigma} \rangle \\ (T : \check{\Sigma} \rightarrow \check{\Psi})^* &= \langle T, ! : \langle \mu\check{\Sigma}, \alpha^\Sigma \rangle \rightarrow T\langle \mu\check{\Psi}, \alpha^\Psi \rangle \rangle \end{aligned} \quad (35)$$

where $!$ is the unique $\check{\Sigma}$ -homomorphism out of the initial algebra $\mu\check{\Sigma}$.

Let M be a modular model of some $\check{\Psi} \in \mathcal{F}$ in the lifting form. For every $\check{\Sigma} \in \mathcal{F}$, we have a unique $(\check{\Sigma} + \check{\Psi})$ -homomorphism out of the initial algebra $(\check{\Sigma} + \check{\Psi})^*$:

$$h_{\check{\Sigma}}^M : (\check{\Sigma} + \check{\Psi})^* \rightarrow M\check{\Sigma}^*, \quad (36)$$

which defines a natural transformation

$$h^M : (- + \check{\Psi})^* \rightarrow M(-)^* : \mathcal{F} \rightarrow \mathcal{F}\text{-ALG}. \quad (37)$$

This is how M modularly handles/interprets $\check{\Psi}$ -operations in programs $(\check{\Sigma} + \check{\Psi})^*$ with both $\check{\Psi}$ -operations and some other $\check{\Sigma}$ -operations, leaving operations from the other theory $\check{\Sigma}$ uninterpreted. Specially, $\check{\Sigma}$ can be the initial object $\langle \text{MON}, \text{Id} \rangle$ of $\mathcal{F}$, whose initial algebra is

the initial monoid I . In this case, the morphism $h_{(\text{MON}, \text{Id})}^M : \Psi^\star \rightarrow MI$ interprets the abstract syntax $\Psi^\star$ with no ‘residual operations’.

7.2 Model transformers

7.2*1. In the previous section, we have seen modular models of theories of monoids with operations as liftings of endofunctors $- + \Psi$ along a fibration P :

$$\begin{array}{ccc} \mathcal{F}\text{-ALG} & \xrightarrow{M} & \mathcal{F}\text{-ALG} \\ P \downarrow & & \downarrow P \\ \mathcal{F} & \xrightarrow{- + \Psi} & \mathcal{F} \end{array} \quad (38)$$

But there is no reason that the idea of modular models is specific to coproducts $- + \Psi$ or the fibration $\mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ for monoids with operations, since we may be interested in ways of combining algebraic theories other than coproducts, and we may be interested in other fibrations of theories and algebras. Thus we shall just study liftings of arbitrary functors along two possibly different fibrations. In this section, we justify the generalisation by showing a few more instances of liftings along fibrations related to modularity.

7.2*2 (Commutative combination). Consider $\mathcal{C} = \langle \text{ENDO}_K(\mathcal{C}), \circ, \text{Id} \rangle$ for some $\mathcal{C}$ that is lkp as a cartesian closed category, e.g. ωCPO or SET . For functors $A_1, A_2 \in \text{ENDO}_K(\mathcal{C})$, we had seen the *Day tensor product* $A_1 \otimes A_2$ in [Section 2.6](#):

$$(A_1 \otimes A_2)n = \int^{m, k \in \mathcal{C}_K} A_1 m \times A_2 k \times n^{m \times k},$$

which is intuitively a pair of A_1 -operation and A_2 -operation that do not depend on each other. This is clearly symmetric, so we have an isomorphism

$$s : A_1 * A_2 \cong A_2 * A_1.$$

Also, there is a canonical morphism $i : A_1 * A_2 \rightarrow A_1 \circ A_2$ as shown in [2.6*3](#).

We define a functor $\otimes : \text{ALG}(\mathcal{C}) \times \text{ALG}(\mathcal{C}) \rightarrow \text{ALG}(\mathcal{C})$ as follows, which is closely related to the *commutative combination*, also known as the *tensor*, of enriched algebraic theories ([Hyland et al. 2006](#)). Let $\check{\Sigma}_i \in \text{ALG}(\mathcal{C})$ be

$$\langle (A_i \square -)\text{-MON} \uparrow (K_{B_i} \vdash L_i = R_i), T_i \rangle, \text{ for } i = 1, 2.$$

We define $\check{\Sigma}_1 \otimes \check{\Sigma}_2$ to be $\check{\Sigma}_1 + \check{\Sigma}_2$ extended with the following (constant) equation

$$o : A_1 * A_2 \vdash f[i \ o] = f[i \ (s \ o)] : \tau$$

where the term f is defined by

$$o' : A_1 \circ A_2 \vdash \mathbf{let} \ (a_1, a_2) = o' \ \mathbf{in} \ \mu \ (op_1 \ (a_1, \eta \ *), op_2 \ (a_2, \eta \ *)) : \tau,$$

and $op_i : A_i \circ \tau \rightarrow \tau, \eta : I \rightarrow \tau, \mu : \tau \circ \tau \rightarrow \tau$ are respectively the algebraic operations, the unit, and the multiplication of the monoid.

Informally, a model of $\check{\Sigma}_1 \otimes \check{\Sigma}_2$ is a monad with an A_1 -operation and an A_2 -operation such that the order of an adjacent pair of an A_1 -operation and an A_2 -operation can be swapped:

$$\mathbf{do} \ \{x \leftarrow op_1; y \leftarrow op_2; k \ x \ y\} = \mathbf{do} \ \{y \leftarrow op_2; x \leftarrow op_1; k \ x \ y\}.$$

The Day tensor $A_1 * A_2$ shows up in the equation to ensure that each of the two operations do not depend on the other’s output, so that it makes sense to swap them. The functor $\otimes$ can be

extended to a monoidal product on $\text{ALG}(\mathcal{C})$ with the theory of monoids with no operations as the unit. Let $\mathcal{F}$ be $\text{ALG}(\mathcal{C})$ and $\check{\Psi} \in \mathcal{F}$, a lifting of $- \otimes \check{\Psi}$ along $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ is then a modular model of $\check{\Psi}$ such that operations of $\check{\Psi}$ commute with all other operations, even if they may be unknown now. This may sound very strong, but the *state monad transformer* gives such a modular model (Yang and Wu 2021, §6).

7.2*3 (Modular handlers). The theory of modular models is inspired by the concept of *modular handlers* studied by Schrijvers et al. (2019) and Yang and Wu (2021) in Haskell. If we re-express them in category theory, a reasonable definition of a *modular handler of an equational system $\check{\Sigma}$ over a category $\mathcal{C}$* is a mapping (not necessarily a functor) from monads M on $\mathcal{C}$ to tuples $\langle A, a, f \rangle$ of an object $A \in \mathcal{C}$, a $\check{\Sigma}$ -algebra $a : \Sigma A \rightarrow A$ on A , and an Eilenberg-Moore algebra of M $f : MA \rightarrow A$ on A :

$$\Sigma A \xrightarrow{a} A \xleftarrow{f} MA$$

The idea is similar to our modular models: for whatever ‘ambient effect’ M , there is a model A of the effect $\check{\Sigma}$, which also models any effect that M supports. In particular, if M has an algebraic operation $b : S \circ M \rightarrow M$ on M , then the object A can ‘forward’ this operation by

$$SA \xrightarrow{S\eta_A} S(MA) \xrightarrow{b} MA \xrightarrow{f} A.$$

Modular handlers in this sense are also an instance of liftings along fibrations. Let $|\text{MND}(\mathcal{C})|$ be the *discrete* category of monads over $\mathcal{C}$, then the identity functor $\text{Id} : |\text{MND}(\mathcal{C})| \rightarrow |\text{MND}(\mathcal{C})|$ is a fibration. For every $\check{\Sigma} \in \text{Eqs}(\mathcal{C})$, we define the functor $T_{\check{\Sigma}} : |\text{MND}(\mathcal{C})| \rightarrow \text{Eqs}(\mathcal{C})$ to be $(-)\text{-ACT} + \check{\Sigma}$, where $(-)\text{-ACT}$ is the equational system of monad algebras from 3.1*8. Let $P : \text{ALG}(\mathcal{C}) \rightarrow \text{Eqs}(\mathcal{C})$ be the Grothendieck construction of $(-)\text{-ALG} : \text{Eqs}(\mathcal{C})^{\text{op}} \rightarrow \text{CAT}$. A modular handler H of $\check{\Sigma}$ is then a lifting of $T_{\check{\Sigma}}$ along Id , P :

$$\begin{array}{ccc} |\text{MND}(\mathcal{C})| & \xrightarrow{H} & \text{ALG}(\mathcal{C}) \\ \text{Id} \downarrow & & \downarrow P \\ |\text{MND}(\mathcal{C})| & \xrightarrow{T_{\check{\Sigma}}} & \text{Eqs}(\mathcal{C}) \end{array}$$

In fact, most examples of modular handlers from Schrijvers et al. (2019) and Yang and Wu (2021) are covariant functors $\text{MND}(\mathcal{C}) \rightarrow \text{ALG}(\mathcal{C})$, apart from those based on *continuation monad transformers* $M \mapsto (- \Rightarrow MR) \Rightarrow MR$.

7.2*4 (Output effects). An intuition for effect handlers is that they *consume* effectful operations, but in almost all implementations of effect handlers, handlers can also *produce* effectful operations. For example, a handler may handle an exception by producing a nondeterministic failure, thus transforming the effect of exceptions to the effect of nondeterminism. Such handlers with both *input effect* (the operations to be handled) and *output effect* (the operations to be generated) can be modelled as liftings along fibrations as well.

Let $\mathcal{F}$ be an operation family (Definition 6*3) closed under binary coproducts. For every $\check{\Psi} \in \mathcal{F}$, we denote by $U_{\check{\Psi}} : \check{\Psi}/\mathcal{F} \rightarrow \mathcal{F}$ the forgetful functor from the coslice category under $\check{\Psi} \in \mathcal{F}$ to $\mathcal{F}$. We also denote by $P_{\check{\Psi}} : (\check{\Psi}/\mathcal{F})\text{-ALG} \rightarrow \check{\Psi}/\mathcal{F}$ the change of base of the

fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ along the functor U_{Ψ} :

$$\begin{array}{ccc} (\check{\Psi}/\mathcal{F})\text{-ALG} & \longrightarrow & \mathcal{F}\text{-ALG} \\ P_{\check{\Psi}} \downarrow & \lrcorner & \downarrow P \\ \check{\Psi}/\mathcal{F} & \xrightarrow{U_{\check{\Psi}}} & \mathcal{F} \end{array}$$

For every pair of $\check{\Sigma}, \check{\Psi} \in \mathcal{F}$, we define a functor $T_{\check{\Sigma}, \check{\Psi}} : \check{\Sigma}/\mathcal{F} \rightarrow \check{\Psi}/\mathcal{F}$

$$T_{\check{\Sigma}, \check{\Psi}} \langle \check{\Phi} \in \mathcal{F}, S : \check{\Sigma} \rightarrow \check{\Phi} \rangle = \langle \check{\Phi} + \check{\Psi}, \iota_2 : \check{\Psi} \rightarrow \check{\Phi} + \check{\Psi} \rangle$$

A modular model M of input effect $\check{\Psi}$ and output effect $\check{\Sigma}$ is then a lifting of the functor $T_{\check{\Sigma}, \check{\Psi}}$ along the fibrations $P_{\check{\Sigma}}$ and $P_{\check{\Psi}}$:

$$\begin{array}{ccc} (\check{\Sigma}/\mathcal{F})\text{-ALG} & \xrightarrow{M} & (\check{\Psi}/\mathcal{F})\text{-ALG} \\ P_{\check{\Sigma}} \downarrow & & \downarrow P_{\check{\Psi}} \\ \check{\Sigma}/\mathcal{F} & \xrightarrow{T_{\check{\Sigma}, \check{\Psi}}} & \check{\Psi}/\mathcal{F} \end{array} \quad (39)$$

Note that it is $\check{\Psi}$ rather than $\check{\Sigma}$ that is the *input* effect, since $\check{\Psi}$ is the effect to be ‘handled’ by this modular model; this is similar to how translations of equational systems $\check{\Psi} \rightarrow \check{\Sigma}$ are functors $\check{\Sigma}\text{-ALG} \rightarrow \check{\Psi}\text{-ALG}$ from the opposite direction.

7.2*5. In the diagram (39), $P_{\check{\Psi}}$ is a pullback of $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ along $U_{\check{\Psi}}$, so functors M making (39) commute are in bijection with functors N making

$$\begin{array}{ccccc} (\check{\Sigma}/\mathcal{F})\text{-ALG} & \xrightarrow{N} & \mathcal{F}\text{-ALG} & & \\ P_{\check{\Sigma}} \downarrow & & \downarrow P & & \\ \check{\Sigma}/\mathcal{F} & \xrightarrow{T_{\check{\Sigma}, \check{\Psi}}} & \check{\Psi}/\mathcal{F} & \xrightarrow{U_{\check{\Psi}}} & \mathcal{F} \end{array}$$

commute. When $\check{\Sigma}$ is the theory of monoids with no operations, this recovers our earlier definition of modular models without output effects (38).

7.2*6. Moreover, the fibration $\mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ in 7.2*4 can be replaced by many other fibrations whose base category is a category of some notion of algebraic theories and total category is a category of pairs of a theory and its model. For example, let FPCAT be the category of small categories with finite products and finite-product-preserving functors; there is a functor $[-, \text{SET}] : \text{FPCAT}^{\text{op}} \rightarrow \text{CAT}$ sending every $\mathcal{C} \in \text{FPCAT}$ to the category of finite-product-preserving functors $\mathcal{C} \rightarrow \text{SET}$, which induces a fibration $P : \text{FPMOD} \rightarrow \text{FPCAT}$, and we can talk about modular models of finite-product theories by replacing $\mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ in 7.2*4 with this fibration. The same thing can be said for *generalised algebraic theories* (Cartmell 1986), *second-order algebraic theories* (Fiore and Mahmoud 2010), and so on for any framework of algebraic theories that has a fibration of models over theories and useful ways of combining theories such as coproducts.

7.2*7. Motivated by the above examples of liftings, we will use ‘model transformers’ as a suggestive synonym for liftings along fibrations. (We could similarly call the functor $T : \mathcal{T} \rightarrow \mathcal{T}'$ a ‘theory transformer’.) Incidentally, Hyland et al. (2006) have a concept of ‘operation transformers’, but they correspond to translations between algebraic theories rather than functors between the categories of theories.

7.2*8 Definition. Given two fibrations $P : \mathcal{A} \rightarrow \mathcal{T}$, $P' : \mathcal{A}' \rightarrow \mathcal{T}'$ and a functor $T : \mathcal{T} \rightarrow \mathcal{T}'$, a (strong/strict) *model transformer* M of T is defined to a (fibred/split) lifting of T along P and P' (Definition 7.1*12).

$$\begin{array}{ccc} \mathcal{A} & \xrightarrow{M} & \mathcal{A}' \\ P \downarrow & & \downarrow P' \\ \mathcal{T} & \xrightarrow{T} & \mathcal{T}' \end{array}$$

The category $\text{LIFT}_{P,P'}(T)$ of liftings will also be denoted by $\text{MOTR}(T)$.

Specially, model transformers of a coproduct functor $- + \Psi : \mathcal{T} \rightarrow \mathcal{T}$ will be called *modular models* of $\Psi \in \mathcal{T}$, reusing the terminology from the last section.

7.2*9. We can similarly generalise *updaters* (Definition 7.1*20) to general fibrations. It seems natural to require the two fibrations to be the same in this case. Given a fibration $P : \mathcal{A} \rightarrow \mathcal{T}$ and a pointed endofunctor $\langle T : \mathcal{T} \rightarrow \mathcal{T}, \eta^T : \text{Id} \rightarrow \mathcal{T} \rangle$, an *updater* for a model transformer of T along P is a natural transformation $u : \text{Id} \rightarrow M$ such that $P \circ u = \eta^T \circ P$:

$$\begin{array}{ccc} & \text{Id} & \\ & \downarrow u & \\ \mathcal{A} & \xrightarrow{M} & \mathcal{A} \\ P \downarrow & & \downarrow P \\ \mathcal{T} & \xrightarrow{T} & \mathcal{T} \\ & \uparrow \eta^T & \\ & \text{Id} & \end{array}$$

We denote by $\text{MOTR}_u(T)$ the category of model transformers of T equipped with an updater and whose morphisms $\langle M, u \rangle \rightarrow \langle N, v \rangle$ are morphisms $\sigma : M \rightarrow N$ that are compatible with the updaters: $v = \sigma \cdot u : \text{Id} \rightarrow N$.

8 Constructions of model transformers

8*1. The definition of model transformers is just a mathematical formulation of semantic modularity, so in this section we will have a look at some concrete examples and general constructions of model transformers. Each subsection below discusses a construction and is basically independent of each other:

- initial model transformers (Section 8.1),
- initial model transformers with an updater (Section 8.2),
- free model transformers (Section 8.3),
- modular models from monoids transformers (Section 8.4),
- limits and colimits of model transformers (Section 8.5),
- modular models in symmetric monoidal categories (Section 8.7).

Among these, the constructions in Section 8.4 and 8.7 are previously known constructions strengthened with consideration of equations. Other constructions are new contributions.

8.1 Initial model transformer

8.1*1. We begin with the *initial model transformer*. Let $P : \mathcal{A} \rightarrow \mathcal{T}$ be a fibration and $P' : \mathcal{A}' \rightarrow \mathcal{T}'$ be a fibration with a cleavage κ such that for every object $\Sigma \in \mathcal{T}'$, the fiber

category $\mathcal{A}'_\Sigma$ has a chosen initial object $\mu\Sigma$. Then for every functor $T : \mathcal{T} \rightarrow \mathcal{T}'$, we define a functor $0_T : \mathcal{A} \rightarrow \mathcal{A}'$ such that

- for every object $A \in \mathcal{A}$, $0_TA := \mu(TPA)$,
- for every morphism $f : A \rightarrow B \in \mathcal{A}$, $0_Tf := \kappa(\mu(TPB), TPf) \cdot v$, where the morphism $v : \mu(TPA) \rightarrow X$ is the unique morphism out of the initial object $\mu(TPA)$ of the fiber category $\mathcal{A}'_{TPA}$:

$$\begin{array}{ccc} & & \mu(TPA) \\ & & \downarrow v \\ A & \xrightarrow{f} & B \\ & & X \xrightarrow{\kappa(\mu(TPB), TPf)} \mu(TPB) \end{array}$$

$$PA \xrightarrow{Pf} PB \qquad TPA \xrightarrow{TPf} TPB$$

8.1*2 Theorem. *In the situation of 8.1*1, the functor $0_T : \mathcal{A} \rightarrow \mathcal{A}'$ is a model transformer of $T : \mathcal{T} \rightarrow \mathcal{T}'$ and is initial in the category $\text{Motr}(T)$.*

$$\begin{array}{ccc} \mathcal{A} & \xrightarrow{0_T} & \mathcal{A}' \\ P \downarrow & & \downarrow P' \\ \mathcal{T} & \xrightarrow{T} & \mathcal{T}' \end{array}$$

Moreover, when the cleavage κ of P' preserves the chosen initial objects up to isomorphism (resp. strictly), i.e. for all $f : \Gamma \rightarrow \Sigma \in \mathcal{T}'$, the domain of $\kappa(\mu\Sigma, f)$ is initial in $\mathcal{A}'_\Gamma$ (resp. exactly $\mu\Gamma$), 0_T is a strong (resp. strict) model transformer.

Proof. The functor 0_T satisfies $P' \circ 0_T = T \circ P$ by construction, so it is a model transformer by definition (Definition 7.2*8). Given any $H : \mathcal{A} \rightarrow \mathcal{A}'$ such that $P' \circ H = T \circ P$, for every $A \in \mathcal{A}$, HA and 0_TA are both in the fibre category $\mathcal{A}'_{TPA}$, so by the initiality of 0_TA , there is a *unique* vertical morphism $u_A : 0_TA \rightarrow HA$. To show that u_A is natural, consider every $f : A \rightarrow B$ and the morphism $\kappa(HB, TPf) : Y \rightarrow HB$, we have the following situation

$$\begin{array}{ccccc} 0_TA = \mu(TPA) & \xrightarrow{v} & X & \xrightarrow{\kappa(0_TB, TPf)} & \mu(TPB) = 0_TB \\ u_A \downarrow & & \downarrow h & & \downarrow u_B \\ HA & \xrightarrow{k} & Y & \xrightarrow{\kappa(HB, TPf)} & HB \\ P \downarrow & & \searrow Hf & & \downarrow P' \\ TPA & \xrightarrow{f} & & & TPB \end{array}$$

where $v : 0_TA \rightarrow X$ is the unique vertical morphism from $\mu(TPA)$ to X , and $h : X \rightarrow Y$ is the unique vertical morphism making the upper-right square commute, obtained from the cartesianess of $\kappa(HB, TPf) : Y \rightarrow HB$. Similarly, k is the unique vertical morphism satisfying $\kappa(HB, TPf) \cdot k = Hf$. Note that 0_Tf is exactly the upper path $\kappa(0_TB, TPf) \cdot v$, and the upper-left square commutes by the initiality of 0_TA . Hence we have the commutativity of the large rectangle, which is the naturality of $u : 0_T \rightarrow H$. This concludes the proof of the initiality of the model transformer 0_T in $\text{Motr}(T)$.

For the second part of the theorem, if the cleavage κ preserves initial algebras up to isomorphism, the object X in the diagram above is also initial among $\mathcal{A}'_{TPA}$, so the morphism

$v : 0_T A \rightarrow X$ is a vertical isomorphism, and $0_T f = \kappa(0_T B, T P f) \cdot v$ is also cartesian. The case for splitting fibrations is similar. $\square$

8.1*3 Example. Let $\mathcal{F}$ be an operation family of monoids such that $\check{\Sigma}\text{-ALG}$ has chosen initial algebras for all $\check{\Sigma} \in \mathcal{F}$. Applying the construction of initial model transformers to the situation of 7.1*13, where $T = - + \check{\Psi}$ for some $\check{\Psi} \in \mathcal{F}$, the model transformer 0_T then maps every $\langle \check{\Sigma}, A, \alpha \rangle$ to the initial algebra of $\check{\Sigma} + \check{\Psi}$, ignoring the ‘existing model’ $\langle A, \alpha \rangle$ of the ‘existing syntax’ $\check{\Sigma}$ completely. Therefore the initial model transformer does not have an updater (Definition 7.1*20) in general.

8.2 Initial updatable model transformers

8.2*1. Instead of completely ignoring the existing model, we can alternatively consider the *free model* of the new syntax over the existing model. As a special case, let $\mathcal{F}$ be an operation family closed under coproducts, for every $\check{\Sigma}, \check{\Psi} \in \mathcal{F}$, Theorem 3.2*13 provides a sufficient condition for the forgetful functor $U : (\check{\Sigma} + \check{\Psi})\text{-ALG} \rightarrow \check{\Sigma}\text{-ALG}$ to have a left adjoint $F : \check{\Sigma}\text{-ALG} \rightarrow (\check{\Sigma} + \check{\Psi})\text{-ALG}$, the idea is to define a model transformer of $\check{\Psi} \in \mathcal{F}$ by sending every $\langle \check{\Sigma}, A, \alpha \rangle \in \mathcal{F}\text{-ALG}$ to $F\langle A, \alpha \rangle$. Moreover, the unit of the adjunction $F \dashv U$ defines an updater. The universal property of the obtained model transformer is that it is the initial one in the category of model transformers with an updater (7.2*9), so we will call it the *initial updatable model transformer* for short.

8.2*2 Theorem. Let $P : \mathcal{A} \rightarrow \mathcal{T}$ be a fibration with a cleavage such that for every morphism $t : \Gamma \rightarrow \Sigma$ in $\mathcal{T}$, there is an adjunction $t_! \dashv t^* : \mathcal{A}_\Sigma \rightarrow \mathcal{A}_\Gamma$. Let $\langle T, \eta \rangle$ be a pointed endofunctor on $\mathcal{T}$. The category of $\text{MOTR}_u(T)$ of model transformers of T with an updater as defined in 7.2*9 has an initial object.

Proof. We define a model transformer $0_T^u : \mathcal{A} \rightarrow \mathcal{A}$ as follows. For every object $A \in \mathcal{A}$, we have a functor $\eta_!^{PA} : \mathcal{A}_{PA} \rightarrow \mathcal{A}_{TPA}$ left adjoint to $\eta_{PA}^* : \mathcal{A}_{TPA} \rightarrow \mathcal{A}_{PA}$, and we define $0_T^u A := \eta_!^{PA} A$. For every $f : A \rightarrow B \in \mathcal{A}$, let $n_B : B \rightarrow \eta_{PB}^* \eta_!^{PB} B$ be the unit of the adjunction $\eta_!^{PB} \dashv \eta_{PB}^*$. Since n_B is a morphism in $\mathcal{A}_{PB}$, it is vertical: $P n_B = id_{PB}$, and $P(n_B \cdot f) = P f$. Therefore there is a unique $h : A \rightarrow (P f)^* \eta_{PB}^* \eta_!^{PB} B$ making the upper square commute:

$$\begin{array}{ccc}
 A & \xrightarrow{f} & B \\
 \downarrow h & & \downarrow n_B \\
 (P f)^* \eta_{PB}^* \eta_!^{PB} B & \xrightarrow{\quad} & \eta_{PB}^* \eta_!^{PB} B \\
 \eta_{PA}^* (T P f)^* \eta_!^{PB} B & \xrightarrow{\quad} & (T P f)^* \eta_{PB}^* \eta_!^{PB} B \\
 & \searrow g & \downarrow \\
 & & \eta_!^{PB} B = 0_T^u B
 \end{array}
 \quad (40)$$

$$\begin{array}{ccc}
 PA & \xrightarrow{P f} & PB \\
 \searrow \eta_{PA} & & \searrow \eta_{PB} \\
 TPA & \xrightarrow{TP f} & TPB
 \end{array}$$

The unlabelled morphisms are the evident cartesian morphisms from the cleavage κ . By the naturality of $\eta : \text{Id} \rightarrow T$, we have the commutativity of the bottom square in $\mathcal{T}$: $\eta_{PB} \cdot Pf = TPf \cdot \eta_{PA}$. Reindexing is pseudofunctorial, so we have

$$(Pf)^* \eta_{PB}^* \eta_!^{PB} B \cong \eta_{PA}^* (TPf)^* \eta_!^{PB} B.$$

Hence the morphism h determines a morphism $A \rightarrow \eta_{PA}^* (TPf)^* \eta_!^{PB} B$, which by the adjunction $\eta_!^{PA} \dashv \eta_{PA}^*$, determines a vertical morphism

$$g : 0_T^u A = \eta_!^{PA} A \rightarrow (TPf)^* \eta_!^{PB} B.$$

By composing g with $\kappa(0_T^u B, TPf) : (TPf)^* \eta_!^{PB} B \rightarrow 0_T^u B$, we obtain a morphism $0_T^u A \rightarrow 0_T^u B$, which is our definition of the action of 0_T^u the morphism $f : A \rightarrow B$. We omit the checking of the functoriality of $0_T^u : \mathcal{A} \rightarrow \mathcal{A}$ here.

The functor 0_T^u by construction is a lifting of $T : \mathcal{T} \rightarrow \mathcal{T}$ along the fibration $P : \mathcal{A} \rightarrow \mathcal{T}$. It has an updater $u : \text{Id} \rightarrow 0_T^u$ given by

$$u_B := \kappa(\eta_{PB}, \eta_!^{PB} B) \cdot n_B : B \rightarrow \eta_!^{PB} B = 0_T^u B$$

for all $B \in \mathcal{A}$ as in the diagram (40) above. To see the naturality of u , for all $f : A \rightarrow B$, the two morphisms $u_B \cdot f$ and $0_T^u f \cdot u_A$ are over the same morphism in $\mathcal{T}$, i.e. $\eta_{PB} \cdot Pf = TPf \cdot \eta_{PA}$, so it is sufficient to show that their induced vertical morphisms in $\mathcal{A}_{PA}$ are equal. It can be calculated that the one for $u_B \cdot f$ is h as in (40) and the one for $0_T^u f \cdot u_A$ is

$$(\eta_{PA}^* g) \cdot n_A = \eta_{PA}^* (e_A \cdot \eta_!^{PA} h) \cdot n_A \quad (41)$$

where $e_A : \eta_!^{PA} \eta_{PA}^* A \rightarrow A$ is the counit of the adjunction $\eta_!^{PA} \dashv \eta_{PA}^*$. Using naturality and triangle identity of the unit n and counit e of the adjunction $\eta_!^{PA} \dashv \eta_{PA}^*$, the morphism (41) can be shown to be exactly h .

As for the initiality of $\langle 0_T^u, u \rangle$ in $\text{Motr}_u(T)$, for every $\langle M, v \rangle \in \text{Motr}_u(T)$ and $B \in \mathcal{A}$, $v_B : B \rightarrow MB$ is over η_{PB} by the definition of updaters. Hence there is a vertical morphism $w : B \rightarrow \eta_!^{PB} MB$ such that $v_B = \kappa(\eta_{PB}, MB) \cdot w$. By the universal property of $n_B : B \rightarrow \eta_{PB}^* \eta_!^{PB} B$, there is a *unique* morphism $\sigma_B : \eta_!^{PB} B = 0_T^u B \rightarrow MB$ such that $\eta_{PB}^* \sigma_B \cdot n_B = w$:

$$\begin{array}{ccc} B & \xrightarrow{n_B} & \eta_{PB}^* \eta_!^{PB} B \\ & \searrow w & \downarrow \eta_{PB}^* \sigma_B \\ & & \eta_{PB}^* MB \end{array}$$

We omit the checking of the naturality of σ here (it is similarly to the proof of Theorem 8.1*2). Since every $\tau : \langle 0_T^u, u \rangle \rightarrow \langle M, v \rangle$ is a vertical natural transformation $\tau : 0_T^u \rightarrow M$ satisfying $\tau_B \cdot u_B = v_B$, it must satisfy that $\eta_{PB}^* \tau_B \cdot n_B = w$. Therefore, σ is the unique morphism $\langle 0_T^u, u \rangle \rightarrow \langle M, v \rangle$ in $\text{Motr}_u(T)$. $\square$

8.2*3 Example. Let $\mathcal{C}$ be a category and $\mathcal{A}$ be a *freeness condition* for $\mathcal{C}$ (Definition 3.1*19) that satisfies the assumption of Theorem 3.2*13 (for example, preserving colimits of β -chains for some limit ordinal β and $\mathcal{C}$ being cocomplete). The fibration $P : \text{ALG}_{\mathcal{A}}(\mathcal{C}) \rightarrow \text{Eqs}_{\mathcal{A}}(\mathcal{C})$ then satisfies the condition of Theorem 8.2*2, so the pointed functor $(- + \dot{\Psi}) : \text{Eqs}_{\mathcal{A}}(\mathcal{C}) \rightarrow \text{Eqs}_{\mathcal{A}}(\mathcal{C})$ for every $\dot{\Psi} \in \text{Eqs}_{\mathcal{A}}(\mathcal{C})$ then has an initial updatable model transformer.

8.2*4 Example. Let $\mathcal{E}$ be a cocomplete monoidal category such that the monoidal product $\square : \mathcal{E} \times \mathcal{E} \rightarrow \mathcal{E}$ preserves colimits of β -chains for some limit ordinal β . Let $\mathcal{F} \subseteq \text{Mon}/\text{Eqs}(\mathcal{E})$

be the operation family containing all equational systems whose signature and context colimits of β -chains for some limit ordinal β . By Theorem 3.2*13, for every morphism $T : \tilde{\Sigma} \rightarrow \tilde{\Psi} \in \mathcal{F}$, the corresponding functor $T : \tilde{\Psi}\text{-ALG} \rightarrow \tilde{\Sigma}\text{-ALG}$ has a left adjoint. Therefore the fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ satisfies the condition of Theorem 8.2*2, and for every $\tilde{\Psi} \in \mathcal{F}$, the functor $- + \tilde{\Psi} : \mathcal{F} \rightarrow \mathcal{F}$ has an initial updater model transformer $0_{-+\tilde{\Psi}}^\mu$, which maps every $\tilde{\Sigma}$ -algebra A to the relative free $(\tilde{\Sigma} + \tilde{\Psi})$ -algebra over A .

8.2*5 Example. For a concrete example, let us instantiate $\mathcal{E}$ in the previous example to be $\langle \text{SET}^{\text{FIN}}, \bullet, V \rangle$ from 2.2*2. As we mentioned in 6*22, the syntax of untyped λ -calculus can be presented as an equational system $\Lambda = \Sigma_{\langle O, a \rangle}\text{-MON}$ for the binding signature $O = \{app, abs\}$ with $a(abs) = \langle 1 \rangle$ and $a(app) = \langle 0, 0 \rangle$. We may also include equations such as the β -law and η -law (Fiore and Hur 2009, §7.2.4). Models of Λ can be obtained from *reflexive objects* $U \cong U^U$ in any cartesian closed category C : every U induces a functor $\bar{U} : \text{FIN} \rightarrow \text{SET}$ with $n \mapsto \mathcal{C}(U^n, U)$. The functor $\bar{U}$ has a monoid structure $[\eta_U, \mu_U]$ (similar to that of the continuation monad), and it is a model of Λ (Hyland 2017):

$$\begin{aligned} abs_U : \bar{U}^V n &\cong \mathcal{C}(U^{n+1}, U) \cong \mathcal{C}(U^n, U^U) \cong \mathcal{C}(U^n, U) \cong \bar{U}n \\ app_U : (\bar{U} \times \bar{U})n &\cong \mathcal{C}(U^n, U \times U) \cong \mathcal{C}(U^n, U^U \times U) \xrightarrow{eval_{U^U \times U}} \mathcal{C}(U^n, U) \cong \bar{U}n \end{aligned}$$

Now consider the theory ST_S of mutable state (Example 5*11) for some finite set S . Its initial updatable model transformer maps the Λ -model on $\bar{U}$ to a $(\Lambda +_{\text{MON}} \text{ST}_S)$ -model whose carrier is the initial algebra

$$\mu X. \bar{U} + X \bullet X + V + X^V + X \times X + \prod_S X + \coprod_S X : \text{FIN} \rightarrow \text{SET}$$

quotiented by equations of Λ and ST_S and equations saying that Λ -operations of the initial algebra acting on $\bar{U}$ are the same as the model $[\eta_U, \mu_U, abs_U, app_U]$ of $\bar{U}$.

8.2*6. Left adjoints to reindexing functors of a fibration are used for modelling Σ -types and $\exists$ -quantification in categorical logic, where the fibration models types or predicates in a context of variables, and reindexing models substitution. In this context, reindexing functors typically preserve the left adjoints in a certain sense (the Beck-Chevalley condition), reflecting the fact substitution commutes with Σ and $\exists$. Dually, in this context reindexing functors usually have right adjoints, which model Π or $\forall$.

However, for fibrations of algebras and theories, the reindexing functors typically do *not* have right adjoints (the translation functors between categories of algebras almost never preserve colimits), and reindexing functors typically do *not* preserve the left adjoints (relative free algebras), so initial updatable model transformers typically are not strong or strict. For example, consider $\mathcal{E} = \langle \text{SET}, \times, 1 \rangle$. We have the following theories in the family $\text{MON}/\text{EQS}_{\mathcal{A}}(\mathcal{E})$: MON with the identity translation; GRP with the inclusion translation $T : \text{MON} \rightarrow \text{GRP}$ (Example 3.2*4); and the theory BLAT of bounded lattices with the translation that maps monoid multiplication to lattice join $\vee$, monoid identity to lattice bottom $\perp$. The following diagram in CAT does not commute:

$$\begin{array}{ccc} \text{MON-ALG} & \xleftarrow{T} & \text{GRP-ALG} \\ \text{F}_{\text{MON}} \downarrow & & \downarrow \text{F}_{\text{GRP}} \\ (\text{MON} + \text{BLAT})\text{-ALG} & \xleftarrow{T + \text{BLAT}} & (\text{GRP} + \text{BLAT})\text{-ALG} \end{array}$$

The theory $\text{MON} + \text{BLAT}$, bounded lattices whose join $\vee$ and $\perp$ form a monoid, is isomorphic to BLAT since $\langle \vee, \perp \rangle$ of a lattice is already a monoid, so $F_{\text{MON}}(TG)$ for every group G is the free bounded lattice over G as a monoid. On the other hand, the theory $\text{GRP} + \text{BLAT}$, bounded lattices whose $\langle \vee, \perp \rangle$ form a group, has only trivial models since for every element x , $\perp = x \vee x^{-1}$, and by the idempotent law of join, $\perp = (x \vee x) \vee x^{-1} = x \vee (x \vee x^{-1}) = x \vee \perp = x$. Therefore $(T + \text{BLAT})(F_{\text{GRP}}G)$ is always the trivial bounded lattice for every group G .

8.3 Free model transformers over ordinary models

8.3*1. Consider model transformers of $- + \check{\Psi} : \mathcal{F} \rightarrow \mathcal{F}$ on an operation family $\mathcal{F}$: for every $\check{\Sigma} \in \mathcal{F}$ and every $A \in \check{\Sigma}\text{-ALG}$, the initial model transformer $0_{\check{\Psi}}$ just ignores the algebra A and freely generates a model of $\check{\Sigma} + \check{\Psi}$, so it is not a practically interesting model transformer. In comparison, the initial updatable model transformer $0''_{\check{\Psi}}$ takes A into account and freely generates a model of $\check{\Sigma} + \check{\Psi}$ that has a $\check{\Sigma}$ -homomorphism from A .

The natural next step is then freely generating a model of $\check{\Sigma} + \check{\Psi}$ that has both a $\check{\Sigma}$ -homomorphism from A and a $\check{\Psi}$ -homomorphism from some fixed $B \in \check{\Psi}\text{-ALG}$, using the construction in Example 3.2*17. In this way, we can turn an ordinary model B of $\check{\Psi}$ to a model transformer of $\check{\Psi}$, and it is also going to be the *free* way.

8.3*2. In this subsection, we fix a fibration $P : \mathcal{A} \rightarrow \mathcal{T}$ with a cleavage such that $\mathcal{T}$ has finite coproducts, and we fix a functor $\oplus : \mathcal{T} \times \mathcal{T} \rightarrow \mathcal{T}$ equipped with a natural transformation $\tau : + \rightarrow \oplus$. For all $\Sigma, \Gamma \in \mathcal{T}$, we define

$$\kappa_1 := (\Sigma \xrightarrow{\iota_1} \Sigma + \Gamma \xrightarrow{\tau} \Sigma \oplus \Gamma) \quad \kappa_2 := (\Gamma \xrightarrow{\iota_2} \Sigma + \Gamma \xrightarrow{\tau} \Sigma \oplus \Gamma)$$

The natural transformation $\kappa_1 : - \rightarrow - \oplus \Gamma$ makes $- \oplus \Gamma$ a pointed functor, enabling us to talk about updaters (7.2*9) of the functor $- \oplus \Gamma$.

As usual, the category $\mathcal{T}$ is expected to be a category of some notion of algebraic theories and each fiber category $\mathcal{A}_\Sigma$ is the category of models of $\Sigma \in \mathcal{T}$. For example, P can be the fibration $\mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ for an operation family and $\oplus$ can be just the coproduct or the commutative combination (7.2*2).

8.3*3 Definition. When the fiber category $\mathcal{A}_0$ of the initial object $0 \in \mathcal{T}$ also has an initial object I , for every $\Gamma \in \mathcal{A}$ we define a functor $U_{\oplus\Gamma} : \text{MOTR}(- \oplus \Gamma) \rightarrow \mathcal{A}_\Gamma$:

$$U_{\oplus\Gamma} M := \kappa_2^* M I$$

where κ_2^* is the reindexing functor $\mathcal{A}_{0 \oplus \Gamma} \rightarrow \mathcal{A}_\Gamma$. The functor $U_{\oplus\Gamma}$ is intuitively the forgetful functor from model transformers of $(\oplus\text{-combination with}) \Gamma$ to ordinary models of Γ . Composing $U_{\oplus\Gamma}$ with the functor that forgets updaters (7.2*9), we also have a functor $\text{MOTR}_u(- \oplus \Gamma) \rightarrow \mathcal{A}_\Gamma$ that we shall also denote by $U_{\oplus\Gamma}$.

8.3*4 Theorem. Assume that all reindexing functors $!^* : \mathcal{A}_\Gamma \rightarrow \mathcal{A}_0$ to the fiber category over the initial object $0 \in \mathcal{T}$ is monadic and every fiber category $\mathcal{A}_\Sigma$ is finitely cocomplete. The functor $U_{\oplus\Gamma} : \text{MOTR}_u(- \oplus \Gamma) \rightarrow \mathcal{A}_\Gamma$ for every $\Gamma \in \mathcal{T}$ defined in Definition 8.3*3 has a left adjoint $F_{\oplus\Gamma} : \mathcal{A}_\Gamma \rightarrow \text{MOTR}_u(- \oplus \Gamma)$.

Proof. Every fibration is equivalent to a split one (Jacobs 1999, Corollary 5.2.5), and the statement is stable under equivalence of fibrations, so we can assume without loss of generality that P is a split fibration. By Mac Lane (1998, §IV.1 Theorem 2), it is sufficient to construct

for every $\langle B, \beta \rangle \in \mathcal{A}_\Gamma$ a model transformer with an updater $M^B \in \text{Motr}_u(- \oplus \Gamma)$ and a universal arrow $e : \langle B, \beta \rangle \rightarrow \text{U}_{\oplus \Gamma} M^B$.

First we observe that for every $\Sigma \in \mathcal{T}$, we have the following functors:

$$\begin{array}{ccc} \mathcal{A}_\Sigma \times \mathcal{A}_\Gamma & \xleftarrow{\langle \kappa_1^*, \kappa_2^* \rangle} & \mathcal{A}_{\Sigma \oplus \Gamma} \\ \downarrow !_\Sigma^* \times !_\Gamma^* & & \downarrow !_{\Sigma \oplus \Gamma}^* \\ \mathcal{A}_0 \times \mathcal{A}_0 & \xleftarrow{\Delta} & \mathcal{A}_0 \end{array}$$

This diagram commutes strictly since $\kappa_1 \cdot !_\Sigma = !_{\Sigma \oplus \Gamma} = \kappa_2 \cdot !_\Gamma : 0 \rightarrow \Sigma \oplus \Gamma$, and we have assumed that P is a split fibration. By the assumption that each fiber category is finitely cocomplete, the category $\mathcal{A}_0$ has binary coproducts, so we have an adjunction $+ \Delta : \mathcal{A}_0 \rightarrow \mathcal{A}_0 \times \mathcal{A}_0$, and moreover the fiber category $\mathcal{A}_{\Sigma \oplus \Gamma}$ has coequalisers. By assumption $!_\Sigma^*$, $!_\Gamma^*$, and $!_{\Sigma \oplus \Gamma}^*$ are monadic functors. The product of monadic functors is also monadic, so $!_\Sigma^* \times !_\Gamma^*$ is monadic. By [Borceux \(1994a, Theorem 4.5.6\)](#) (c.f. our discussion in [3.2*15](#)), the functor $\langle \kappa_1^*, \kappa_2^* \rangle$ on the top of the diagram has a left adjoint $F_{\Sigma \oplus \Gamma} : \mathcal{A}_\Sigma \times \mathcal{A}_\Gamma \rightarrow \mathcal{A}_{\Sigma \oplus \Gamma}$.

Now we define M^B via an oplax transformation by [Lemma 7.1*16](#):

$$M^B : \mathcal{A}_- \rightarrow \mathcal{A}_{-\oplus \Gamma} : \mathcal{T}^{\text{op}} \rightarrow \text{CAT}.$$

We define the component M_Σ^B at every $\Sigma \in \mathcal{T}$ to be $F_{\Sigma \oplus \Gamma} \langle -, B \rangle : \mathcal{A}_\Sigma \rightarrow \mathcal{A}_{\Sigma \oplus \Gamma}$. For every morphism $t : \Sigma \rightarrow \Phi$, we have the following functors:

$$\begin{array}{ccc} \mathcal{A}_\Phi \times \mathcal{A}_\Gamma & \xrightleftharpoons[\tau]{\langle \kappa_1^*, \kappa_2^* \rangle} & \mathcal{A}_{\Phi \oplus \Gamma} \\ t^* \times id \downarrow & & \downarrow (t \oplus id)^* \\ \mathcal{A}_\Sigma \times \mathcal{A}_\Gamma & \xrightleftharpoons[\tau]{\langle \kappa_1^*, \kappa_2^* \rangle} & \mathcal{A}_{\Sigma \oplus \Gamma} \\ & F_{\Sigma \oplus \Gamma} & \end{array}$$

By the naturality of κ , we have strict commutativity:

$$id : (t^* \times id) \circ \langle \kappa_1^*, \kappa_2^* \rangle = \langle t^* \kappa_1^*, \kappa_2^* \rangle = \langle \kappa_1^*, \kappa_2^* \rangle \circ (t \oplus id)^*,$$

which determines a canonical natural transformation

$$\tau : F_{\Sigma \oplus \Gamma} \circ (t^* \times id) \rightarrow (t \oplus id)^* \circ F_{\Phi \oplus \Gamma}$$

called the *mate* or the *conjugate* of id ([Mac Lane 1998, §IX.7](#)). Namely, τ is the transpose along the adjunction $F_{\Sigma \oplus \Gamma} \dashv \langle \kappa_1^*, \kappa_2^* \rangle$ of

$$(t^* \times id) \xrightarrow{\eta} \langle \kappa_1^*, \kappa_2^* \rangle \circ F_{\Sigma \oplus \Gamma} \circ (t^* \times id) \xrightarrow{id} \langle \kappa_1^*, \kappa_2^* \rangle \circ (t \oplus id)^* \circ F_{\Phi \oplus \Gamma}.$$

We define the 2-cell $M_t^B := (\tau \circ \langle \text{Id}, K_B \rangle) : M_\Sigma^B \circ t^* \rightarrow (t \oplus \Gamma)^* \circ M_\Phi^B$.

Now we define an updater $u : \text{Id} \rightarrow M^B$ for M^B . For every $A \in \mathcal{A}$, letting $\Sigma := PA$, we define u_A to be the composite

$$A \xrightarrow{\pi_1 \eta_{\langle A, B \rangle}} \kappa_1^* F_{\Sigma \oplus \Gamma} \langle A, B \rangle \xrightarrow{\overline{\kappa_1}} M^B A$$

where $\eta_{\langle A, B \rangle} : \langle A, B \rangle \rightarrow \langle \kappa_1^*, \kappa_2^* \rangle (F_{\Sigma \oplus \Gamma} \langle A, B \rangle)$ is the unit of the adjunction, $\overline{\kappa_1}$ is the cartesian morphism over κ_1 . The naturality of u is essentially a consequence of the naturality of η .

We have defined a model transformer M^B with an updater u for every $B \in \mathcal{A}_\Gamma$, and what remains is to define a universal arrow $e : B \rightarrow \text{U}_{\oplus \Gamma} \langle M^B, u \rangle$. By [Definition 8.3*3](#),

$U_{\oplus\Gamma}\langle M^B, u \rangle$ is $\kappa_2^*(F_{0\oplus\Gamma}\langle I, B \rangle) \in \mathcal{A}_\Gamma$. Therefore we define $e : B \rightarrow U_{\oplus\Gamma}\langle M^B, u \rangle$ to be the second projection of the unit

$$\eta_{\langle I, B \rangle} : \langle I, B \rangle \rightarrow \langle \kappa_1^*, \kappa_2^* \rangle(F_{0\oplus\Gamma}\langle I, B \rangle).$$

To show the universality of e , given any model transformer $\langle N, v \rangle \in \text{Motr}_u(- \oplus \Gamma)$ with a morphism $f : B \rightarrow U_{\oplus\Gamma}\langle N, v \rangle$ in $\mathcal{A}_\Gamma$, we need to show that there is a unique $\sigma : \langle M^B, u \rangle \rightarrow \langle N, v \rangle$ in $\text{Motr}(- \oplus \Gamma)$ such that $(U_{\oplus\Gamma}\sigma) \cdot e = f$.

First of all, the codomain of f is by definition $\kappa_2^*(NI)$, where I is the initial object of $\mathcal{A}_0$. It is not hard to see that the object I is also the initial object of the total category $\mathcal{A}$, so for every $A \in \mathcal{A}$, we have a morphism

$$\bar{f}_A := ((\kappa_2^*N!) \cdot f) : B \rightarrow \kappa_2^*NA.$$

Now recall that a morphism σ in $\text{Motr}_u(- \oplus \Gamma)$ is a vertical natural transformation $M^B \rightarrow N$ that commutes with the updaters u and v . For every $A \in \mathcal{A}$, letting $\Sigma := PA$, the updater v at A is a morphism $v_A : A \rightarrow \kappa_1^*(NA)$. Paired with $\bar{f}_A$, we have a morphism $\langle v_A, \bar{f}_A \rangle : \langle A, B \rangle \rightarrow \langle \kappa_1^*, \kappa_2^* \rangle(NA)$, and we define $\sigma_A : M^B A = F_{\Sigma\oplus\Gamma}\langle A, B \rangle \rightarrow NA$ to be the transpose of $\langle v_A, \bar{f}_A \rangle$ along the adjunction $F_{\Sigma\oplus\Gamma} \dashv \langle \kappa_1^*, \kappa_2^* \rangle$. We omit the verification of naturality here.

What remains is to show that σ defined above is the unique morphism $\langle M^B, u \rangle \rightarrow \langle N, v \rangle$ satisfying $(U_{\oplus\Gamma}\sigma) \cdot e = f$. First of all, σ satisfies this equation since by definition $U_{\oplus\Gamma}\sigma = \kappa_2^*\sigma_I$ and σ_I makes the following triangle commute

$$\begin{array}{ccc} \langle I, B \rangle & \xrightarrow{\eta_{\langle I, B \rangle}} & \langle \kappa_1^*, \kappa_2^* \rangle F_{I\oplus\Gamma}\langle I, B \rangle \\ & \searrow \langle v_I, \bar{f}_I \rangle & \downarrow \langle \kappa_1^*, \kappa_2^* \rangle \sigma_I \\ & & \langle \kappa_1^*, \kappa_2^* \rangle NI \end{array}$$

in the category $\mathcal{A}_0 \times \mathcal{A}_\Gamma$. The second projection of this commutativity diagram is exactly $(U_{\oplus\Gamma}\sigma) \cdot e = f$. For the uniqueness of σ , given another τ satisfying $(U_{\oplus\Gamma}\tau) \cdot e = f$, for every $\Sigma \in \mathcal{T}$ and $A \in \mathcal{A}_\Sigma$, the naturality of τ for the unique morphism $! : I \rightarrow A$ implies the commutativity of the right trapezium below:

$$\begin{array}{ccccc} B & \xrightarrow{e} & \kappa_2^*F_{I\oplus\Gamma}\langle I, B \rangle & \xrightarrow{\kappa_2^*M^B!} & \kappa_2^*F_{\Sigma\oplus\Gamma}\langle A, B \rangle \\ & \searrow f & \downarrow \kappa_2^*\tau_I & & \downarrow \kappa_2^*\tau_A \\ & & \kappa_2^*NI & \xrightarrow{\kappa_2^*N!} & \kappa_2^*NA \\ & \searrow \bar{f}_A & & & \uparrow \end{array} \quad (42)$$

Moreover, it can be shown that the following diagram in $\mathcal{A} \times \mathcal{A}$ commutes by expanding out the definition of the action of $M^B : \mathcal{A} \rightarrow \mathcal{A}$ on morphisms:

$$\begin{array}{ccc} \langle I, B \rangle & \xrightarrow{\eta_{\langle I, B \rangle}} & \langle \kappa_1^*, \kappa_2^* \rangle F_{0\oplus\Gamma}\langle I, B \rangle \\ \langle !, B \rangle \downarrow & & \downarrow \langle \kappa_1^*, \kappa_2^* \rangle M^B! \\ \langle A, B \rangle & \xrightarrow{\eta_{\langle A, B \rangle}} & \langle \kappa_1^*, \kappa_2^* \rangle F_{\Sigma\oplus\Gamma}\langle A, B \rangle \end{array}$$

Applying the second projection to this commutative diagram,

$$\pi_2\eta_{\langle A, B \rangle} = \pi_2(\eta_{\langle A, B \rangle} \cdot \langle !, B \rangle) = \pi_2(\langle \kappa_1^*, \kappa_2^* \rangle M^B ! \cdot \eta_{\langle I, B \rangle}) = \kappa_2^* M^B ! \cdot \pi_2\eta_{\langle I, B \rangle}.$$

Recall that e is exactly $\pi_2\eta_{\langle I, B \rangle}$, so the top horizontal path of the diagram (42) is equal to $\pi_2\eta_{\langle A, B \rangle}$. Then the diagram (42) implies that $\kappa_2^* \tau_A \cdot \pi_2\eta_{\langle A, B \rangle} = \bar{f}_A$. This, together with the fact τ is compatible with the updaters,

$$\begin{array}{ccc} A & \xrightarrow{u_A} & \kappa_1^* F_{\Sigma \oplus \Gamma} \langle A, B \rangle \\ & \searrow v_A & \downarrow \kappa_1^* \tau_A \\ & & \kappa_1^* N A \end{array}$$

implies that τ_I is the transpose of $\langle v_A, \bar{f}_A \rangle$ along $F_{\Sigma \oplus \Gamma} \dashv \langle \kappa_1^*, \kappa_2^* \rangle$, so $\tau_I = \sigma_I$. $\square$

8.3*5 Example. Let $\mathcal{C}$ be a cocomplete category and $\mathcal{A}$ be the freeness condition (3.1*19) containing all pairs of endofunctors that preserve colimits of β -chains for some limit ordinal β . By Theorem 3.1*10 and Theorem 3.2*13, the fibration $P : \text{ALG}_{\mathcal{A}}(\mathcal{C}) \rightarrow \text{Eqs}_{\mathcal{A}}(\mathcal{C})$ of algebras and equational systems in $\mathcal{A}$ then satisfies the assumptions of Theorem 8.3*4. For every $\tilde{\Psi} \in \text{Eqs}_{\mathcal{A}}(\mathcal{C})$, instantiating $\oplus$ to be $+$: $\text{Eqs}_{\mathcal{A}}(\mathcal{C}) \times \text{Eqs}_{\mathcal{A}}(\mathcal{C}) \rightarrow \text{Eqs}_{\mathcal{A}}(\mathcal{C})$, Theorem 8.3*4 constructs a modular model F_B of $\tilde{\Psi}$ from an ordinary model B of $\tilde{\Psi}$.

8.3*6 Example. Similarly, let $\mathcal{F}$ and $\mathcal{E}$ be the operation family and monoidal category in Example 8.2*4. Theorem 8.3*4 lets us construct modular models of theories of monoids with operations from ordinary models.

In particular, if $\mathcal{E} = \langle \text{ENDO}_{\kappa}(\mathcal{C}), \circ, \text{Id} \rangle$ for some lkp $\mathcal{C}$, and $\tilde{\Psi} \in \mathcal{F}$ be the theory of monads with some scoped operations, then Theorem 8.3*4 lets us construct a modular model F_B of $\tilde{\Psi}\mathcal{F}$ from a monad B equipped with a $\tilde{\Psi}$ -operation. For any theory $\tilde{\Sigma} \in \mathcal{F}$ of monads with operations, every monad A equipped with a $\tilde{\Sigma}$ -operation is sent by the modular model F_B to a new monad C with monad morphisms $A \rightarrow C$ and $B \rightarrow C$ that are respectively a $\tilde{\Sigma}$ -homomorphism and a $\tilde{\Psi}$ -homomorphism.

8.3*7 Example. Let $\mathcal{E}$ be a monoidal category, $\mathcal{F}$ be the operation family $\text{ALG}(\mathcal{E})$ of algebraic operations on $\mathcal{E}$ -monoids, and $\tilde{\Psi} \in \mathcal{F}$. If the fibration $P : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}$ satisfies the assumption of Theorem 8.3*4, the free modular model F_B of $\tilde{\Psi}$ over some $B \in \tilde{\Psi}\text{-ALG}$ has a simple characterisation – for every $\tilde{\Sigma} \in \text{ALG}(\mathcal{E})$ and $A \in \tilde{\Sigma}\text{-ALG}$, F_B simply maps A to the coproduct of B and A treated as monoids in $\mathcal{E}$. This is because an algebraic operation $S \circ M \rightarrow M$ on a monoid M is equivalently a morphism $S \rightarrow M$ (Lemma 5*15), so the initial monoid with both $\tilde{\Sigma}$ and $\tilde{\Psi}$ operations together with a $\tilde{\Sigma}$ -homomorphism from A and a $\tilde{\Psi}$ -homomorphism from B is the same thing as the initial monoid with monoid morphisms from A and B , i.e. the coproduct of A and B as monoids:

$$\begin{array}{ccccc} S & \longrightarrow & A & & B & \longleftarrow & S' \\ & \searrow & & \searrow & & \swarrow & \\ & & & M & & & \end{array}$$

8.4 Modular models from monoid transformers

8.4*1. The concept of modular models is directly inspired by Moggi's *monad transformers* and their generalisation, *monoid transformers* (Jaskelioff and Moggi 2010), to monoids in monoidal categories. A monoid transformer maps every monoid M in a monoidal category

$\mathcal{E}$ to a monoid TM together with a monoid morphism $i : M \rightarrow TM$. The central question about monoid transformers is:

If there is an operation on the monoid M , can this operation be transformed to an operation on TM too?

The standard terminology here is to *lift* the operation to TM rather than to *transform* but we use the latter to avoid the confusion with liftings along fibrations.

This question was first formulated by Moggi (1989a, §4.1), accompanied by a basic result (Moggi 1989a, Proposition 4.1.3): operations of the form $\alpha : A \rightarrow M$ for some fixed endofunctor A can always be transformed to $A \rightarrow TM$, namely $i \cdot \alpha : A \rightarrow TM$. About 20 years later, Jaskelioff and Moggi (2010) gave a new result: for *functorial monoid transformers* T on a *left-closed* monoidal category, every operation on M of the form $A \boxtimes M \rightarrow M$ can be lifted to $A \boxtimes TM \rightarrow TM$.

What we have done in this paper is bringing *equations* on operations into the view and formulating transformations of operations as model transformers. In this subsection, we adapt the constructions of Moggi (1989a) and Jaskelioff and Moggi (2010) to our setting with equations (Theorem 8.4*3 and Theorem 8.4*10).

8.4*2. In this subsection, we fix a monoidal category $\mathcal{E}$ with finite coproducts that is right-distributive: $(\coprod_{i \in S} A_i) \boxtimes B \cong \coprod_{i \in S} (A_i \boxtimes B)$, which ensures that $\text{ALG}(\mathcal{E})$ and $\text{SCP}(\mathcal{E})$ from Section 6 are closed under coproducts.

8.4*3 Theorem. *Let $\mathcal{F}$ be $\text{ALG}(\mathcal{E})$ and $\ddot{\Psi} = \langle \dot{\Psi}, T_\Psi \rangle \in \mathcal{F}$. Every functor $H : \text{MON}(\mathcal{E}) \rightarrow \dot{\Psi}\text{-ALG}$ together with a natural transformation $\tau : \text{Id} \rightarrow T_\Psi \circ H$*

$$\begin{array}{ccc} & \dot{\Psi}\text{-ALG} & \\ H \nearrow & \uparrow \tau & \searrow T_\Psi \\ \text{MON}(\mathcal{E}) & \xrightarrow{\text{Id}} & \text{MON}(\mathcal{E}) \end{array}$$

defines a strict modular model M of $\ddot{\Psi} \in \mathcal{F}$ making the following commute:

$$\begin{array}{ccc} \mathcal{F}\text{-ALG} & \xrightarrow{\tilde{M}} & (\mathcal{F} + \ddot{\Psi})\text{-ALG} \\ \downarrow & & \downarrow \\ \text{MON}(\mathcal{E}) & \xrightarrow{H} & \dot{\Psi}\text{-ALG} \end{array} \quad (43)$$

where $\tilde{M} : \mathcal{F}\text{-ALG} \rightarrow (\mathcal{F} + \ddot{\Psi})\text{-ALG}$ is the functor corresponding to M by items 1 and 2 of Theorem 7.1*17, and the unlabelled vertical arrows are the evident projection functors. Moreover, M has an updater $u_{\langle \dot{\Sigma}, T_\Sigma, A, \alpha \rangle} = \tau_{\langle A, T_\Sigma \alpha \rangle}$.

Proof. For every object $\langle \dot{\Sigma}, T_\Sigma, A, \alpha \rangle$ of $\mathcal{F}\text{-ALG}$ with

$$\dot{\Sigma} = (S \circ -)\text{-MON} \upharpoonright (K_G \vdash L = R),$$

we define $M : \mathcal{F}\text{-ALG} \rightarrow \mathcal{F}\text{-ALG}$ to send it to an $(\dot{\Sigma} + \dot{\Psi})$ -algebra with the same carrier of $H\langle A, T_\Sigma \alpha \rangle \in \dot{\Psi}\text{-ALG}$. Since $H\langle A, T_\Sigma \alpha \rangle$ already has a $\dot{\Psi}$ -algebra, we only need to equip it with an $(S \circ -)$ -operation. This can be done by the observation (Jaskelioff and Moggi 2010, Theorem 3.4) that *algebraic* operations can be transformed along monoid morphisms.

Namely, the transformed operation is

$$\alpha^\# = \llbracket s : S, h : H_A \vdash \mu^H(\tau_A(\alpha_S(s, \eta^A)), h) : H_A \rrbracket \quad (44)$$

where H_A and τ_A stand for the carrier of $H\langle A, T_\Sigma \alpha \rangle$ and $\tau_{\langle A, T_\Sigma \alpha \rangle} : A \rightarrow H_A$ respectively, and $\alpha_S : S \circ A \rightarrow A$ is the component of α for the algebraic operation on A .

We also need to show that the operation (44) satisfies the equation $K_G \vdash L = R$. This follows from the functoriality of

$$L, R : ((S \circ -) + \Sigma_{\text{MON}})\text{-ALG} \rightarrow G\text{-ALG},$$

which implies that the following diagrams commute

$$\begin{array}{ccc} G & \xrightarrow{L\langle A, \alpha \rangle} & A \\ & \searrow L\langle H_A, \alpha^\# \rangle & \downarrow \tau_A \\ & & H_A \end{array} \quad \text{and} \quad \begin{array}{ccc} G & \xrightarrow{R\langle A, \alpha \rangle} & A \\ & \searrow R\langle H_A, \alpha^\# \rangle & \downarrow \tau_A \\ & & H_A \end{array}$$

If α satisfies the equation $L = R$ (i.e. $L\langle A, \alpha \rangle = R\langle A, \alpha \rangle$), so does $\alpha^\#$. It can be shown that M is a *strict* modular model of Ψ by the same argument for the special case of exception monad transformers in Example 7.1*8. $\square$

8.4*4. Example 7.1*8 is exactly this theorem applied to the exception monad transformer. The *state monad transformer* $A \mapsto (A(S \times -))^S$ for a set S with $|S| < \kappa$ together with its model for the theory ST_S of *mutable state* (Example 5*11) yields a modular model of ST_S in $\text{ALG}(\text{ENDO}_\kappa(\mathcal{C}))$. The *list monad transformer* $A \mapsto \mu X.A(1 + (- \times X))$ (Jaskelioff and Moggi 2010) with its model for *explicit nondeterminism* also gives rise to a modular model.

8.4*5. Now we move on to scoped operations. First we recall that $\text{SCP}_I(\mathcal{E})$ from 6*20 is the operation family of scoped operations on monoids and *transliterations*. Let us again start with a concrete example.

8.4*6 Example. The theory Ec of *exception throwing and catching* in Example 5*17 is in the family $\text{SCP}_I(\mathcal{E})$ for $\mathcal{E} = (\text{ENDO}_\kappa(\text{SET}), \circ, \text{Id})$. A strict modular model for it can be constructed by extending the modular model of throwing in Example 7.1*8 with (1) a model of catching on $C_A = A \circ (1 + \text{Id})$ and (2) a way to transform scoped operations on A to C_A .

For (1), we define $\text{catch} : (C_A \times C_A) \circ C_A \rightarrow C_A$ by $\text{catch} := \mu^C \cdot ((c \cdot s) \circ C_A)$ where $\mu^C : C_A \circ C_A \rightarrow C_A$ is the multiplication on C_A defined in Example 7.1*8, and s is the following morphism in which the unlabelled arrow is the canonical strength for the functor $A \in \text{ENDO}_\kappa(\text{SET})$:

$$s : C_A \times C_A = (A \circ (1 + \text{Id})) \times C_A \rightarrow A \circ ((1 + \text{Id}) \times C_A) \cong A \circ (C_A + \text{Id} \times C_A),$$

and lastly the morphism $c : A \circ (C_A + \text{Id} \times C_A) \rightarrow C_A$ is denoted by

$$\begin{aligned} a : A, b : (C_A + \text{Id} \times C_A) &\vdash \mu^C \left((a, \iota_2 *) \right), \\ \text{case } b \text{ of } \{ \iota_1 h \mapsto h; \iota_2 ih \mapsto \eta^C(\pi_1 ih) \} & : C_A \end{aligned}$$

The operational idea for this term is that the computation a is first executed and b is its result. The case $b = \iota_1 h$ means that an exception is thrown and h is the exception handler, so h is executed in this case. On the other hand, the case $b = \iota_2 ih$ means a normal termination, and the handler is ignored by $\pi_1 ih$.

For (2), to transform a scoped operation $\alpha : S \circ A \circ A \rightarrow A$ on A to C_A , we define $\alpha^\# : S \circ C_A \circ C_A \rightarrow C_A$ by the term

$$s : S, a : A, m : 1 + \text{Id}, k : C_A \vdash \mu^C((\alpha(s, a, \eta^A), m), k) : C_A.$$

The transformed operation $\alpha^\#$ satisfies any constant equation $K_C \vdash L = R$ whenever α does by the same argument as in the proof of Theorem 8.4*3.

8.4*7. Theorem 8.4*3 constructs modular models for algebraic operations from monoid transformers. Jaskielioff and Moggi (2010) shows that this is also possible for scoped operations, provided that the monoid transformer is *functorial*.

8.4*8 Definition (Jaskielioff and Moggi (2010)). A *functorial monoid transformer* on a monoidal category $\mathcal{E}$ consists of two functors $\dot{F} : \text{MON}(\mathcal{E}) \rightarrow \text{MON}(\mathcal{E})$ and $F : \mathcal{E} \rightarrow \mathcal{E}$ and two natural transformations $\dot{\sigma} : \text{Id} \rightarrow \dot{F}$ and $\sigma : \text{Id} \rightarrow F$:

$$\begin{array}{ccc} & \dot{F} & \\ & \uparrow \dot{\sigma} & \\ \text{MON}(\mathcal{E}) & \xrightarrow{\text{Id}} & \text{MON}(\mathcal{E}) \\ \downarrow U_{\text{MON}} & \uparrow F & \downarrow U_{\text{MON}} \\ \mathcal{E} & \xrightarrow{\text{Id}} & \mathcal{E} \\ & \uparrow \sigma & \end{array}$$

such that $U_{\text{MON}} \circ \dot{\sigma} = \sigma \circ U_{\text{MON}}$

8.4*9. As shown by Jaskielioff and Moggi (2010), many monad transformers in programming languages are functorial, including the exception monad transformer $M(E + -)$ for monads M , the state monad transformer $S \Rightarrow M(S \times -)$, the writer monad transformer $M(W \times -)$, and the (generalised) resumption monad transformer $\mu X. M(\Sigma X + -)$ (Cenciarelli and Moggi 1993). However, the seemingly functorial list transformer $L_M := \mu X. M(1 + (- \times X))$ is in fact not functorial, because the associated natural transformation $\dot{\sigma} : M \rightarrow L_M$ defined by $a : M \vdash (a, \iota_2 \langle *, (\eta^M, \iota_1 \langle \rangle) \rangle) : M \circ (1 + \text{Id} \times L_M)$ refers to the unit η^M of M , which is not a part of the underlying functor of M .

8.4*10 Theorem. Assume that the monoidal category $\mathcal{E}$ is left-closed. Let $\ddot{\Psi}$ be some $\langle \ddot{\Psi}, T_\Psi \rangle \in \text{SCP}_l(\mathcal{E})$. A functorial monoid transformer $\langle \dot{F}, F, \dot{\sigma}, \sigma \rangle$ and a functor $H : \text{MON}(\mathcal{E}) \rightarrow \ddot{\Psi}\text{-ALG}$ such that $\dot{F} = T_\Psi \cdot H$ induce a strict modular model M of $\ddot{\Psi} \in \text{SCP}_l(\mathcal{E})$ with an updater $u_{\langle \ddot{\Sigma}, T_\Sigma, A, \alpha \rangle} = \sigma_{\langle A, T_\Sigma \alpha \rangle}$.

Proof sketch. Compared to Theorem 8.4*3, what is new is how scoped operations $\alpha : S \square A \square A \rightarrow A$ on a monoid $\langle A, \eta^A, \mu^A \rangle$ are transformed to FA . Such a transformation (not necessarily the unique one) is given by Jaskielioff and Moggi (2010), whose insight is that scoped operations on A are the same as an algebraic operation on the monoid A/A , into which A embeds by Cayley's theorem (Example 4.2*4), and we already know how to transform algebraic operations along monoid transformers. This is why we need (left) closedness in the assumption. We briefly record the transformation below and refer the reader to Jaskielioff and Moggi (2010, §5.1) for more details.

Firstly, recall that we have the Cayley embedding $e : A \rightarrow A/A$ and its retraction $r : A/A \rightarrow A$ defined as follows:

$$e = (a : A \vdash \lambda x. \mu^A(a, x) : A/A) \quad r = (f : A/A \vdash f \eta^A : A).$$

We can similarly transpose the scoped operation $\alpha : S \sqcap A \sqcap A \rightarrow A$ on A to obtain a morphism $\tilde{\alpha} : S \rightarrow A/A$:

$$\tilde{\alpha} = (s : S \vdash \lambda x. \alpha(s, x, \eta^A)) : A/A$$

The transformed operation $\alpha^\# : S \sqcap FA \sqcap FA \rightarrow FA$ is then defined by

$$s : S, a : FA, b : FA \vdash \mu^{FA} \left(Fr \left(\mu^{F(A/A)} (\sigma_{A/A} (\tilde{\alpha} s), Fe a) \right), b \right) : FA$$

This transformed operation $\alpha^\#$ preserves any constant equation $K_C \vdash L = R$ satisfied by α by the same argument for Theorem 8.4*3. Moreover, the definition of $\alpha^\#$ is natural w.r.t. $\text{SCP}_I(\mathcal{E})$: given any transliteration $f : S' \rightarrow S$ between scoped operations, we have

$$\alpha^\# \cdot (f \sqcap FA \sqcap FA) = (\alpha \cdot (f \sqcap A \sqcap A))^\#,$$

which can be directly checked or deduced from the general fact that all the term formers of monoidal algebraic theories are natural, similar to the abstraction theorem of simply typed lambda calculus (Reynolds 1983). $\square$

8.4*11 Remark. The theorem above needs the monoid transformer $\dot{F}$ to be over some $F : \mathcal{E} \rightarrow \mathcal{E}$ because the retract $r : A/A \rightarrow A$ of the Cayley embedding (Example 4.2*4) used in the proof is not a monoid morphism, so we need $F : \mathcal{E} \rightarrow \mathcal{E}$ to have $Fr : F(A/A) \rightarrow A$. The requirement of having $\sigma : \text{Id} \rightarrow F$ below $\dot{\sigma} : \text{Id} \rightarrow \dot{F}$ is also essential. It is needed for showing that the updater is an algebra-homomorphism (Jaskelioff and Moggi 2010, Lemma 5.3) and the equations are preserved, which are omitted in the proof sketch above.

8.4*12. A mistake in the earlier paper (Yang and Wu 2023) by the authors is that the strict modular models from Theorem 8.4*10 and Example 8.4*6 were claimed to be w.r.t. the family $\text{SCP}(\mathcal{E})$ rather than $\text{SCP}_I(\mathcal{E})$. This is wrong because the operation transformation $\alpha^\# : S \sqcap FA \sqcap FA \rightarrow FA$ from a scoped operation $\alpha : S \sqcap A \sqcap A \rightarrow A$ in these modular models is not natural with respect to translations in $\text{SCP}(\mathcal{E})$, i.e. there exist translations T such that $(T\alpha)^\# \neq T(\alpha^\#)$.

For a counterexample, consider the modular model of exception catching from Example 8.4*6. Let $\tilde{\Sigma} \in \text{SCP}(\text{ENDO}_K(\text{SET}))$ be the theory of monads with a binary scoped operation $b : (\text{Id} \times \text{Id}) \circ A \circ A \rightarrow A$, and let $\tilde{\Sigma}' \in \text{SCP}(\text{ENDO}_K(\text{SET}))$ be an arbitrary theory. In the category SCP (but not in SCP_I), we have a translation $T : \tilde{\Sigma} \rightarrow \tilde{\Sigma}'$ that as a functor sends every $\langle A, \mu^A, \eta^A, \alpha \rangle \in \tilde{\Sigma}'\text{-ALG}$ to the $\tilde{\Sigma}$ -algebra $\langle A, \mu^A, \eta^A, \mu^A \cdot (c \circ A) \rangle$ where c is

$$(\text{Id} \times \text{Id}) \circ A = A \times A \xrightarrow{s} A \circ (\text{Id} \times A) \xrightarrow{A \circ \pi_2} A \circ A \xrightarrow{\mu^A} A$$

and the arrow s is the canonical strength $s_n : An \times An \rightarrow A(n \times An)$ for the set-endofunctor A . Note that this translation completely ignores the original operation α . It is perhaps more intuitive to use the syntax of an ordinary programming language here, say Haskell, which would be the following:

$$\begin{aligned} T\alpha :: A x \rightarrow A x \rightarrow (x \rightarrow A y) \rightarrow A y \\ T\alpha m n k = \mathbf{do} _ \leftarrow m; x \leftarrow n; k x \end{aligned}$$

In prose, T translates the binary scoped operation $b(x, y)$ to the computation that first runs x , ignores its result, and then runs y .

The operation lifting $\alpha^\#$ in Example 8.4*6 for a binary scoped operation $\alpha :: A x \rightarrow A x \rightarrow (x \rightarrow A y) \rightarrow A y$ would be the following in Haskell:


```

data Maybe x    = Nothing | Just x
data MaybeT A x = MaybeT (A (Maybe x))
 $\alpha^\# :: \text{MaybeT } A \ x \rightarrow \text{MaybeT } A \ x \rightarrow (x \rightarrow \text{MaybeT } A \ y) \rightarrow \text{MaybeT } A \ y$ 
 $\alpha^\# (\text{MaybeT } m') (\text{MaybeT } n') k = \text{do } x \leftarrow \text{MaybeT } (\alpha \ m' \ n' \ \text{return}); k \ x$ 

```

Now we can see that the two binary scoped operation $(T\alpha)^\#$ and $T(\alpha^\#)$ are not equal. The operation $T(\alpha^\#)$ written in Haskell would be

```

 $b_1 :: \text{MaybeT } A \ x \rightarrow \text{MaybeT } A \ x \rightarrow (x \rightarrow \text{MaybeT } A \ y) \rightarrow \text{MaybeT } A \ y$ 
 $b_1 \ m \ n \ k = \text{do } \_ \leftarrow m; x \leftarrow n; k \ x$ 

```

while the operation $(T\alpha)^\#$ would be

```

 $b_2 (\text{MaybeT } m') (\text{MaybeT } n') k = \text{do } x \leftarrow \text{MaybeT } (\text{do } \_ \leftarrow m'; n'); k \ x$ 

```

The difference is that when m throws an exception, i.e. when m' returns *Nothing*, b_1 will stop after m , whereas b_2 will continue as n' .

8.5 Colimits and limits of model transformers

8.5*1. The category of model transformers (i.e. liftings along fibrations) inherits many properties of categories of ordinary models. As a first step, in the following we show colimits and *reindexing-stable limits* of ordinary models can be lifted to model transformers.

8.5*2 Theorem. *Let $P : \mathcal{A} \rightarrow \mathcal{T}$ be a fibration and $P' : \mathcal{A}' \rightarrow \mathcal{T}'$ be a fibration with a cleavage κ , and let $T : \mathcal{T} \rightarrow \mathcal{T}'$ be a functor and $\mathcal{D}$ be a category. If every fiber category $\mathcal{A}'_\Sigma$ of P' has (chosen) $\mathcal{D}$ -indexed colimits, the category $\text{MotR}(T)$ of model transformers of T also has $\mathcal{D}$ -indexed colimits, which are computed fiberwise.*

Moreover, if reindexing functors of P' preserve (or strictly preserve) $\mathcal{D}$ -indexed colimits, the full subcategory of $\text{MotR}(T)$ containing strong (or strict) model transformers is closed under $\mathcal{D}$ -indexed colimits in $\text{MotR}(T)$.

Proof. Let $M : \mathcal{D} \rightarrow \text{MotR}(T)$ be a $\mathcal{D}$ -diagram of model transformers. We define a functor $C : \mathcal{A} \rightarrow \mathcal{A}'$ that sends every object $A \in \mathcal{A}$ to the colimit of $M_i A$ in the fiber category $\mathcal{A}_{TPA}$. For every morphism $f : A \rightarrow B$ in $\mathcal{A}$, let $\alpha_i^A : M_i A \rightarrow CA$ and $\alpha_i^B : M_i B \rightarrow CB$ be the colimiting cocones:

$$\begin{array}{ccccc}
 M_i A & \xrightarrow{\quad} & M_j A & & \\
 \searrow \alpha_i^A & & \swarrow \alpha_j^A & \searrow M_j f & \\
 & CA & & M_i B & \xrightarrow{\quad} & M_j B \\
 & & & \searrow \alpha_i^B & & \swarrow \alpha_j^B \\
 & & & & CB
 \end{array}$$

Writing $r := TPf$, for every $i \in \mathcal{D}$, the morphism $M_i f : M_i A \rightarrow M_i B$ factors as a vertical morphism $v_i : M_i A \rightarrow r^*(M_i B)$ in $\mathcal{A}'_{TPA}$ followed by a cartesian morphism. The reindexing

functor r^* sends the cocone α_i^B in $\mathcal{A}'_{TPB}$ to a cocone $r^*\alpha_i^B : r^*M_iB \rightarrow r^*CB$ in $\mathcal{A}'_{TPA}$:

$$\begin{array}{ccccc}
 M_iA & \xrightarrow{\quad} & M_jA & & \\
 \downarrow v_i & \searrow \alpha_i^A & \swarrow \alpha_j^A & \downarrow v_j & \\
 & & CA & & \\
 \downarrow & & \vdots & & \\
 r^*M_iB & \xrightarrow{\quad} & r^*M_jB & & M_iB \xrightarrow{\quad} M_jB \\
 \searrow r^*\alpha_i^B & & \swarrow r^*\alpha_j^B & & \searrow \alpha_i^B \quad \swarrow \alpha_j^B \\
 & & r^*CB & \xrightarrow{\quad \kappa(CB,r) \quad} & CB
 \end{array} \tag{45}$$

The composite $(r^*\alpha_i^B) \cdot v_i : M_iA \rightarrow r^*CB$ can be checked to be a cocone too. By the universal property of CA as a colimit of M_iA , we have a unique vertical morphism $u : CA \rightarrow r^*CB$ such that $u \cdot \alpha_i^A = r^*\alpha_i^B \cdot v_i$. We define the action of C on the morphism $f : A \rightarrow B$ to be $\kappa(CB, r) \cdot u : CA \rightarrow CB$. The functoriality of C is a consequence of the functoriality of M_i and the universal property of CA as colimits. For example, if $f : A \rightarrow B$ above is $id_A : A \rightarrow A$, it can be checked by diagram chasing that for all $i \in \mathcal{D}$, $Cid_A \cdot \alpha_i^A = \alpha_i^A \cdot M_iid_A = \alpha_i^A$, so $Cid_A = id_A$. The case for $C(g \cdot f) = Cg \cdot Cf$ is more complex but similar.

The functor C is by construction a lifting of T . For each i , we show that the family of morphisms $\alpha_i^A : M_iA \rightarrow CA$ is natural in A . In the following diagram,

$$\begin{array}{ccccc}
 M_iA & & & & \\
 \downarrow v_i & \searrow \alpha_i^A & & & \\
 & & CA & & \\
 \downarrow & & \vdots & & \\
 r^*M_iB & \xrightarrow{\quad} & M_iB & \xrightarrow{\quad \kappa(M_iB, r) \quad} & \\
 \searrow r^*\alpha_i^B & & \downarrow u & & \searrow \alpha_i^B \\
 & & r^*CB & \xrightarrow{\quad \kappa(CB, r) \quad} & CB
 \end{array}$$

we have $Cf \cdot \alpha_i^A = \kappa(CB, r) \cdot u \cdot \alpha_i^A = \kappa(CB, r) \cdot r^*(\alpha_i^B) \cdot v_i$. The morphism $r^*\alpha_i^B$, which is the image of α_i^B under reindexing r^* , is by definition the unique morphism making the square at the bottom commute, so we have $\kappa(CB, r) \cdot r^*(\alpha_i^B) \cdot v_i = \alpha_i^B \cdot \kappa(M_iB, r) \cdot v_i = \alpha_i^B \cdot M_if$. Hence we have shown the required naturality: $Cf \cdot \alpha_i^A = \alpha_i^B \cdot M_if$, and thus we have a cocone $\langle \alpha_i \rangle_{i \in \mathcal{D}}$ in $\text{Motr}(T)$.

Given any cocone $\langle \beta_i : M_i \rightarrow N \rangle_{i \in \mathcal{D}}$, for every $A \in \mathcal{A}$, $\langle \beta_i^A \rangle$ is a cocone in $\mathcal{A}_{TPA}$ from M_iA to NA , so there is a unique mediating morphism $\sigma^A : CA \rightarrow NA$ such that $\sigma^A \cdot \alpha_i^A = \beta_i^A$. It can be checked by diagram chasing that the family of morphisms σ^A is natural in A , so C is the colimit of M_i in $\text{Motr}(T)$.

Finally, by the construction of the colimit C above, we can see that if reindexing functors of P' (strictly) preserve $\mathcal{D}$ -indexed colimits in fiber categories, then when $f : A \rightarrow B$ is cartesian and all M_i are strong, the two cocones in the left of (45) are isomorphic (the same), therefore C is strong (strict) too. $\square$

8.5*3. The situation for limits is slightly different: we need reindexing functors to preserve limits in fiber categories for $\text{Motr}(T)$ to inherit these limits. This requirement is not too demanding though, since in many fibrations of algebras and theories, reindexing functors are right adjoints so they preserve all limits.

8.5*4 Theorem. Let $P : \mathcal{A} \rightarrow \mathcal{T}$ be a fibration and $P' : \mathcal{A}' \rightarrow \mathcal{T}'$ be a fibration with a cleavage κ , and let $T : \mathcal{T} \rightarrow \mathcal{T}'$ be a functor and $\mathcal{D}$ be a category. If every fiber category $\mathcal{A}'_\Sigma$ of P' has (chosen) $\mathcal{D}$ -indexed limits, and reindexing functors preserve these limits, then the category $\text{Motr}(T)$ has $\mathcal{D}$ -indexed limits. Moreover, the subcategory containing strong/strict model transformers is closed under these limits.

Proof sketch. Similar to the case of colimits above, the limit L of a diagram M_i of model transformers is defined fiberwise: for every object $A \in \mathcal{A}$, LA is defined to be the (chosen) limit of $M_i A$ in the fiber category $\mathcal{A}'_{PTA}$. However, the action of L on a morphism $f : A \rightarrow B$ is different from the situation of colimits:

$$\begin{array}{ccccc}
 & & LA & & \\
 & \alpha_i^A \swarrow & \vdots & \searrow \alpha_j^A & \\
 M_i A & \xrightarrow{\quad u \quad} & M_j A & & \\
 v_i \downarrow & & \downarrow & & \downarrow v_j \\
 r^* M_i B & \xrightarrow{\quad r^* \alpha_j^B \quad} & r^* M_j B & \xrightarrow{\quad \kappa(LB, r) \quad} & LB \\
 & \nwarrow r^* \alpha_i^B & \nearrow r^* \alpha_j^B & & \nwarrow \alpha_i^B \nearrow \alpha_j^B \\
 & & & & M_i B \xrightarrow{\quad \quad \quad} M_j B
 \end{array}$$

By reindexing the limiting cone $\alpha_i^B : LB \rightarrow M_i B$ along $r := TPf$, we have a cone $r^* \alpha_i^B : r^* LB \rightarrow r^* M_i B$. Let $v_i : M_i A \rightarrow r^* M_i B$ be the unique vertical morphism $\kappa(M_i B) \cdot v_i = M_i$. We have a cone $(v_i \cdot \alpha_i^A) : LA \rightarrow r^* M_i B$. Now we use the assumption that r^* preserves $\mathcal{D}$ -limits, so $r^* \alpha_i^B : r^* LB \rightarrow r^* M_i B$ is still a limiting cone, and we have a vertical morphism $u : LA \rightarrow r^* LB$. The rest of this proof is similar to the proof of Theorem 8.5*2. $\square$

8.5*5. Under the assumptions of Theorem 8.5*4 and additionally that P, P' are the same fibration, and $T : \mathcal{T} \rightarrow \mathcal{T}$ is equipped with $\eta : \text{Id} \rightarrow T$, the category $\text{Motr}_u(T)$ of updatable model transformers also has $\mathcal{D}$ -indexed limits. In fact, limits in $\text{Motr}_u(T)$ are *strictly created* by the forgetful functor $U : \text{Motr}_u(T) \rightarrow \text{Motr}(T)$, which means that for every diagram $D : \mathcal{D} \rightarrow \text{Motr}_u(T)$, whenever $U \circ D$ has a limiting cone $\alpha_i : L \rightarrow UD_i$ in $\text{Motr}(T)$, there exists a unique updater u for L making $\alpha_i : \langle L, u \rangle \rightarrow D_i$ a limiting cone in $\text{Motr}_u(T)$.

To prove this, recall that an updater u for a model transformer M is a natural transformation $u : \text{Id} \rightarrow M$ over $\eta : \text{Id} \rightarrow T$. For a $\mathcal{D}$ -indexed diagram $\langle M_i, u_i \rangle$ in $\text{Motr}_u(T)$, for every $A \in \mathcal{A}$, we have a vertical morphism $u_i^A : A \rightarrow \eta_A^* M_i A$. Since morphisms in $\text{Motr}_u(T)$ are compatible with updaters, u_i^A is a cone over $\eta_A^* M_i A$. In the proof of Theorem 8.5*4, the limit L of M_i is computed pointwise and fiberwise, so LA is the limit of $M_i A$ in the fiber $\mathcal{A}_{TPA}$. Moreover, the limit LA is preserved by reindexing η_A^* , so $\eta_A^* LA$ is a limit of $\eta_A^* M_i A$ in $\mathcal{A}_{PA}$, and the cone $u_i^A : A \rightarrow \eta_A^* M_i A$ then gives us a unique mediating morphism $u_A : A \rightarrow \eta_A^* LA$. It can be checked that this u is natural and is an updater for L .

Note however, the forgetful functor $\text{Motr}_u(T) \rightarrow \text{Motr}(T)$ does not create colimits: a cone $u_i^A : A \rightarrow \eta_A^* M_i A$ does not give us a morphism $A \rightarrow \eta_A^* CA$ into the colimit that commutes with $\eta_A^* \alpha_i^A : \eta_A^* M_i A \rightarrow \eta_A^* CA$ for all $i \in \mathcal{D}$.

8.6 Composition and fusion of model transformer

8.6*1. Model transformers are readily composable horizontally. Let M and N be two (strict/strong) model transformers of functors S and T respectively,

$$\begin{array}{ccccc} \mathcal{A} & \xrightarrow{M} & \mathcal{A}' & \xrightarrow{N} & \mathcal{A}'' \\ P \downarrow & & \downarrow P' & & \downarrow P'' \\ \mathcal{T} & \xrightarrow{S} & \mathcal{T}' & \xrightarrow{T} & \mathcal{T}'' \end{array}$$

it is immediate that the composite functor $N \circ M$ is a (strict/strong) model transformers of $T \circ S : \mathcal{T}' \rightarrow \mathcal{T}''$. Moreover, when P, P' , and P'' are the same fibration, and the functors S and T are pointed, an updater u of M and an updater v of N can be composed horizontally to an updater $v \circ u : \text{Id} \rightarrow N \circ M$ as well.

In particular, the composition of a modular model M of $\Sigma \in \mathcal{T}$ (i.e. a model transformer of $- + \Sigma : \mathcal{T} \rightarrow \mathcal{T}$) and a modular model N of $\Phi \in \mathcal{T}$ gives us a modular model of $\Sigma + \Phi$ via the isomorphism $- + (\Sigma + \Phi) \cong (- + \Sigma) + \Phi$.

8.6*2 Example. Let M_E be the modular model of *exception* throwing and catching (Example 8.4*6), and M_S be the modular model of *mutable state* arising from the state monad transformer by Theorem 8.4*3. The composite $M_S \circ M_E$ is a modular model of $\text{Ec} + \text{ST}_S$, the theories of exception and mutable state.

8.6*3. Coproducts of theories are commutative, $\Sigma + \Phi \cong \Phi + \Sigma$, but the composition of modular models is of course not. For example, the opposite order $M_E \circ M_S$ of composing the modular models in Example 8.6*2 gives rise to another modular model of the coproduct $\text{Ec} + \text{ST}_S$. Both $M_S \circ M_E$ and $M_E \circ M_S$ satisfy the respective equations of exception and mutable state, but they validate different interaction equations: $M_S \circ M_E$ additionally validates commutativity of stateful operations and exception throwing, so the following program equivalence is validated by $M_S \circ M_E$:

$$\text{catch } (\text{do put } s; \text{throw}) h = \text{catch } (\text{do throw}; \text{put } s) h = \text{catch throw } h = h,$$

where the second step $\text{throw}; \text{put } s = \text{throw}$ is due to the algebraicity of throw as a nullary operation. On the other hand, $M_E \circ M_S$ validates

$$\text{catch } (\text{do put } s; p) h = \text{do put } s; \text{catch } p h,$$

so $\text{catch } (\text{do put } s; \text{throw}) h = \text{do put } s; \text{catch throw } h = \text{do put } s; h$. An operational interpretation is that when an exception is caught, $M_S \circ M_E$ rolls back to the state before the *catch*, whereas $M_E \circ M_S$ keeps the state as it is. Both behaviours are desirable depending on the scenario. More discussion about interaction of effects can be found in Yang and Wu (2021).

8.6*4. A straightforward but useful result about composites of model transformers is the *fusion lemma* below: interpreting a term with two model transformers sequentially is equal to interpreting with the composite model transformer. Therefore two consecutive interpretations can be combined into one, eliminating the need to generate the intermediate result that is consumed immediately, a program optimisation known as *short-cut fusion* (Gill et al. 1993; Hinze et al. 2011).

Generalising the natural transformation $h^M : (- + \check{\Psi})^* \rightarrow M(-)^*$ in 7.1*23, let $P : \mathcal{A} \rightarrow \mathcal{T}$ be a fibration with a cleavage such that all fiber categories have initial objects. We then have

a functor $(-)^* : \mathcal{T} \rightarrow \mathcal{A}$ that maps every object $\Sigma \in \mathcal{T}$ to the initial object 0_Σ in the fiber $\mathcal{A}_\Sigma$, and $(-)^*$ maps every morphism $t : \Sigma \rightarrow \Gamma$ to the unique morphism $0_\Sigma \rightarrow t^*0_\Gamma$ followed by the cartesian morphism over t . For every model transformer $M : \mathcal{A} \rightarrow \mathcal{A}$ of some functor $T : \mathcal{T} \rightarrow \mathcal{T}$, we then have a unique natural transformation $h^M : (T-)^* \rightarrow M(-)^*$:

$$\begin{array}{ccc}
 \mathcal{A} & \xrightarrow{M} & \mathcal{A} \\
 \uparrow (-)^* & \swarrow h^M & \uparrow (-)^* \\
 \mathcal{T} & \xrightarrow{T} & \mathcal{T}
 \end{array}
 \begin{array}{c}
 P \quad \quad P \\
 \curvearrowright \quad \quad \curvearrowleft
 \end{array}$$

that interprets the abstract syntax $(T\Sigma)^*$ with the model $M\Sigma^*$ for every $\Sigma \in \mathcal{T}$.

8.6*5 Lemma (Fusion). For $i \in \{1, 2, 3\}$, let $P^i : \mathcal{A}^i \rightarrow \mathcal{T}^i$ be a cloven fibration such that every fiber category has initial objects. Given model transformers $N : \mathcal{A}^1 \rightarrow \mathcal{A}^2$ of $S : \mathcal{T}^1 \rightarrow \mathcal{T}^2$ and $M : \mathcal{A}^2 \rightarrow \mathcal{A}^3$ of $T : \mathcal{T}^2 \rightarrow \mathcal{T}^3$, we have

$$\begin{array}{ccccc}
 \mathcal{A}^1 & \xrightarrow{N} & \mathcal{A}^2 & \xrightarrow{M} & \mathcal{A}^3 \\
 (-)^* \uparrow & \swarrow h^N & \uparrow & \swarrow h^M & \uparrow (-)^* \\
 \mathcal{T}^1 & \xrightarrow{S} & \mathcal{T}^2 & \xrightarrow{T} & \mathcal{T}^3
 \end{array}
 =
 \begin{array}{ccccc}
 \mathcal{A}^1 & \xrightarrow{N} & \mathcal{A}^2 & \xrightarrow{M} & \mathcal{A}^3 \\
 (-)^* \uparrow & \swarrow h^{M \circ N} & \uparrow & \swarrow & \uparrow (-)^* \\
 \mathcal{T}^1 & \xrightarrow{S} & \mathcal{T}^2 & \xrightarrow{T} & \mathcal{T}^3
 \end{array}$$

i.e. $h_\Sigma^{M \circ N} = (M h_\Sigma^N) \cdot h_{S\Sigma}^M : (TS\Sigma)^* \rightarrow MN\Sigma^*$ for every $\Sigma \in \mathcal{T}^1$.

Proof. The components at Σ of these two natural transformations are both the *unique* morphism out of the initial object of the fiber category over $TS\Sigma$. $\square$

8.7 Modular models in symmetric monoidal categories

8.7*1. In this subsection, we will have a look at some constructions of modular models that are only possible in symmetric monoidal categories $\mathcal{E}$, such as $\langle \mathcal{C}, \times, 1 \rangle$ for cartesian monoids and $\langle \text{ENDO}_\kappa(\text{SET}), *, \text{Id} \rangle$ for applicative functors.

8.7*2. To begin with, we can upgrade ordinary models of algebraic and scoped operations to modular models by using the fact that in a symmetric $\mathcal{E}$, two monoids $\langle A, \mu^A, \eta^A \rangle$ and $\langle B, \mu^B, \eta^B \rangle$ induces a monoid structure on $A \square B$.

8.7*3 Theorem (Independent Combination). *Let $\mathcal{E}$ be a symmetric monoidal category and $\mathcal{F}$ be $\text{ALG}(\mathcal{E})$ or $\text{SCP}_l(\mathcal{E})$. For each $\check{\Psi} \in \mathcal{F}$, every $\check{A} \in \check{\Psi}\text{-ALG}$ induces a strict modular model M of $\check{\Psi} \in \mathcal{F}$ such that $M\langle \check{\Sigma}, B, \beta \rangle$ is carried by $A \square B$, and M has an updater $u_{\langle \check{\Sigma}, B, \beta \rangle} = (B \cong I \square B \xrightarrow{\eta^{A \square B}} A \square B)$.*

Proof. Given a monoid $\langle B, \mu^B, \eta^B \rangle$, $A \square B$ has the following monoid structure:

$$\begin{aligned}
 \eta^{A \square B} &= (I \cong I \square I \xrightarrow{\eta^A \square \eta^B} A \square B) \\
 \mu^{A \square B} &= ((A \square B) \square (A \square B) \cong (A \square A) \square (B \square B) \xrightarrow{\mu^A \square \mu^B} A \square B)
 \end{aligned}$$

Moreover, we can transform a scoped operation $\alpha : C \square A \square A \rightarrow A$ on A to a scoped operation $\alpha^\#$ on $A \square B$ as follows:

$$C \square (A \square B) \square (A \square B) \cong C \square (A \square A) \square (B \square B) \xrightarrow{\alpha \square \mu^B} A \square B$$

Symmetrically, every scoped operation on B can also be transformed to $A \sqcap B$. Furthermore, algebraic operations are special cases of scoped operations, so they can be transformed in the same way. The preservation of (constant) equations of the operation transformation is the same as the proof of Theorem 8.4*3. $\square$

8.7*4. For $\mathcal{E} = \langle \text{ENDOK}(\text{SET}), *, \text{Id} \rangle$, the intuition for $A * B$ is that two applicative-computations A and B are combined in the way that they execute *independently*, and operations act on $A * B$ pointwise.

There is another way to compose two applicatives, namely $A \circ B$ (McBride and Paterson 2008). In this way, the B -computation can *depend* on the result of A .

8.7*5 Theorem (Dependent Combination). *Let $\mathcal{E}$ be $\langle \text{ENDOK}(\text{SET}), *, \text{Id} \rangle$ and $\mathcal{F}$ be $\text{ALG}(\mathcal{E})$ or $\text{SCP}_I(\mathcal{E})$. For each $\Psi \in \mathcal{F}$, every $\dot{A} \in \Psi\text{-ALG}$ induces a strict modular model M of Ψ such that $M_{\Sigma} \langle B, \beta \rangle$ is carried by $A \circ B$, and M has an updater $u_{\Sigma, B, \beta} = \eta^A \circ B$.*

Proof sketch. Given two applicative functors $\langle A, \mu^A, \eta^A \rangle$ and $\langle B, \mu^B, \eta^B \rangle$, their composition $A \circ B$ as functors can be equipped with an applicative structure with unit $\eta^{A \circ B} = \eta^A \circ \eta^B$ and the following multiplication $\mu^{A \circ B}$:

$$\begin{aligned} ((A \circ B) * (A \circ B))n &\cong \int^{m,k} A(Bm) \times A(Bk) \times n^{m \times k} \\ &\xrightarrow{f} \int^{m,k} A(Bm) \times A(Bk) \times (Bn)^{Bm \times Bk} \\ &\xrightarrow{g} \int^{m',k'} Am' \times Ak' \times (Bn)^{m' \times k'} \\ &\cong (A * A)(Bn) \xrightarrow{\mu^A} A(Bn) \end{aligned}$$

where the arrow g is the substitution of the bound variables of the coend $m' = Bm$ and $k' = Bk$; the arrow f uses functoriality of the coend and the morphism $n^{m \times k} \rightarrow (Bn)^{Bm \times Bk}$ given by the transpose of the following morphism:

$$n^{m \times k} \times Bm \times Bk \xrightarrow{\iota_{m,k}} \int^{m,k} Bm \times Bk \times n^{m,k} \cong (B * B)n \xrightarrow{\mu^B} Bn.$$

To transform a scoped operation $\alpha : S * A * A \rightarrow A$ to $A \circ B$, we use the fact that there is a canonical morphism $s : S * (A \circ B) \rightarrow (S * A) \circ B$ as follows:

$$\begin{aligned} (S * (A \circ B))n &\cong \int^{m,k} Sm \times A(Bk) \times n^{m \times k} \\ &\rightarrow \int^{m,k} Sm \times A(Bk) \times (Bn)^{m \times Bk} \\ &\rightarrow \int^{m,k'} Sm \times Ak' \times (Bn)^{m \times k'} \\ &\cong (S * A)(Bn) \end{aligned}$$

where the first step uses the action of the functor B on morphisms: $n^k \rightarrow (Bn)^{(Bk)}$, and the second steps is the substitution of the bound variable $k' = Bk$. We define the transformation of α to $A \circ B$ to be

$$\begin{aligned} S * (A \circ B) * (A \circ B) &\xrightarrow{s * (A \circ B)} ((S * A) \circ B) * (A \circ B) \\ &\xrightarrow{(\bar{\alpha} \circ B) * (A \circ B)} (A \circ B) * (A \circ B) \xrightarrow{\mu} A \circ B \end{aligned}$$

where $\bar{\alpha} = (S * A \xrightarrow{S * A * \eta^A} S * A * A \xrightarrow{\alpha} A)$.

To transform a scoped operation $\beta : G * B * B \rightarrow B$ to $A \circ B$, we need the following canonical morphism $t : G * (A \circ B) \rightarrow A \circ (G * B)$:

$$\begin{aligned} (G * (A \circ B))n &\cong \int^{m,k} Gm \times A(Bk) \times n^{m \times k} \\ &\rightarrow \int^{m,k} A(Gm \times Bk \times n^{m \times k}) \\ &\rightarrow \int^{m,k} A((G * B)n) \\ &\cong A((G * B)n) \end{aligned}$$

where the first step uses the canonical strength of A to push Gm and $n^{m \times k}$ inwards; the second step uses the coprojection $\iota_{m,k} : Gm \times Bk \times n^{m \times k} \rightarrow (G * B)n$. With t we define the transformed scoped operation on $A \circ B$:

$$\begin{aligned} G * (A \circ B) * (A \circ B) &\xrightarrow{t * (A \circ B)} (A \circ (G * B)) * (A \circ B) \\ &\xrightarrow{(A \circ \bar{\beta}) * (A \circ B)} (A \circ B) * (A \circ B) \\ &\xrightarrow{\mu^{A \circ B}} A \circ B \end{aligned}$$

where $\bar{\beta} = (G * B \xrightarrow{G * B * \eta^B} G * B * B \xrightarrow{\beta} G)$. $\square$

8.7*6. To see the difference between Theorem 8.7*3 and Theorem 8.7*5, let $\dot{A}$ be the applicative functor induced by the exception monad $\bar{E} + \text{Id}$. It is a model of the applicative version of the theory Et_E of exception *throwing*, equipped with an operation *throw* : $\bar{E} * (\bar{E} + \text{Id}) \rightarrow (\bar{E} + \text{Id})$. Using Theorem 8.7*5, it can be extended to a modular model using $(\bar{E} + \text{Id}) \circ B \cong (\bar{E} + B)$ for all applicatives B . In this model, it holds that for all elements $x, y \in (\bar{E} + B)X$,

$$\text{throw } \langle e, x \rangle = \iota_1 e = \text{throw } \langle e, y \rangle,$$

which means that exception throwing discards any B -computation. But it is not true for the independent composition $(\bar{E} + \text{Id}) * B$.

8.7*7. Our final example is an interesting modular model for *phased computation*, generalising the construction that [Kidney and Wu \(2021\)](#) and [Gibbons et al. \(2022\)](#) use for *breadth-first search*. The theory $\text{PhA} \in \text{SCP}(\mathcal{E})$ has a unary scoped operation *later*. The intention is that a program may have multiple phases of execution, and the operation *later* p delays the execution of p to the next phase.

For example, if F is an applicative functor in Haskell with $\text{later} :: F\ a \rightarrow F\ a$, given $p_i :: F\ a$, the following Haskell program of type $F\ a$

```
do later (do later p3
              p21)
  p11
  later    p22
  p12
```

is supposed to execute p_{11} and p_{12} at phase 1, p_{22} and p_{21} at phase 2, and p_3 at phase 3. A standard example of such an applicative functor F is the nested list functor $[[a]]$, where the i -th element of the outer list contains all possible outcomes of the computation at phase i , and $\text{later } xs = [[] : xs]$.

In a symmetric closed monoidal category $\mathcal{E}$ such that every object has a free monoid over it, given a monoid $\dot{A} = \langle A, \mu^A, \eta^A \rangle$, [Kidney and Wu's \[2021\]](#) idea can be abstracted as equipping (the carrier of) the free monoid $S_A = \mu X$. $A \sqcap X + I$ over A with a nonstandard monoid structure $\langle S_A, \mu^{S_A}, \eta^{S_A} \rangle$ with $\eta^{S_A} : I \rightarrow \mu X$. $A \sqcap X + I$ given by $\vdash$ in $(\iota_2 *)$ where $out : (S_A \cong A \sqcap S_A + I) : in$ is the isomorphism for the initial algebra, and μ^{S_A} is denoted by $s : S_A, t : S_A \vdash m : S_A$ where m is

$$\begin{array}{ll} \text{case } (out\ s, out\ t) \text{ of} & \\ (\iota_1\ (a, x), \iota_1\ (a', y)) & \mapsto in\ (\mu^A(a, a'), \mu^{S_A}(x, y)) \\ (\iota_2\ *, \quad y) & \mapsto in\ y \\ (x, \quad \iota_2\ *) & \mapsto in\ x \end{array}$$

Note that the use of variable x and a' in the first case does not match their order in the context, so we need a symmetric monoidal category, and we also need closedness for interpreting structural recursion on the initial algebra S_A . This construction is essentially the same idea as the list object in the category of (ordinary) monoids that we saw in [3.1*14](#). The intuition is that $S_A = \mu X$. $A \sqcap X + I$ is a list of A -computations at each phase, and μ^{S_A} merges two lists by multiplying computations at the same phase. The *later* operation on S_A is defined as

$$p : S_A, k : S_A \vdash \mu^{S_A} (in\ (\iota_1\ (\eta^A, p), k)) : S_A.$$

The construction $A \mapsto S_A$ is a functorial monoid transformer, so we can use [Theorem 8.4*10](#) to obtain a modular model of phasing in $\text{SCP}_I(\mathcal{E})$.

8.7*8. To summarise this section, we have studied modular constructions of algebraic structures in the framework of lifting functors $T : \mathcal{T} \rightarrow \mathcal{T}'$ along two fibrations $P : \mathcal{A} \rightarrow \mathcal{T}$ and $P' : \mathcal{A}' \rightarrow \mathcal{T}'$. The base categories $\mathcal{T}$ and $\mathcal{T}'$ of the fibrations contain some notion of algebraic theories, and the total categories $\mathcal{A}$ and $\mathcal{A}'$ contain models of all these theories. The functor $T : \mathcal{T} \rightarrow \mathcal{T}'$ transforms every theory in a certain way, for example, by combining it with another fixed theory. Liftings of T along P and P' send an object in every fiber category $\mathcal{A}_\Sigma$ to an object in the fiber $\mathcal{A}'_{TP\Sigma}$, so we call them *model transformers*. We can intuitively think of a functor $T : \mathcal{T} \rightarrow \mathcal{T}'$ as a theory $T\Sigma$ parameterised by some potential future extension with $\Sigma \in \mathcal{T}$, then a model transformer M of T can be thought of as a model of the ‘parameterised theory’ T .

We have also seen a handful of universal constructions of model transformers as well as some more concrete constructions, such as using monoid transformers. Lastly, we comment that liftings along fibrations have many other applications in computer science, such as in logical relations for computational types (lifting a computational monad T along a fibration of predicates over sets) ([Katsumata 2005](#)), in Hoare logics (lifting a computation monad T along a fibration of specifications over types) ([Aguirre et al. 2022](#)), and in behaviour metrics of states of automata (lifting a coalgebra encoding an automaton along a fibration of metric spaces over sets) ([Baldan et al. 2014](#)). It is an interesting question for the future to find out whether the lifting techniques developed in these contexts give interesting modular models of algebraic theories.

References

- Samson Abramsky and Achim Jung. 1995. Domain Theory. In *Handbook of Logic in Computer Science*. Oxford University Press/Oxford, 1–168. <https://doi.org/10.1093/oso/9780198537625.003.0001>

- J. Adamek, J. Rosicky, E. M. Vitale, and F. W. Lawvere. 2010. *Algebraic Theories*. Cambridge University Press, Cambridge. 413–418 pages. <https://doi.org/10.1017/CBO9780511760754>
- Jiří Adámek. 1974. Free Algebras and Automata Realizations in the Language of Categories. *Commentationes Mathematicae Universitatis Carolinae* 015, 4 (1974), 589–602. <http://eudml.org/doc/16649>
- Jiří Adámek and Jiří Rosický. 1994. *Locally Presentable and Accessible Categories*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511600579>
- Alejandro Aguirre, Shin-ya Katsumata, and Satoshi Kura. 2022. Weakest preconditions in fibrations. *Mathematical Structures in Computer Science* 32, 4 (2022), 472–510. <https://doi.org/10.1017/S0960129522000330>
- Nathanael Arkor. 2022. *Monadic and Higher-Order Structure*. Ph.D. Dissertation. University of Cambridge. <https://doi.org/10.17863/CAM.86347>
- Nathanael Arkor and Marcelo Fiore. 2020. Algebraic Models of Simple Type Theories: A Polynomial Approach. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science (Saarbrücken, Germany) (LICS '20)*. Association for Computing Machinery, New York, NY, USA, 88–101. <https://doi.org/10.1145/3373718.3394771>
- Robert Atkey. 2009. Parameterised notions of computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376. <https://doi.org/10.1017/S095679680900728X>
- Steve Awodey, Jonas Frey, and Sam Speight. 2018. Impredicative Encodings of (Higher) Inductive Types. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (Oxford, United Kingdom) (LICS '18)*. Association for Computing Machinery, 76–85. <https://doi.org/10.1145/3209108.3209130>
- Casper Bach Poulsen and Cas van der Rest. 2023. Hefty Algebras: Modular Elaboration of Higher-Order Algebraic Effects. *Proc. ACM Program. Lang.* 7, POPL, Article 62 (2023), 31 pages. <https://doi.org/10.1145/3571255>
- Paolo Baldan, Filippo Bonchi, Henning Kerstan, and Barbara König. 2014. Behavioral Metrics via Functor Lifting. In *34th International Conference on Foundation of Software Technology and Theoretical Computer Science (FSTTCS 2014) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 29)*, Venkatesh Raman and S. P. Suresh (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 403–415. <https://doi.org/10.4230/LIPIcs.FSTTCS.2014.403>
- Andrej Bauer and Matija Pretnar. 2015. Programming with algebraic effects and handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015), 108–123. <https://doi.org/10.1016/j.jlamp.2014.02.001>
- Francis Borceux. 1994a. *Handbook of Categorical Algebra: Volume 2, Categories and Structures*. Vol. 2. Cambridge University Press.
- Francis Borceux. 1994b. *Handbook of Categorical Algebra: Volume 3, Sheaf Theory*. Vol. 3. Cambridge University Press.
- Georg Cantor. 1891. Über eine elementare Frage der Mannigfaltigkeitslehre. *Jahresbericht der Deutschen Mathematiker-Vereinigung* 1 (1891), 75–78.
- John Cartmell. 1986. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic* 32 (1986), 209–243. [https://doi.org/10.1016/0168-0072\(86\)90053-9](https://doi.org/10.1016/0168-0072(86)90053-9)
- Luca Castellano, Giorgio De Michelis, and Lucia Pomello. 1987. Concurrency vs interleaving: An instructive example. *Bulletin of the European Association for Theoretical Computer Science (EATCS)* 31 (1987).
- Pietro Cenciarelli and Eugenio Moggi. 1993. *A Syntactic Approach to Modularity in Denotational Semantics*. Technical Report. In *Proceedings of the Conference on Category Theory and Computer Science*. <https://doi.org/10.1.1.41.7807>
- Paul M. Cohn. 1981. *Universal Algebra*. Springer Dordrecht. <https://doi.org/10.1007/978-94-009-8399-1>
- Thierry Coquand and Gérard Huet. 1988. The calculus of constructions. *Information and Computation* 76, 2 (1988), 95–120. [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3)
- Roy L. Crole. 1994. *Categories for Types*. Cambridge University Press.
- Brian Day. 1970. On Closed Categories of Functors. In *Reports of the Midwest Category Seminar IV*, S. Mac Lane, H. Applegate, M. Barr, B. Day, E. Dubuc, Phreilambud, A. Pultr, R. Street, M. Tierney, and S. Swierczkowski (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–38.
- Andrzej Filinski. 1999. Representing Layered Monads. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (San Antonio, Texas, USA) (POPL '99)*. Association for Computing Machinery, New York, NY, USA, 175–188. <https://doi.org/10.1145/292540.292557>
- Marcelo Fiore. 2008. Second-Order and Dependently-Sorted Abstract Syntax. In *Proceedings of the 2008 23rd Annual IEEE Symposium on Logic in Computer Science (LICS '08)*. IEEE Computer Society, 57–68. <https://doi.org/10.1109/LICS.2008.38>

- Marcelo Fiore and Chung-Kil Hur. 2007. Equational Systems and Free Constructions. In *Proceedings of the 34th International Conference on Automata, Languages and Programming* (Wrocław, Poland) (ICALP'07). Springer-Verlag, Berlin, Heidelberg, 607–618. https://doi.org/10.1007/978-3-540-73420-8_53
- Marcelo Fiore and Chung-Kil Hur. 2009. On the Construction of Free Algebras for Equational Systems. *Theoretical Computer Science* 410, 18 (2009), 1704–1729. <https://doi.org/10.1016/j.tcs.2008.12.052>
- Marcelo Fiore and Chung-Kil Hur. 2010. Second-Order Equational Logic. In *Computer Science Logic*, Anuj Dawar and Helmut Veith (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 320–335.
- Marcelo Fiore and Ola Mahmoud. 2010. Second-Order Algebraic Theories. In *Mathematical Foundations of Computer Science 2010*, Petr Hliněný and Antonín Kučera (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 368–380.
- Marcelo Fiore and Ola Mahmoud. 2014. Functorial Semantics of Second-Order Algebraic Theories. <https://doi.org/10.48550/arxiv.1401.4697>
- Marcelo Fiore, Andrew M. Pitts, and S. C. Steenkamp. 2022. Quotients, Inductive Types, & Quotient Inductive Types. *Logical Methods in Computer Science* 18, 2 (2022), 15:1–15:37. [https://doi.org/10.46298/lmcs-18\(2:15\)2022](https://doi.org/10.46298/lmcs-18(2:15)2022)
- Marcelo Fiore, Gordon Plotkin, and Daniele Turi. 1999. Abstract Syntax and Variable Binding. In *Proceedings. 14th Symposium on Logic in Computer Science*. 193–202. <https://doi.org/10.1109/LICS.1999.782615>
- Marcelo Fiore and Philip Saville. 2017. List Objects with Algebraic Structure. In *2nd International Conference on Formal Structures for Computation and Deduction (FSCD 2017) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 84)*, Dale Miller (Ed.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 16:1–16:18. <https://doi.org/10.4230/LIPIcs.FSCD.2017.16>
- Marcelo Fiore and Sam Staton. 2014. Substitution, jumps, and algebraic effects. *Proceedings of the Joint Meeting of the 23rd EACSL Annual Conference on Computer Science Logic, CSL 2014 and the 29th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2014* (2014). <https://doi.org/10.1145/2603088.2603163>
- Marcelo Fiore and Dmitrij Szamozvancev. 2022. Formal Metatheory of Second-Order Abstract Syntax. *Proc. ACM Program. Lang.* 6, POPL, Article 53 (2022), 29 pages. <https://doi.org/10.1145/3498715>
- Peter Freyd. 1964. *Abelian Categories*. Harper & Row, New York. <http://www.tac.mta.ca/tac/reprints/articles/3/tr3.pdf> Reprinted with author's comment as Theory and Applications of Categories Reprints No. 3, 2003.
- Neil Ghani and Tarmo Uustalu. 2004. Coproducts of Ideal Monads. *RAIRO - Theoretical Informatics and Applications* 38, 4 (2004), 321–342. <https://doi.org/10.1051/ita:2004016>
- Neil Ghani, Tarmo Uustalu, and Makoto Hamana. 2006. Explicit Substitutions and Higher-Order Syntax. *Higher-Order and Symbolic Computation* (2006). <https://doi.org/10.1007/s10990-006-8748-4>
- Jeremy Gibbons, Donnacha Oisín Kidney, Tom Schrijvers, and Nicolas Wu. 2022. Breadth-First Traversal via Staging. In *Mathematics of Program Construction*, Ekaterina Komendantskaya (Ed.). Springer International Publishing, Cham, 1–33. https://doi.org/10.1007/978-3-031-16912-0_1
- Andy Gill and Edward Kmett. 2012. mt1: Monad classes, using functional dependencies. <https://hackage.haskell.org/package/mt1>.
- Andrew Gill, John Launchbury, and Simon Peyton Jones. 1993. A Short Cut to Deforestation. In *Proceedings of the Conference on Functional Programming Languages and Computer Architecture* (Copenhagen, Denmark) (FPCA '93). Association for Computing Machinery, New York, NY, USA, 223–232. <https://doi.org/10.1145/165180.165214>
- Jean-Yves Girard. 1986. The System F of variable types, fifteen years later. *Theoretical Computer Science* 45 (1986), 159–192. [https://doi.org/10.1016/0304-3975\(86\)90044-7](https://doi.org/10.1016/0304-3975(86)90044-7)
- J. A. Goguen, J. W. Thatcher, E. G. Wagner, and J. B. Wright. 1977. Initial Algebra Semantics and Continuous Algebras. *J. ACM* 24, 1 (1977), 68–95. <https://doi.org/10.1145/321992.321997>
- Daniel Gratzer. 2023. *Syntax and semantics of modal type theory*. Ph.D. Dissertation. Aarhus University. <https://pure.au.dk/portal/en/publications/syntax-and-semantics-of-modal-type-theory>
- Ralf Hinze. 2012. Kan Extensions for Program Optimisation Or: Art and Dan Explain an Old Trick. In *Mathematics of Program Construction*, Jeremy Gibbons and Pablo Nogueira (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 324–362. https://doi.org/10.1007/978-3-642-31113-0_16
- Ralf Hinze, Thomas Harper, and Daniel W. H. James. 2011. Theory and Practice of Fusion. In *Implementation and Application of Functional Languages*, Jurriaan Hage and Marco T. Morazán (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 19–37. https://doi.org/10.1007/978-3-642-24276-2_2
- John Hughes. 1986. A Novel Representation of Lists and its Application to the Function "reverse". *Inf. Process. Lett.* 22 (1986), 141–144.

- John Hughes. 2000. Generalising Monads to Arrows. *Science of Computer Programming* 37, 1 (2000), 67–111. [https://doi.org/10.1016/S0167-6423\(99\)00023-4](https://doi.org/10.1016/S0167-6423(99)00023-4)
- J.M.E. Hyland. 1988. A small complete category. *Annals of Pure and Applied Logic* 40, 2 (1988), 135–165. [https://doi.org/10.1016/0168-0072\(88\)90018-8](https://doi.org/10.1016/0168-0072(88)90018-8)
- J.M.E. Hyland, Gordon Plotkin, and John Power. 2006. Combining Effects: Sum and Tensor. *Theoretical Computer Science* 357, 1 (2006), 70–99. <https://doi.org/10.1016/j.tcs.2006.03.013>
- Martin Hyland. 2017. Classical Lambda Calculus in Modern Dress. *Mathematical Structures in Computer Science* 27, 5 (2017), 762–781. <https://doi.org/10.1017/S0960129515000377>
- Bart Jacobs. 1999. *Categorical Logic and Type Theory*. Number 141 in Studies in Logic and the Foundations of Mathematics. North Holland, Amsterdam.
- Bart Jacobs, Chris Heunen, and Ichiro Hasuo. 2009. Categorical Semantics for Arrows. *Journal of Functional Programming* 19, 3-4 (2009), 403–438. <https://doi.org/10.1017/S0956796809007308>
- Mauro Jaskielioff. 2009. Modular Monad Transformers. In *Programming Languages and Systems*, Giuseppe Castagna (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 64–79. https://doi.org/10.1007/978-3-642-00590-9_6
- Mauro Jaskielioff and Eugenio Moggi. 2010. Monad Transformers as Monoid Transformers. *Theoretical Computer Science* 411 (2010), 4441–4466. <https://doi.org/10.1016/j.tcs.2010.09.011>
- Niles Johnson and Donald Yau. 2020. 2-Dimensional Categories. arXiv:2002.06055 [math.CT] <https://arxiv.org/abs/2002.06055>
- Ohad Kammar, Sam Lindley, and Nicolas Oury. 2013. Handlers in Action. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming* (Boston, Massachusetts, USA) (ICFP '13). Association for Computing Machinery, New York, NY, USA, 145–158. <https://doi.org/10.1145/2500365.2500590>
- Shin-ya Katsumata. 2005. A Semantic Formulation of $\mathbb{T}\mathbb{T}$ -Lifting and Logical Predicates for Computational Metalanguage. In *Computer Science Logic (Lecture Notes in Computer Science, Vol. 3634)*, Luke Ong (Ed.). Springer Berlin Heidelberg, 87–102. https://doi.org/10.1007/11538363_8
- Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '14). Association for Computing Machinery, New York, NY, USA, 633–645. <https://doi.org/10.1145/2535838.2535846>
- Shin-ya Katsumata, Dylan McDermott, Tarmo Uustalu, and Nicolas Wu. 2022. Flexible Presentations of Graded Monads. *Proc. ACM Program. Lang.* 6, ICFP, Article 123 (2022), 29 pages. <https://doi.org/10.1145/3547654>
- G.M. Kelly. 1980. A unified treatment of transfinite constructions for free algebras, free monoids, colimits, associated sheaves, and so on. *Bulletin of the Australian Mathematical Society* 22, 1 (Aug. 1980), 1–83. <https://doi.org/10.1017/s0004972700006353>
- G.M. Kelly and John Power. 1993. Adjunctions whose counits are coequalizers, and presentations of finitary enriched monads. *Journal of Pure and Applied Algebra* 89, 1 (1993), 163–179. [https://doi.org/10.1016/0022-4049\(93\)90092-8](https://doi.org/10.1016/0022-4049(93)90092-8)
- G. M. Kelly. 1982. Structures defined by finite limits in the enriched context, I. *Cahiers de Topologie et Géométrie Différentielle Catégoriques* 23, 1 (1982), 3–42.
- Donnacha Oisín Kidney and Nicolas Wu. 2021. Algebras for Weighted Search. *Proc. ACM Program. Lang.* 5, ICFP, Article 72 (2021), 30 pages. <https://doi.org/10.1145/3473577>
- Oleg Kiselyov and Hiromi Ishii. 2015. Freer Monads, More Extensible Effects. *SIGPLAN Not.* 50, 12 (2015), 94–105. <https://doi.org/10.1145/2887747.2804319>
- Anders Kock. 1972. Strong Functors and Monoidal Monads. *Archiv der Mathematik* 23 (1972), 113–120. <https://doi.org/10.1007/BF01304852>
- Joachim Lambek. 1958. The Mathematics of Sentence Structure. *The American Mathematical Monthly* 65, 3 (1958), 154–170. <http://www.jstor.org/stable/2310058>
- Joachim Lambek. 1968. A fixpoint theorem for complete categories. *Mathematische Zeitschrift* 103 (1968), 151–161. <https://doi.org/10.1007/BF01110627>
- J. Lambek and P. J. Scott. 1986. *Introduction to higher order categorical logic*. Cambridge University Press.
- Thomas Lamiaux and Benedikt Ahrens. 2024. An Introduction to Different Approaches to Initial Semantics. arXiv:2401.09366 [cs.LO] <https://arxiv.org/abs/2401.09366>
- Sheng Liang, Paul Hudak, and Mark Jones. 1995. Monad Transformers and Modular Interpreters. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '95). ACM, 333–343. <https://doi.org/10.1145/199448.199528>

- Zhaohui Luo. 1994. *Computation and reasoning: a type theory for computer science*. Oxford University Press.
- Saunders Mac Lane. 1998. *Categories for the Working Mathematician* (2nd ed.). Springer, Berlin.
- Saunders MacLane. 1965. Categorical algebra. *Bull. Amer. Math. Soc.* 71, 1 (1965), 40 – 106. <https://doi.org/bams/1183526392>
- Conor McBride and Ross Paterson. 2008. Applicative Programming with Effects. *Journal of Functional Programming* 18, 1 (2008), 1–13. <https://doi.org/10.1017/S0956796807006326>
- Dylan McDermott and Tarmo Uustalu. 2022a. Flexibly Graded Monads and Graded Algebras. In *Mathematics of Program Construction*, Ekaterina Komendantskaya (Ed.). Springer International Publishing, Cham, 102–128. https://doi.org/10.1007/978-3-031-16912-0_4
- Dylan McDermott and Tarmo Uustalu. 2022b. What Makes a Strong Monad? *Electronic Proceedings in Theoretical Computer Science* 360 (2022), 113–133. <https://doi.org/10.4204/EPTCS.360.6>
- Eugenio Moggi. 1989a. *An Abstract View of Programming Languages*. Technical Report ECS-LFCS-90-113. Edinburgh University, Department of Computer Science.
- Eugenio Moggi. 1989b. Computational Lambda-Calculus and Monads. In *Proceedings. Fourth Annual Symposium on Logic in Computer Science*. 14–23. <https://doi.org/10.1109/LICS.1989.39155>
- Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)
- Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à La Carte. *Proc. ACM Program. Lang.* 2, ICFP, Article 79 (2018), 29 pages. <https://doi.org/10.1145/3236774>
- Jaap van Oosten. 2008. *Realizability: an introduction to its categorical side* (1st ed.). Elsevier, Oxford.
- Ross Paterson. 2012. Constructing applicative functors. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 7342 LNCS (2012), 300–323. https://doi.org/10.1007/978-3-642-31113-0_15
- Ruben P. Pieters, Exequiel Rivas, and Tom Schrijvers. 2020. Generalized Monoidal Effects and Handlers. *Journal of Functional Programming* 30 (2020), e23. <https://doi.org/10.1017/S0956796820000106>
- Maciej Piróg. 2016. Eilenberg–Moore Monoids and Backtracking Monad Transformers. *Electronic Proceedings in Theoretical Computer Science* 207 (2016), 23–56. <https://doi.org/10.4204/EPTCS.207.2>
- Maciej Piróg, Tom Schrijvers, Nicolas Wu, and Mauro Jaskielioff. 2018. Syntax and Semantics for Operations with Scopes. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*. Association for Computing Machinery, 809–818. <https://doi.org/10.1145/3209108.3209166>
- Gordon Plotkin and John Power. 2001. Semantics for Algebraic Operations. *Electronic Notes in Theoretical Computer Science* 45 (2001), 332–345. [https://doi.org/10.1016/S1571-0661\(04\)80970-8](https://doi.org/10.1016/S1571-0661(04)80970-8)
- Gordon Plotkin and John Power. 2002. Notions of Computation Determine Monads. In *Foundations of Software Science and Computation Structures, 5th International Conference (FOSSACS 2002)*, Mogens Nielsen and Uffe Engberg (Eds.). Springer, 342–356. https://doi.org/10.1007/3-540-45931-6_24
- Gordon Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Applied Categorical Structures* 11, 1 (2003), 69–94. <https://doi.org/10.1023/A:1023064908962>
- Gordon Plotkin and John Power. 2004. Computational Effects and Operations: An Overview. *Electronic Notes in Theoretical Computer Science* 73 (2004), 149–163. <https://doi.org/10.1016/j.entcs.2004.08.008>
- Gordon Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *Programming Languages and Systems*, Giuseppe Castagna (Ed.). Springer Berlin Heidelberg, 80–94. https://doi.org/10.1007/978-3-642-00590-9_7
- Gordon Plotkin and Matija Pretnar. 2013. Handling Algebraic Effects. *Logical Methods in Computer Science* 9, 4 (2013). [https://doi.org/10.2168/lmcs-9\(4:23\)2013](https://doi.org/10.2168/lmcs-9(4:23)2013)
- John Power. 1999. Enriched Lawvere theories. *Theory and Applications of Categories* 6, 7 (1999), 83–93.
- John C. Reynolds. 1983. Types, Abstraction and Parametric Polymorphism. In *IFIP Congress*.
- Exequiel Rivas and Mauro Jaskielioff. 2017. Notions of Computation as Monoids. *Journal of Functional Programming* 27, September (2017). <https://doi.org/10.1017/S0956796817000132> arXiv:1406.4823
- Tom Schrijvers, Maciej Piróg, Nicolas Wu, and Mauro Jaskielioff. 2019. Monad Transformers and Modular Algebraic Effects: What Binds Them Together. In *Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell, Haskell@ICFP 2019, Berlin, Germany, August 18-23, 2019*. 98–113. <https://doi.org/10.1145/3331545.3342595>
- Mike Spivey. 1990. A functional theory of exceptions. *Science of Computer Programming* 14, 1 (1990), 25–42. [https://doi.org/10.1016/0167-6423\(90\)90056-J](https://doi.org/10.1016/0167-6423(90)90056-J)

- Ian Stark. 2008. Free-algebra models for the π -calculus. *Theoretical Computer Science* 390, 2 (2008), 248–270. <https://doi.org/10.1016/j.tcs.2007.09.024> Foundations of Software Science and Computational Structures.
- Thomas Streicher. 2023. Fibered Categories a la Jean Benabou. arXiv:1801.02927 [math.CT] <https://arxiv.org/abs/1801.02927>
- Wouter Swierstra. 2008. Data types à la carte. *Journal of Functional Programming* 18, 4 (2008), 423–436. <https://doi.org/10.1017/S0956796808006758>
- Birthe van den Berg and Tom Schrijvers. 2024. A framework for higher-order effects & handlers. *Sci. Comput. Program.* 234, C (2024), 32 pages. <https://doi.org/10.1016/j.scico.2024.103086>
- Philip Wadler. 1990. Comprehending Monads. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming* (Nice, France) (*LFP '90*). ACM, 61–78. <https://doi.org/10.1145/91556.91592>
- Nicolas Wu, Tom Schrijvers, and Ralf Hinze. 2014. Effect Handlers in Scope. *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell - Haskell '14* (2014), 1–12. <https://doi.org/10.1145/2633357.2633358>
- Zhixuan Yang, Marco Paviotti, Nicolas Wu, Birthe van den Berg, and Tom Schrijvers. 2022. Structured Handling of Scoped Effects. Springer-Verlag, Berlin, Heidelberg, 462–491. https://doi.org/10.1007/978-3-030-99336-8_17
- Zhixuan Yang and Nicolas Wu. 2021. Reasoning about Effect Interaction by Fusion. *Proc. ACM Program. Lang.* 5, ICFP, Article 73 (2021), 29 pages. <https://doi.org/10.1145/3473578>
- Zhixuan Yang and Nicolas Wu. 2023. Modular Models of Monoids with Operations. *Proc. ACM Program. Lang.* 7, ICFP, Article 208 (2023), 38 pages. <https://doi.org/10.1145/3607850>